A Formal Dynamic Semantics for Newspeak

Design draft 0.14: Machine-checked companion

October 12, 2026

Abstract

This document develops a complete dynamic semantics for Newspeak. "Complete" means that surface conveniences, nested classes, mixin application, all access levels and send forms, initialization, closures and non-local return, actors, eventual references, and causally connected reflective program change must either have rules or appear as explicit outstanding obligations. This draft fixes the semantic domains and the access-controlled method-dispatch kernel, the complete sequential machine, its lifting to isolated actors with asynchronous sends and E-ordered delivery, and the mirror-capability layer for transactional program, object, activation, continuation, and complete actor- stack change. Reified per-frame control, paused actors, atomic stack installation, and ordinary-VM resumption provide the machinery required by an integrated debugger. The reflection rules make VM-cohort propagation and the ordering of installs with active turns and in-flight messages explicit. An optional final layer makes garbage collection, WeakArray, WeakMap, and their observable interaction with exact-instance and mixin-application enumeration explicit without changing the preceding transition rules. A source-front-end layer now defines PEG recognition, AST construction, stable source identities, static elaboration, and a metacircular compilation pipeline whose parser, elaborator, compiler, and Simulator are ordinary reflectively changeable Newspeak objects. A Lean companion formalizes every layer, proves the determinism, preservation, equivalence, fixed-point, transaction, and refinement obligations summarized in the final section, and contains no admitted theorem or additional axiom. The remaining compiler/Simulator work is implementation verification: a concrete installed pair must supply the mandatory artifact-admission and frame-representation certificate already specified here. This draft intentionally does not replace the language specification yet.

Contents

1 Authority, scope, and method **4**

2 Surface and core phrases **4**

3 Semantic domains **5**

4 Class-chain operations **8**
4.1 Public lookup . . . 8
4.2 Protected lookup . . . 8

5 Lexical classes and enclosing receivers **9**

6 Dispatch **10**
6.1 Ordinary sends . . . 10
6.2 Outer and self sends . . . 10
6.3 Super sends . . . 11
6.4 Implicit receiver selection . . . 12
6.5 DNU fallback . . . 13

7 Sequential abstract machine **13**
7.1 Configurations and frames . . . 14
7.2 Expression and message order . . . 14
7.3 From requests to invocation . . . 15
7.4 Closure creation and activation allocation . . . 15
7.5 Local initialization and bodies . . . 17
7.6 Completion and return . . . 18

8 Class construction and initialization **19**
8.1 Normalized class expressions . . . 19
8.2 Mixin application . . . 21
8.3 Message templates and primary factories . . . 21
8.4 Superclass and mixin initialization . . . 22
8.5 Slot groups . . . 24
8.6 Synthesized slot and nested-class methods . . . 25

9 Remaining sequential language **26**
9.1 Atomic and tuple literals . . . 26
9.2 Pattern literals . . . 27
9.3 Object literals . . . 28
9.4 Modules and value status . . . 29
9.5 Top-level programs, compilation units, and entry . . . 30
9.6 The exception-library boundary . . . 31

10 Actors and eventual references **31**
10.1 Global domains and isolation . . . 31
10.2 Actor configurations and causal delivery . . . 33
10.3 Asynchronous-send evaluation and routing . . . 34
10.4 Turns and promise settlement . . . 35
10.5 Actor creation . . . 37

11 Mirrors and reflective program change **37**
11.1 VM cohorts, mirror objects, and authority . . . 37
11.2 Reified control and stack images . . . 39
11.3 Inspection . . . 40
11.4 Reflective transactions . . . 42
11.5 Source change and re-elaboration . . . 46
11.6 Class graphs, object layouts, and individual objects . . . 47
11.7 Activations and continuations . . . 49

11.8 Debugger stack installation and full-speed resumption . . . 49
11.9 Preparation, validation, and atomic commit . . . 52
11.10Causal connection, turns, and other VMs . . . 57

12 Optional garbage-collection layer **58**
12.1 The collectable store . . . 58
12.2 WeakArray and WeakMap . . . 59
12.3 Admissible collection and identity history . . . 60
12.4 Observable consequences . . . 62

13 Source front ends and metacircular compilation **63**
13.1 Source text and PEG recognition . . . 63
13.2 Concrete trees, metadata, and stable AST identities . . . 71
13.3 Binding, elaboration, and static well-formedness . . . 78
13.4 Reified language services . . . 86
13.5 Ordinary change, phase behavior, and bootstrap . . . 87

14 Established invariants and mechanization status **88**
14.1 Machine-checked status . . . 90
14.2 Auditable relational bridge . . . 91

1 Authority, scope, and method

The normative source is working version 0.109 of the NEWSPEAK Programming Language Draft Specification (henceforth *the Specification*) [Bra26]. Section 3.3 of *Modules as Objects in Newspeak* [BvdAB+10] is explanatory prior art. If the sources disagree, the Specification governs.

The surface language is parsed by the PEG in Specification §4–§7. Section 13 gives that PEG a recognition semantics, maps its concrete trees to stable AST identities, and defines the static elaboration judgment that rejects ill-formed uses of self, outer, super, returns, and class names.

The semantics is a small-step transition system. This choice makes evaluation order, abrupt control transfer, actors, and transactions visible without encoding them as host-language accidents. Metalevel records below are not a claim that runtime entities are inaccessible: Section 11 reifies them through mirror capabilities and preserves that causal connection.

Base/reflective separation. The base relation holds the program definition P fixed. In a machine configuration, P is the static program, H is the heap, and $\mathcal{A}$ is the activation stack. The latter includes the active activations and their continuation links. At the sequential boundary a reflective step has the schematic form

$$\langle P, H, \mathcal{A}\rangle \xrightarrow{\mathsf{reflectiveStep}(\Delta)} \langle P', H', \mathcal{A}'\rangle, \tag{1.1}$$

where validation and commit are atomic; Section 11 refines this schema for the actor/VM state. This separation is methodological, not semantic simplification: the base domains already carry stable declaration, mixin, class, object, and activation identities needed by reflection.

2 Surface and core phrases

Let x range over identifiers, s over selector symbols, and $\bar{e}$ over ordered expression lists. The dispatch-relevant core expressions are

$$\begin{aligned} e \;::=\;& v \mid \mathsf{self} \mid \mathsf{atom}(\omega) \mid \mathsf{tuple}(\bar{e}) \mid \mathsf{pattern}(p, d) \\ \mid\;& \mathsf{objectLiteral}(\chi_o, e_S) \mid \mathsf{closureLiteral}(q, \bar{x}, \ell, b) \\ \mid\;& \mathsf{send}(e, s, \bar{e}) \mid \mathsf{implicit}(s, \bar{e}, d) \\ \mid\;& \mathsf{selfSend}(s, \bar{e}, D) \mid \mathsf{outerSend}(s, \bar{e}, D_t, D_0) \\ \mid\;& \mathsf{superSend}(s, \bar{e}) \mid \mathsf{asyncSend}(e, s, \bar{e}) \mid \cdots \end{aligned} \tag{2.1}$$

There is deliberately no variable-reference case x. A standalone identifier x is a unary message clause with an implicit receiver and elaborates to $\mathsf{implicit}(\#x, \epsilon, d)$, where d is the lexical declaration selected for that send. This remains true when d declares a parameter or local slot; the resulting send is directed to the appropriate activation. The metavariable x is retained only for binding occurrences such as formal parameters and slot declarations.

Here d records the innermost lexical declaration selected for an implicit send, if any; D_0 is the immediately enclosing class declaration and D_t is the named lexical target of an outer send. These annotations are computed from the AST, not guessed dynamically. Retaining distinct send nodes is essential: ordinary, self, outer, and super sends grant different authority.

A super send deliberately carries no declaration parameter. Its receiver is the current instance, and its lookup start is the superclass of the current activation's runtime current class. That current class is installed by method dispatch and captured by closures. A lexical declaration identity would not identify the particular runtime application of its mixin from which super lookup must continue; using it for that purpose would be incorrect when one declaration has multiple applications.

Elaboration may remove syntax whose specified expansion preserves all observable behavior. For example, a cascade evaluates by the specified closure expansion, which evaluates its receiver exactly once. Elaboration must not turn a self send into an ordinary send, including when self is parenthesized.

Slot declarations and nested-class declarations induce methods before runtime evaluation:

- an immutable slot induces a getter with the slot's access;
- a mutable slot induces a getter and a one-argument setter, both with the slot's access; the setter returns its receiver;
- a lazy slot induces a getter with the slot's access and is physically initialized to nil. On a read, the getter returns the stored value if it is non-nil; otherwise it evaluates the slot initializer, writes the result directly into the slot, and returns it. The ::= form also induces the same-access setter described above, whereas the = form induces no setter;
- a nested class induces a memoizing getter with the declaration's access.

Initialization writes slots directly and therefore does not invoke or dispatch through these synthesized setters. Because nil is the lazy-state sentinel, an initializer that returns nil is evaluated again on the next read. Resetting a lazy slot to nil for orthogonal synchronization will be modeled as reflective change rather than ordinary initialization or mutation.

3 Semantic domains

We assume pairwise disjoint, countably infinite metalevel identity sets

$$\begin{aligned} &o \in \mathit{ObjId}, \qquad C \in \mathit{ClassId}, \quad M \in \mathit{MixinId}, \quad a \in \mathit{ActId}, \\ &c \in \mathit{ClosureId}, \quad g \in \mathit{MirrorId}, \quad A \in \mathit{ActorId}. \end{aligned} \tag{3.1}$$

Their tagged union is *ObjRef*. Every member of *ObjRef* is a Newspeak object and, when live, has a class. The disjoint tags prevent a semantic rule from confusing (for example) the identity of a class object with the identity of one of its ordinary instances; they do not introduce non-object runtime values. Pause tokens $r_s \in \mathit{PauseToken}$ are drawn from a separate countably infinite set. They are unforgeable protocol tokens rather than Newspeak objects: entering a paused actor state allocates a fresh token, and every successful paused-stack edit allocates another. A stale debugger request therefore cannot mutate a stack that has since been edited or resumed. Continuation links are not a separate identity domain: throughout, $k \in \mathit{ActId} \uplus \{\mathsf{nil}\}$. In accordance with the Specification, a non-nil continuation object is the sender activation at which execution will resume. Mirror identities are included in *ObjRef*; their unforgeable heap records are introduced in Section 11. Source declarations also have stable AST identities. We write $d \in \mathit{DeclId}$ for an arbitrary declaration, $K \in \mathit{ClassBodyDeclId} \subseteq \mathit{DeclId}$ for a declaration that induces a mixin, with the disjoint partition $\mathit{ClassBodyDeclId} = \mathit{ClassDeclId} \uplus \mathit{ObjLitDeclId}$. Write $D \in \mathit{ClassDeclId}$ for a named class declaration, $L \in \mathit{ObjLitDeclId}$ for an object-literal declaration, and $q \in \mathit{ActDeclId} \subseteq \mathit{DeclId}$ for a method or closure declaration. The domains *ClassBodyDeclId* and *ActDeclId* are disjoint. Finally, $z \in \mathit{SiteId}$ is an expression site and $i_f \in \mathit{MethodId} \subseteq \mathit{ActDeclId}$ is a method declaration. A class-declaration identity is not a class name: evaluation of one declaration may produce many distinct runtime classes. Class creation provenance is the tagged domain

$$\chi_C \in \mathit{ClassOrigin} ::= \mathsf{expression}(K) \mid \mathsf{actor}(K). \tag{3.2}$$

The tag distinguishes evaluation of a source expression from application of a top-level mixin while seeding a new actor. Both retain the inducing source declaration for reflection.

An access level is $\alpha \in \{\mathsf{public}, \mathsf{protected}, \mathsf{private}\}$. Absence of an access modifier elaborates to $\mathsf{protected}$; a top-level class declaration is always public.

A message is $\mu = \langle s, \bar{o} \rangle$. A method definition is

$$f = \langle i_f, s, \alpha, \bar{x}, \ell, b, K \rangle, \tag{3.3}$$

where ℓ are local-slot declarations, b is the body, and K is the lexically enclosing class or object-literal declaration. The identity i_f names the method declaration independently of its selector and current body. This lets *P.body* and, later, method mirrors refer to the same declaration when its body is reflectively replaced. The reflective rules will specify when an edit preserves i_f and when deletion followed by addition allocates a fresh identity. Elaboration assigns each synthesized getter, setter, or nested-class getter a method identity derived from its inducing declaration and selector. Each mixin contains at most one direct method per selector after synthesized members have been installed.

The program definition P contains immutable source structure:

$$\begin{aligned}
P.source &: DeclId \rightharpoonup AST, \\
P.unit &: AST, \\
P.declMixin &: ClassBodyDeclId \to MixinId, \\
P.mixin &: M \rightharpoonup \langle K, methodMap, slotDecls, nestedDecls, initializer \rangle, \\
P.lexParent &: D \rightharpoonup D, \\
P.body &: MethodId \to AST, \\
P.scopeChain &: SiteId \to DeclId^*, \\
P.activationMembers &: ActDeclId \to \mathcal{P}_{\mathrm{fin}}(Selector), \\
P.owner &: ClassBodyDeclId \to \left\{ \begin{array}{l} \mathsf{activationOwner}, \mathsf{classOwner}, \\ \mathsf{objectLiteralOwner}, \mathsf{topOwner} \end{array} \right\}, \\
P.actorSeed &: MixinId \rightharpoonup \langle ClassBodyDeclId, MixinId, FactoryDescriptor \rangle, \\
P.atom &: Atom \to ObjRef, \\
P.platform &: PlatformName \rightharpoonup ObjRef, \\
P.patternVariable &: SiteId \rightharpoonup AST.
\end{aligned} \tag{3.4}$$

In the base relation these maps are immutable; reflective installation replaces the complete P atomically. The map *P.declMixin* is injective and the first component of $P.mixin(P.declMixin(K))$ is K. Every subsequent occurrence of a field such as *P.mixin*, *P.scopeChain*, *P.owner*, or *P.actorSeed* refers to the maps collected in Equation (3.4). More precisely, if

$$P.mixin(M) = \langle K, \phi, \delta, \nu, \iota \rangle, \tag{3.5}$$

then $\mathrm{methods}_P(M) = \phi$, where $\phi : Selector \rightharpoonup MethodDef$ is the finite map of methods defined directly by M, including methods introduced by elaboration. The remaining components are the slot declarations δ, nested-class declarations ν, and normalized mixin initializer ι, whose form is

$$\iota = \langle i_I, s_I, \bar{x}_I, \bar{g}, b_I \rangle. \tag{3.6}$$

Here i_I is its stable synthetic method identity, s_I and $\bar{x}_I$ are its selector and formals, $\bar{g}$ is an ordered list of sequential or simultaneous slot groups, and b_I is its initializer body. For an f having the form in Equation (3.3), the projections are

$$\begin{aligned}
\iota_P(M) = \iota, \quad & \mathsf{methodId}(f) = i_f, \quad \mathrm{selector}(f) = s, \\
\mathrm{access}(f) = \alpha, \quad & \mathsf{ownerDecl}(f) = K.
\end{aligned} \tag{3.7}$$

For the mixin record in Equation (3.5), δ is the static, source-ordered sequence of identified physical slot declarations, including the backing declarations of lazy slots. Write d_j for a member whose stable declaration identity is j. Define the ordered projection

$$\begin{aligned} \mathsf{ownSlots}_P(M) &= \langle j \mid d_j \in \delta \rangle, \\ P.mixin(M) &= \langle K, \phi, \delta, \nu, \iota \rangle. \end{aligned} \tag{3.8}$$

The comprehension preserves the order of δ and returns ϵ when δ is empty. Neither δ nor j denotes runtime storage: they are shared static metadata for M in the current program P. Equation (8.16) separately qualifies each j by its runtime application C. The groups in ι refer to the non-lazy members of Equation (3.8); lazy backing slots have no eager initializer group. The front end derives δ by Equation (13.74); reflective installation may replace it by atomically replacing P.

The heap H is a finite map from the identities in Equation (3.1) to tagged records. A factory descriptor is

$$q = \langle i_I, s_f, \bar{x}, \kappa \rangle, \qquad \kappa ::= \mathsf{bodyInitializer}(m_S) \mid \mathsf{appliedMixinInitializer}(m_S, m_M), \tag{3.9}$$

where i_I identifies the class's synthesized instance initializer, s_f and $\bar{x}$ describe its primary factory, m_S is the deferred superclass initializer message, and m_M is the deferred message for an applied mixin. The records relevant to the base sequential semantics are:

$$\begin{aligned} H(o) &= \mathsf{object}\langle C, \sigma, \eta \rangle, \\ H(C) &= \mathsf{class}\langle M, S, eo, C_m, q, \chi_C \rangle, \\ H(a) &= \mathsf{activation}\langle p, \rho, \ell, o, C, k, h, q_a, \gamma \rangle, \\ H(c) &= \mathsf{closure}\langle q, \bar{x}, \ell, b, a, o, C, h \rangle. \end{aligned} \tag{3.10}$$

σ maps application-qualified physical slot identities (C, j) to values, and η maps application-qualified nested-class declaration identities to the class objects already created for this receiver. A class record contains its mixin M, superclass S, enclosing object eo, metaclass C_m, primary factory description q, and stable creation provenance χ_C from Equation (3.2); $q = \mathsf{none}$ only for a non-instantiable platform class such as a metaclass. An activation records its method/closure provenance p, parameter environment ρ, local slots ℓ, current instance o, current class C, continuation object k, home method activation h, continuability flag q_a, and lexical environment γ. The latter maps an enclosing method or closure declaration to its current activation and an enclosing object-literal declaration to that literal's runtime value; write $\mathsf{scope}_H(a) = \gamma$. A closure captures its source declaration q, the instantiating activation, current instance, current class, and home method activation. In particular, $\mathrm{classOf}_H(C) = C_m$ for a class object. The standard platform classes of mixins, activations, closures, and continuations supply the corresponding cases of $\mathrm{classOf}_H$; their identities are left as platform parameters until their reflective protocols are formalized.

We write $\mathrm{classOf}_H(o) = C$, $\mathrm{mixin}_H(C) = M$, $\mathrm{super}_H(C) = S$, and $\mathrm{decl}_P(M) = D$ for the corresponding projections of Equations (3.5) and (3.10). The distinguished class Top terminates lookup and is not an instantiable Newspeak class. In the non-reflective semantics, every live class chain is finite and ends at Top. The distinguished names used throughout abbreviate platform entries; in particular,

$$P.platform(\mathsf{Object}) = \mathsf{Object}, \qquad P.platform(\mathsf{Top}) = \mathsf{Top}. \tag{3.11}$$

4 Class-chain operations

Direct method selection, using the mixin map in Equation (3.5), is

$$\mathsf{direct}_{P,H}(C, s) = \begin{cases} f & \text{methods}_P(\text{mixin}_H(C))(s) = f, \\ \uparrow & \text{otherwise.} \end{cases} \tag{4.1}$$

Unrestricted definition lookup follows the superclass chain and ignores accessibility. Its named rules use direct selection from Equation (4.1):

$$\frac{C = \mathsf{Top}}{P, H \vdash \text{lookup}_U(s, C) = \uparrow} \quad \text{(U-Top)}$$

$$\frac{\mathsf{direct}(C, s) = f}{P, H \vdash \text{lookup}_U(s, C) = \langle f, C \rangle} \quad \text{(U-Here)}$$

$$\frac{C \neq \mathsf{Top} \quad \mathsf{direct}(C, s) = \uparrow \quad P, H \vdash \text{lookup}_U(s, \text{super}_H(C)) = r}{P, H \vdash \text{lookup}_U(s, C) = r} \quad \text{(U-Super)}$$

The defining class is independently defined by the following partial function:

$$\frac{C = \mathsf{Top}}{P, H \vdash \text{definingClass}(s, C) = \uparrow} \quad \text{(Defining-Top)}$$

$$\frac{\mathsf{direct}(C, s) = f}{P, H \vdash \text{definingClass}(s, C) = C} \quad \text{(Defining-Here)}$$

$$\frac{C \neq \mathsf{Top} \quad \mathsf{direct}(C, s) = \uparrow \quad P, H \vdash \text{definingClass}(s, \text{super}_H(C)) = C_f}{P, H \vdash \text{definingClass}(s, C) = C_f} \quad \text{(Defining-Super)}$$

It follows directly from these rules and the unrestricted-lookup rules that $P, H \vdash \text{lookup}_U(s, C) = \langle f, C_f \rangle$ implies $P, H \vdash \text{definingClass}(s, C) = C_f$.

4.1 Public lookup

Public lookup implements ordinary sends. A protected direct declaration is a *barrier*: it makes lookup fail rather than exposing a public superclass method. A private direct declaration is not inherited and is transparent to this lookup. Every occurrence of direct below denotes Equation (4.1).

$$\frac{C = \mathsf{Top}}{P, H \vdash \text{lookup}_{pub}(s, C) = \uparrow} \quad \text{(Pub-Top)}$$

$$\frac{\mathsf{direct}(C, s) = f \quad \text{access}(f) = \mathsf{public}}{P, H \vdash \text{lookup}_{pub}(s, C) = \langle f, C \rangle} \quad \text{(Pub-Hit)}$$

$$\frac{\mathsf{direct}(C, s) = f \quad \text{access}(f) = \mathsf{protected}}{P, H \vdash \text{lookup}_{pub}(s, C) = \uparrow} \quad \text{(Pub-Protected-Barrier)}$$

$$\frac{C \neq \mathsf{Top} \quad (\mathsf{direct}(C,s) = \uparrow \ \vee \ \text{access}(\mathsf{direct}(C,s)) = \mathsf{private}) \quad P,H \vdash \text{lookup}_{pub}(s, \text{super}_H(C)) = r}{P, H \vdash \text{lookup}_{pub}(s, C) = r} \quad \text{(Pub-Super)}$$

4.2 Protected lookup

Protected lookup implements the virtual part of self, outer, and super sends. Both public and protected methods participate; private declarations remain lexical and therefore are transparent.

These rules likewise use direct selection from Equation (4.1).

$$\frac{C = \mathsf{Top}}{P, H \vdash \mathrm{lookup}_{prot}(s, C) = \uparrow} \quad \text{(PROT-TOP)}$$

$$\frac{\mathsf{direct}(C, s) = f \quad \mathrm{access}(f) \in \{\mathsf{public}, \mathsf{protected}\}}{P, H \vdash \mathrm{lookup}_{prot}(s, C) = \langle f, C \rangle} \quad \text{(PROT-HIT)}$$

$$\frac{C \neq \mathsf{Top} \quad (\mathsf{direct}(C,s) = \uparrow \vee \mathrm{access}(\mathsf{direct}(C,s)) = \mathsf{private}) \quad P, H \vdash \mathrm{lookup}_{prot}(s, \mathrm{super}_H(C)) = r}{P, H \vdash \mathrm{lookup}_{prot}(s, C) = r} \quad \text{(PROT-SUPER)}$$

5 Lexical classes and enclosing receivers

For a well-formed heap H, define the finite superclass-chain sequence by primitive recursion:

$$\begin{aligned} \mathsf{chain}_H(\mathsf{Top}) &= \langle \mathsf{Top} \rangle, \\ \mathsf{chain}_H(R) &= \langle R \rangle \cdot \mathsf{chain}_H(\operatorname*{super}_H(R)) \quad \text{if } R \neq \mathsf{Top}. \end{aligned} \tag{5.1}$$

Here $\cdot$ is sequence concatenation and indices are zero-based. The base invariant that superclass links are acyclic and reach Top makes this definition total and gives a unique finite sequence for every live class. Superclass membership and depth are then, from the chain in Equation (5.1),

$$\begin{aligned} C \preceq_H R \iff \exists j \,.\, 0 \leq j < |\mathsf{chain}_H(R)| \ \wedge \ \mathsf{chain}_H(R)[j] = C, \\ \mathsf{depth}_H(C, R) = \min\{j \mid \mathsf{chain}_H(R)[j] = C\}, \end{aligned} \tag{5.2}$$

where depth is undefined when $C \not\preceq_H R$.

The lexical-class chain is the corresponding static construction. Define the partial function

$$\begin{aligned} \mathsf{lexAncestor}_P(D, 0) &= D, \\ \mathsf{lexAncestor}_P(D, n+1) &= P.\mathit{lexParent}(\mathsf{lexAncestor}_P(D, n)), \end{aligned} \tag{5.3}$$

where the second clause is undefined if either operand on its right is undefined. Using Equation (5.3), define for a lexical ancestor D_t of D_0

$$\mathsf{lexDepth}_P(D_t, D_0) = \min\{k \mid \mathsf{lexAncestor}_P(D_0, k) = D_t\}. \tag{5.4}$$

For a declaration D, let the indices of its applications in R's chain from Equation (5.1) be

$$\mathsf{appIndices}_{P,H}(R, D) = \left\{ j \;\middle|\; \begin{array}{l} 0 \leq j < |\mathsf{chain}_H(R)|, \quad \mathsf{chain}_H(R)[j] \neq \mathsf{Top}, \\ \mathrm{decl}_P(\mathrm{mixin}_H(\mathsf{chain}_H(R)[j])) = D \end{array} \right\}. \tag{5.5}$$

Using Equation (5.5), the nearest application is the partial function

$$\mathsf{nearestApp}_{P,H}(R, D) = \begin{cases} \mathsf{chain}_H(R)[\min I] & I = \mathsf{appIndices}_{P,H}(R, D) \neq \varnothing, \\ \uparrow & I = \varnothing. \end{cases} \tag{5.6}$$

Minimal index means most specific. This is significant when the same mixin occurs repeatedly in one hierarchy.

For a runtime receiver o_t and lexical target declaration D_t, define the target application from Equation (5.6):

$$\mathsf{targetApp}_{P,H}(o_t, D_t) = \mathsf{nearestApp}_{P,H}(\operatorname*{class\mathrm{Of}}_H(o_t), D_t). \tag{5.7}$$

Well-formed outer sends preserve the invariant that this application exists.

The partial relation $P, H \vdash \mathsf{enc}(o, C, n) = o_n$ formalizes the Specification's nth enclosing object of o with respect to C. Its premises use superclass membership from Equation (5.2), lexical ancestry from Equation (5.3), and nearest application from Equation (5.6):

$$\frac{C \preceq_H \mathrm{classOf}_H(o)}{P, H \vdash \mathsf{enc}(o, C, 0) = o} \quad \text{(Enc-Zero)}$$

$$\frac{C \preceq_H \mathrm{classOf}_H(o) \quad H(C).eo = o_1}{P, H \vdash \mathsf{enc}(o, C, 1) = o_1} \quad \text{(Enc-One)}$$

$$\frac{\begin{array}{c} n>1 \quad P,H \vdash \mathsf{enc}(o,C,n-1)=o_{n-1} \quad D_{n-1}=\mathsf{lexAncestor}_P(\mathrm{decl}_P(\mathrm{mixin}_H(C)),n-1) \\ S_{n-1}=\mathsf{nearestApp}(\mathrm{classOf}_H(o_{n-1}),D_{n-1}) \quad H(S_{n-1}).eo=o_n \end{array}}{P, H \vdash \mathsf{enc}(o, C, n) = o_n} \quad \text{(Enc-Step)}$$

The separate $n = 1$ rule is intentional. It preserves the Specification's behavior under repeated mixin application rather than recomputing a possibly different application from the receiver's dynamic class.

6 Dispatch

A dispatch result is $\mathsf{invoke}(f, o, C_f, \mu)$, $\mathsf{missing}(o, C_0, \mu)$, or $\mathsf{runtimeError}(\mathsf{invalidSuper}(C))$. C_f is the current class to install in the new method activation; C_0 is the class at which DNU lookup must begin. The third form is an internal failure, deliberately not a DNU request; Section 9.6 reifies it as a platform exception and signals it through ordinary dispatch. For compact rules, $\mathsf{outerDispatch}_a$ denotes outer dispatch in activation a.

6.1 Ordinary sends

After receiver and arguments have evaluated left-to-right to o and $\bar{o}$, let $\mu = \langle s, \bar{o} \rangle$ and $R = \mathrm{classOf}_H(o)$. Dispatch uses the public lookup of Section 4.1:

$$\frac{P, H \vdash \mathrm{lookup}_{pub}(s, R) = \langle f, C_f \rangle}{P, H \vdash \mathsf{ordinaryDispatch}(o, \mu) \Downarrow \mathsf{invoke}(f, o, C_f, \mu)} \quad \text{(Send-Public)}$$

$$\frac{P, H \vdash \mathrm{lookup}_{pub}(s, R) = \uparrow}{P, H \vdash \mathsf{ordinaryDispatch}(o, \mu) \Downarrow \mathsf{missing}(o, R, \mu)} \quad \text{(Send-Missing)}$$

6.2 Outer and self sends

Private selection is a static operation on the program. Define

$$\mathsf{privateMethod}_P(D, s) = \begin{cases} f & \text{if there exists } M \in \mathrm{dom}(P.\mathit{mixin}) \text{ such that} \\ & \mathrm{decl}_P(M) = D, \quad \mathrm{methods}_P(M)(s) = f, \quad \mathrm{access}(f) = \mathsf{private}, \\ \uparrow & \text{otherwise.} \end{cases} \tag{6.1}$$

Well-formedness gives each class declaration one static mixin identity, and each mixin at most one direct method per selector, so the first case determines a unique f. Thus this operation selects only a private method declared directly by D; it does not search a runtime superclass chain. In particular, a direct public or protected method of selector s makes $\mathsf{privateMethod}_P(D, s) = \uparrow$.

Suppose the current activation has current instance o_0 and current class C_0. If the immediately enclosing class declaration is D_0 and target D_t is its kth lexical class ancestor, obtain o_t from

$\mathsf{enc}(o_0, C_0, k)$. Let N_t be the corresponding class $N_t = \mathsf{targetApp}(o_t, D_t)$ from Equation (5.7), and let $\mu = \langle s, \bar{o} \rangle$.

If Equation (6.1) selects a private method f directly declared by D_t, dispatch is lexically bound:

$$\frac{P, H \vdash \mathsf{enc}(o_0, C_0, k) = o_t \quad \mathsf{privateMethod}_P(D_t, s) = f}{P, H \vdash \mathsf{outerDispatch}_a(D_t, D_0, \mu) \Downarrow \mathsf{invoke}(f, o_t, N_t, \mu)} \quad \text{(Outer-Private)}$$

Otherwise dispatch remains virtual and sees public or protected methods via Section 4.2:

$$\frac{P,H \vdash \mathsf{enc}(o_0,C_0,k)=o_t \quad \mathsf{privateMethod}_P(D_t,s)=\uparrow \quad R=\mathrm{classOf}_H(o_t) \quad P,H \vdash \mathrm{lookup}_{prot}(s,R)=\langle f,C_f \rangle}{P, H \vdash \mathsf{outerDispatch}_a(D_t, D_0, \mu) \Downarrow \mathsf{invoke}(f, o_t, C_f, \mu)} \quad \text{(Outer-Protected)}$$

A failed protected lookup is explicit:

$$\frac{P,H \vdash \mathsf{enc}(o_0,C_0,k)=o_t \quad \mathsf{privateMethod}_P(D_t,s)=\uparrow \quad R=\mathrm{classOf}_H(o_t) \quad P,H \vdash \mathrm{lookup}_{prot}(s,R)=\uparrow}{P, H \vdash \mathsf{outerDispatch}_a(D_t, D_0, \mu) \Downarrow \mathsf{missing}(o_t, R, \mu)} \quad \text{(Outer-Missing)}$$

A self send lexically enclosed immediately by D_0 is the depth-zero case

$$\mathsf{selfDispatch}_a(\mu) \equiv \mathsf{outerDispatch}_a(D_0, D_0, \mu). \tag{6.2}$$

It is therefore not an ordinary send: it can select a private method declared in D_0 and can dynamically select protected methods.

Clarification obligation. The N_t current class in Outer-Private is the actual class application on the enclosing path, not merely the class found by unrestricted lookup of s on o_t. Otherwise a more-derived declaration of the same selector could cause a statically selected private method to execute with the wrong current class. The Specification appears to require this result but does not state it directly.

6.3 Super sends

For current instance o, current class $C \notin \{\mathsf{Object}, \mathsf{Top}\}$, $S = \mathrm{super}_H(C)$, and $\mu = \langle s, \bar{o} \rangle$, dispatch uses the protected lookup of Section 4.2:

$$\frac{P, H \vdash \mathrm{lookup}_{prot}(s, S) = \langle f, C_f \rangle}{P, H, a \vdash \mathsf{superDispatch}(\mu) \Downarrow \mathsf{invoke}(f, o, C_f, \mu)} \quad \text{(Super-Hit)}$$

$$\frac{P, H \vdash \mathrm{lookup}_{prot}(s, S) = \uparrow}{P, H, a \vdash \mathsf{superDispatch}(\mu) \Downarrow \mathsf{missing}(o, S, \mu)} \quad \text{(Super-Missing)}$$

The missing result is processed by the DNU fallback of Section 6.5; in particular, S is the class at which DNU lookup begins.

The two excluded current classes have explicit error rules:

$$\frac{C = \mathsf{Object}}{P, H, a \vdash \mathsf{superDispatch}(\mu) \Downarrow \mathsf{runtimeError}(\mathsf{invalidSuper}(\mathsf{Object}))} \quad \text{(Super-Object-Error)}$$

$$\frac{C = \mathsf{Top}}{P, H, a \vdash \mathsf{superDispatch}(\mu) \Downarrow \mathsf{runtimeError}(\mathsf{invalidSuper}(\mathsf{Top}))} \quad \text{(Super-Top-Error)}$$

Neither error proceeds to DNU fallback.

6.4 Implicit receiver selection

Lexical resolution precedes dynamic inheritance. If

$$P.scopeChain(z) = \langle d_0, \ldots, d_n \rangle, \tag{6.3}$$

then the declarations are ordered from the construct immediately surrounding site z outwards. The static predicate $\mathsf{declares}_P(d, s)$ is

$$\begin{array}{rcl} \mathsf{declares}_P(K, s) & \iff & \exists M \in \mathrm{dom}(P.mixin)\,.\,\mathrm{decl}_P(M) = K \wedge s \in \mathrm{dom}(\mathrm{methods}_P(M)), \\ \mathsf{declares}_P(q, s) & \iff & s \in P.activationMembers(q). \end{array} \tag{6.4}$$

The method maps include accessors synthesized for class or object-literal slots and nested classes; activation members include accessors for parameters and locals. Using the scope sequence in Equation (6.3) and the predicate in Equation (6.4), define

$$\begin{gathered} J_{P,z,s} = \{ j \mid 0 \le j \le n \wedge \mathsf{declares}_P(d_j, s) \}, \\ \mathsf{bind}_P(z, s) = \begin{cases} d_{\min J_{P,z,s}} & J_{P,z,s} \neq \varnothing, \\ \uparrow & J_{P,z,s} = \varnothing. \end{cases} \end{gathered} \tag{6.5}$$

Thus bind selects the innermost lexical declaration, independently of access level. Elaboration annotates the core send with $d = \mathsf{bind}_P(z, s)$ as defined by Equation (6.5). It also retains the immediately enclosing class declaration D_0 as static context. This context is written on the judgment rather than repeated inside the core node below.

Suppose the message clause has evaluated to $\mu = \langle s, \bar{o} \rangle$ and the current activation is a. The judgment

$$P, H, a, D_0 \vdash \mathsf{implicitDispatch}(d, \mu) \Downarrow r \tag{6.6}$$

is defined by the following four rules, where r ranges over dispatch results. For a class declaration, implicit dispatch is exactly outer dispatch:

$$\frac{d = D_t \in \mathit{ClassDeclId} \quad P, H \vdash \mathsf{outerDispatch}_a(D_t, D_0, \mu) \Downarrow r}{P, H, a, D_0 \vdash \mathsf{implicitDispatch}(d, \mu) \Downarrow r} \quad \textsc{(Implicit-Class)}$$

For an enclosing object literal, its lexical environment supplies the literal value and the statically selected direct method is invoked without an access check:

$$\frac{\begin{array}{c} d = L \in \mathit{ObjLitDeclId} \\ \mathsf{scope}_H(a)(L) = o_L \\ P, H \vdash \mathrm{lookup}_U(s, \mathrm{classOf}_H(o_L)) = \langle f, C_f \rangle \\ \mathrm{decl}_P(\mathrm{mixin}_H(C_f)) = L \end{array}}{P, H, a, D_0 \vdash \mathsf{implicitDispatch}(d, \mu) \Downarrow \mathsf{invoke}(f, o_L, C_f, \mu)} \quad \textsc{(Implicit-Literal)}$$

For a method or closure declaration, the lexical environment supplies that declaration's current activation, and the specified send to the activation is an ordinary send:

$$\frac{\begin{array}{c} d = q \in \mathit{ActDeclId} \\ \mathsf{scope}_H(a)(q) = a_q \\ P, H \vdash \mathsf{ordinaryDispatch}(a_q, \mu) \Downarrow r \end{array}}{P, H, a, D_0 \vdash \mathsf{implicitDispatch}(d, \mu) \Downarrow r} \quad \textsc{(Implicit-Activation)}$$

Finally, failure of lexical binding is a self send:

$$\frac{d = \uparrow \quad P, H \vdash \mathsf{outerDispatch}_a(D_0, D_0, \mu) \Downarrow r}{P, H, a, D_0 \vdash \mathsf{implicitDispatch}(d, \mu) \Downarrow r} \quad \textsc{(Implicit-Self)}$$

These rules are mutually exclusive because the declaration-identity domains are disjoint. Their premises are total for a well-formed annotation. A reflective edit that changes a scope chain or member map must rebuild the affected bind annotations and lexical environments in the same atomic install transaction.

ECOOP-style derived presentation. After nearer activation and object-literal scopes have failed to declare s, the class-only part of the search can instead be written recursively using Equation (6.4):

$$\mathsf{scan}_P(D,s) = \begin{cases} D & \mathsf{declares}_P(D,s), \\ \mathsf{scan}_P(D',s) & \neg\mathsf{declares}_P(D,s) \ \wedge \ P.\mathit{lexParent}(D) = D', \\ \uparrow & \neg\mathsf{declares}_P(D,s) \ \wedge \ P.\mathit{lexParent}(D) = \uparrow. \end{cases} \tag{6.7}$$

Unlike the 2010 algorithm, the result must retain both the receiver and the declaration that granted lexical authority. Write irecv for this access-aware form of *implicitReceiver*. Using Equations (6.7) and (5.4), define

$$\mathsf{irecv}_{P,H}(o_0,C_0,D_0,s) = \begin{cases} \langle o_t, D_t\rangle & \mathsf{scan}_P(D_0,s) = D_t, \\ & k = \mathsf{lexDepth}_P(D_t,D_0), \quad P,H \vdash \mathsf{enc}(o_0,C_0,k) = o_t, \\ \langle o_0, \uparrow\rangle & \mathsf{scan}_P(D_0,s) = \uparrow. \end{cases} \tag{6.8}$$

The first result dispatches by $\mathsf{outerDispatch}_a(D_t, D_0, \mu)$; the second by $\mathsf{outerDispatch}_a(D_0, D_0, \mu)$. Consequently a private method is considered only as the direct declaration at the selected lexical target. If that check fails, lookup_{prot} performs virtual lookup and skips every private method encountered on the runtime class chain.

Equivalence obligation. For a well-formed fixed P, H, and after ruling out a nearer activation or object-literal binding, the elaborated rules and the ECOOP-style presentation must produce the same dispatch result. In terms of Equations (6.5) and (6.7), the static part reduces to

$$\mathsf{bind}_P(z,s) = D_t \quad \Longleftrightarrow \quad \mathsf{scan}_P(D_0,s) = D_t, \tag{6.9}$$

With D_0 the first class scope at z, the receiver equality follows from lexDepth in Equation (5.4) and the rules for enc. The companion Lean development proves this equivalence as `binding_equivalence`, from an explicit well-formed class-binding context connecting *P.scopeChain*, *P.lexParent*, and the finite class chain. It then proves the resulting dynamic dispatch equivalence as `implicitClassDispatch_equivalence`.

6.5 DNU fallback

Missing dispatch does not perform an ordinary send of doesNotUnderstand:. It uses unrestricted lookup from Section 4, starting at the supplied class C_0, and invokes the result on the original receiver with a freshly reified mirror of the original message:

$$\frac{P,H \vdash \mathrm{lookup}_U(\mathsf{doesNotUnderstand:}, C_0) = \langle f, C_f\rangle \quad H' = H[m \mapsto \mathsf{messageMirror}(\mu)]}{P,H \vdash \mathsf{missing}(o,C_0,\mu) \longrightarrow P,H' \vdash \mathsf{invoke}(f,o,C_f,\langle \mathsf{doesNotUnderstand:}, m\rangle)} \quad \text{(DNU)}$$

Consequently a class's DNU implementation is honored regardless of its access modifier. The required protected implementation on Object ensures that unrestricted lookup succeeds in every well-formed base program and ultimately throws MessageNotUnderstood unless overridden.

7 Sequential abstract machine

This section closes the dispatch loop with a deterministic small-step machine. It covers left-to-right message construction, sequencing, closure creation, method and closure invocation, tail-call-observable continuation links, and local and non-local return. Simultaneous slot clauses remain with the class initialization layer; their absence is stated explicitly below.

7.1 Configurations and frames

A request records the authority of a send after its receiver, if any, has been evaluated:

$$\begin{aligned}\theta ::= {} & \mathsf{ordinaryRequest}(o) \mid \mathsf{implicitRequest}(d, D_0) \\ & \mid \mathsf{outerRequest}(D_t, D_0) \mid \mathsf{selfRequest}(D_0) \mid \mathsf{superRequest}.\end{aligned} \tag{7.1}$$

An intra-activation frame stack π is built from

$$\begin{aligned}F ::= {} & \mathsf{recv}(s, \bar{e}) \mid \mathsf{args}(\theta, s, \bar{o}, \bar{e}) \mid \mathsf{seq}(\tau, \bar{s}) \\ & \mid \mathsf{returnFrame} \mid \mathsf{init}(j, \bar{\ell}, b, \tau), \\ \pi ::= {} & \epsilon \mid \pi \cdot F, \qquad \tau ::= \mathsf{methodBody}(o) \mid \mathsf{closureBody}.\end{aligned} \tag{7.2}$$

Here $\bar{s}$ is a statement list, $\bar{\ell}$ is a normalized local-slot list, and j is a physical local-slot identity. An activation stack and machine configuration are

$$\mathcal{A} ::= \epsilon \mid \mathcal{A} \cdot \langle a, \pi \rangle, \qquad \Sigma = \langle P, H, \mathcal{A}, t \rangle. \tag{7.3}$$

We abbreviate the configuration of Equation (7.3) as $\langle H; \mathcal{A}; t \rangle_P$ when this improves the layout. The rightmost activation is current; t is its control term. Suspended evaluation frames live in π, while the corresponding activation and its observable continuation link live in H. We write $\longmapsto$ for one machine step and omit unchanged P below.

7.2 Expression and message order

Evaluation of an ordinary send begins with its receiver:

$$\langle P, H, \mathcal{A} \cdot \langle a, \pi \rangle, \mathsf{send}(e, s, \bar{e}) \rangle \longmapsto \langle P, H, \mathcal{A} \cdot \langle a, \pi \cdot \mathsf{recv}(s, \bar{e}) \rangle, e \rangle. \quad (\textsc{Send-Receiver}).$$

Once the receiver is o, the request is $\mathsf{ordinaryRequest}(o)$. All other send forms in Equation (2.1) determine their request without evaluating a receiver:

$$\begin{aligned}\mathsf{request}_{D_0}(\mathsf{implicit}(s, \bar{e}, d)) &= \langle \mathsf{implicitRequest}(d, D_0), s, \bar{e} \rangle, \\ \mathsf{request}_{D_0}(\mathsf{selfSend}(s, \bar{e}, D_0)) &= \langle \mathsf{selfRequest}(D_0), s, \bar{e} \rangle, \\ \mathsf{request}_{D_0}(\mathsf{outerSend}(s, \bar{e}, D_t, D_0)) &= \langle \mathsf{outerRequest}(D_t, D_0), s, \bar{e} \rangle, \\ \mathsf{request}_{D_0}(\mathsf{superSend}(s, \bar{e})) &= \langle \mathsf{superRequest}, s, \bar{e} \rangle.\end{aligned} \tag{7.4}$$

The following rules use Equation (7.4) to start an argument list; e_s ranges over its four non-ordinary send forms:

$$\frac{\mathsf{request}_{D_0}(e_s) = \langle \theta, s, \epsilon \rangle}{\langle P, H, \mathcal{A} \cdot \langle a, \pi \rangle, e_s \rangle \longmapsto \langle P, H, \mathcal{A} \cdot \langle a, \pi \rangle, \mathsf{dispatch}(\theta, \langle s, \epsilon \rangle) \rangle} \quad (\textsc{Send-No-Args})$$

$$\frac{\mathsf{request}_{D_0}(e_s) = \langle \theta, s, \langle e_1, \bar{e} \rangle \rangle}{\langle P, H, \mathcal{A} \cdot \langle a, \pi \rangle, e_s \rangle \longmapsto \langle P, H, \mathcal{A} \cdot \langle a, \pi \cdot \mathsf{args}(\theta, s, \epsilon, \bar{e}) \rangle, e_1 \rangle} \quad (\textsc{Send-First-Arg})$$

The receiver frame from Equation (7.2) uses the same two continuations:

$$\begin{aligned}&\langle P, H, \mathcal{A} \cdot \langle a, \pi \cdot \mathsf{recv}(s, \epsilon) \rangle, o \rangle \longmapsto \langle P, H, \mathcal{A} \cdot \langle a, \pi \rangle, \mathsf{dispatch}(\mathsf{ordinaryRequest}(o), \langle s, \epsilon \rangle) \rangle, \\ &\langle P, H, \mathcal{A} \cdot \langle a, \pi \cdot \mathsf{recv}(s, \langle e_1, \bar{e} \rangle) \rangle, o \rangle \longmapsto \langle P, H, \mathcal{A} \cdot \langle a, \pi \cdot \mathsf{args}(\mathsf{ordinaryRequest}(o), s, \epsilon, \bar{e}) \rangle, e_1 \rangle.\end{aligned} \tag{7.5}$$

Argument values are accumulated on the left, so these rules enforce source order:

$$\begin{aligned}\langle P, H, \mathcal{A} \cdot \langle a, \pi \cdot \mathsf{args}(\theta, s, \bar{o}, \langle e, \bar{e}\rangle)\rangle, o'\rangle &\longmapsto \langle P, H, \mathcal{A} \cdot \langle a, \pi \cdot \mathsf{args}(\theta, s, \bar{o} \cdot \langle o'\rangle, \bar{e})\rangle, e\rangle,\\ \langle P, H, \mathcal{A} \cdot \langle a, \pi \cdot \mathsf{args}(\theta, s, \bar{o}, \epsilon)\rangle, o'\rangle &\longmapsto \langle P, H, \mathcal{A} \cdot \langle a, \pi\rangle, \mathsf{dispatch}(\theta, \langle s, \bar{o} \cdot \langle o'\rangle\rangle)\rangle.\end{aligned} \tag{7.6}$$

Standalone self reduces to the current instance recorded in $H(a)$; parentheses have already been eliminated without changing a parenthesized self send's syntactic classification. Formally,

$$\frac{H(a).o = o}{\langle P, H, \mathcal{A} \cdot \langle a, \pi\rangle, \mathsf{self}\rangle \longmapsto \langle P, H, \mathcal{A} \cdot \langle a, \pi\rangle, o\rangle} \quad \text{(Self-Value)}.$$

7.3 From requests to invocation

For $n \geq 0$, let $\mathsf{valueSelector}(n)$ be #value for $n = 0$ and the symbol consisting of n repetitions of the keyword value: otherwise. Let $\mathsf{closureCall}_H(o, \mu)$ hold exactly when $H(o)$ is a closure with n parameters and $\mathrm{selector}(\mu) = \mathsf{valueSelector}(n)$. Define the partial request-resolution function, over the requests of Equation (7.1), by the existing dispatch judgments:

$$\begin{aligned}&\mathsf{resolve}_{P,H,a}(\mathsf{ordinaryRequest}(o), \mu) = r\\ &\quad\iff \neg\mathsf{closureCall}_H(o, \mu) \ \wedge\ P, H \vdash \mathsf{ordinaryDispatch}(o, \mu) \Downarrow r,\\ &\mathsf{resolve}_{P,H,a}(\mathsf{implicitRequest}(d, D_0), \mu) = r\\ &\quad\iff P, H, a, D_0 \vdash \mathsf{implicitDispatch}(d, \mu) \Downarrow r,\\ &\mathsf{resolve}_{P,H,a}(\mathsf{outerRequest}(D_t, D_0), \mu) = r\\ &\quad\iff P, H \vdash \mathsf{outerDispatch}_a(D_t, D_0, \mu) \Downarrow r,\\ &\mathsf{resolve}_{P,H,a}(\mathsf{selfRequest}(D_0), \mu) = r\\ &\quad\iff P, H \vdash \mathsf{outerDispatch}_a(D_0, D_0, \mu) \Downarrow r,\\ &\mathsf{resolve}_{P,H,a}(\mathsf{superRequest}, \mu) = r\\ &\quad\iff P, H, a \vdash \mathsf{superDispatch}(\mu) \Downarrow r.\end{aligned} \tag{7.7}$$

The request-resolution step applies Equation (7.7):

$$\frac{\mathsf{resolve}_{P,H,a}(\theta, \mu) = r}{\langle P, H, \mathcal{A} \cdot \langle a, \pi\rangle, \mathsf{dispatch}(\theta, \mu)\rangle \longmapsto \langle P, H, \mathcal{A} \cdot \langle a, \pi\rangle, r\rangle} \quad \text{(Resolve)}$$

and the DNU rule of Section 6.5 rewrites a missing control term to invoke while allocating its message mirror. A runtimeError control term is reified and signaled at the standard-library boundary in Section 9.6.

Before ordinary request resolution, a closure receiver has the specified invocation case

$$\frac{H(c) = \mathsf{closure}\langle q, \bar{x}, \ell, b, a_c, o, C, h\rangle \quad \mu = \langle \mathsf{valueSelector}(|\bar{x}|), \bar{o}\rangle}{\begin{aligned}&\langle H; \mathcal{A} \cdot \langle a, \pi\rangle; \mathsf{dispatch}(\mathsf{ordinaryRequest}(c), \mu)\rangle_P\\ &\quad\longmapsto \langle H; \mathcal{A} \cdot \langle a, \pi\rangle; \mathsf{invokeClosure}(c, \mu)\rangle_P\end{aligned}} \quad \text{(Closure-Value)}$$

which is disjoint from Resolve for that request.

7.4 Closure creation and activation allocation

Evaluating a closure literal allocates a fresh closure object and captures the current activation state. Let

$$\mathsf{home}_H(a) = \begin{cases} a & H(a).p \in \mathit{MethodId},\\ H(a).h & \text{otherwise.}\end{cases} \tag{7.8}$$

For fresh c, the allocation rule uses home_H from Equation (7.8):

$$\frac{H(a).o = o \quad H(a).C = C \quad h = \mathsf{home}_H(a)}{\langle H; \mathcal{A} \cdot \langle a, \pi\rangle; \mathsf{closureLiteral}(q, \bar{x}, \ell, b)\rangle_P \longmapsto \langle H[c \mapsto \mathsf{closure}\langle q, \bar{x}, \ell, b, a, o, C, h\rangle]; \mathcal{A} \cdot \langle a, \pi\rangle; c\rangle_P} \quad \text{(Closure-New)}$$

The continuation installed in a callee is

$$\mathsf{calleeKont}_H(a, \pi) = \begin{cases} a & \pi \neq \epsilon, \\ H(a).k & \pi = \epsilon. \end{cases} \tag{7.9}$$

Thus an observable continuation points to the sender exactly when the sender has further computation. Define method-activation allocation for $f = \langle i_f, s, \alpha, \bar{x}, \ell, b, K\rangle$ and $\mu = \langle s, \bar{o}\rangle$, with $|\bar{x}| = |\bar{o}|$, by

$$\begin{aligned} \rho' &= [\bar{x} \mapsto \bar{o}], & \lambda' &= [\bar{\ell} \mapsto \mathsf{uninit}], \\ \gamma' &= \{i_f \mapsto a'\} \cup \begin{cases} \{K \mapsto o\} & K \in \mathit{ObjLitDeclId}, \\ \varnothing & \text{otherwise}, \end{cases} \\ \mathsf{newM}(P, H, a', f, o, C_f, \mu, k) &= H[a' \mapsto \mathsf{activation}\langle i_f, \rho', \lambda', o, C_f, k, a', \mathsf{true}, \gamma'\rangle]. \end{aligned} \tag{7.10}$$

where a' is fresh and $\bar{\ell}$ lists the physical local-slot identities. Allocation itself does not evaluate local initializers. The non-tail and tail invocation rules implement Equations (7.9) and (7.10):

$$\frac{\pi \neq \epsilon \quad k = a \quad H' = \mathsf{newM}(P, H, a', f, o, C_f, \mu, k)}{\langle H; \mathcal{A} \cdot \langle a, \pi\rangle; \mathsf{invoke}(f, o, C_f, \mu)\rangle_P \longmapsto \langle H'; \mathcal{A} \cdot \langle a, \pi\rangle \cdot \langle a', \epsilon\rangle; \mathsf{initialize}(a', \ell, b, \mathsf{methodBody}(o))\rangle_P} \quad \text{(Invoke-NonTail)}$$

$$\frac{\pi = \epsilon \quad k = H(a).k \quad H' = \mathsf{newM}(P, H, a', f, o, C_f, \mu, k)}{\langle H; \mathcal{A} \cdot \langle a, \epsilon\rangle; \mathsf{invoke}(f, o, C_f, \mu)\rangle_P \longmapsto \langle H'; \mathcal{A} \cdot \langle a', \epsilon\rangle; \mathsf{initialize}(a', \ell, b, \mathsf{methodBody}(o))\rangle_P} \quad \text{(Invoke-Tail)}$$

The old activation remains in H for mirrors and debugging even when its dynamic stack entry is replaced.

For closure activation, suppose the captured data satisfy

$$H(c) = \mathsf{closure}\langle q, \bar{x}, \ell, b, a_c, o, C, h\rangle, \qquad \mu = \langle \mathsf{valueSelector}(|\bar{x}|), \bar{o}\rangle. \tag{7.11}$$

Under Equation (7.11), with a' fresh, define

$$\begin{aligned} \rho_c &= [\bar{x} \mapsto \bar{o}], & \lambda_c &= [\bar{\ell} \mapsto \mathsf{uninit}], \\ \gamma_c &= \mathsf{scope}_H(a_c)[q \mapsto a'], \\ \mathsf{newC}(H, a', c, \bar{o}, k) &= H[a' \mapsto \mathsf{activation}\langle c, \rho_c, \lambda_c, o, C, k, h, \mathsf{true}, \gamma_c\rangle]. \end{aligned} \tag{7.12}$$

The two closure invocation cases use the activation allocated by Equation (7.12):

$$\frac{\pi \neq \epsilon \quad k = a \quad H' = \mathsf{newC}(H, a', c, \bar{o}, k)}{\langle H; \mathcal{A} \cdot \langle a, \pi\rangle; \mathsf{invokeClosure}(c, \mu)\rangle_P \longmapsto \langle H'; \mathcal{A} \cdot \langle a, \pi\rangle \cdot \langle a', \epsilon\rangle; \mathsf{initialize}(a', \ell, b, \mathsf{closureBody})\rangle_P} \quad \text{(Closure-Invoke-NonTail)}$$

$$\frac{\pi = \epsilon \quad k = H(a).k \quad H' = \mathsf{newC}(H, a', c, \bar{o}, k)}{\langle H; \mathcal{A} \cdot \langle a, \epsilon\rangle; \mathsf{invokeClosure}(c, \mu)\rangle_P \longmapsto \langle H'; \mathcal{A} \cdot \langle a', \epsilon\rangle; \mathsf{initialize}(a', \ell, b, \mathsf{closureBody})\rangle_P} \quad \text{(Closure-Invoke-Tail)}$$

7.5 Local initialization and bodies

Normalize a sequential local declaration as

$$\lambda ::= \mathsf{immutable}(j, e) \mid \mathsf{mutable}(j, e) \mid \mathsf{mutable}(j, \mathsf{none}) \mid \mathsf{lazyImmutable}(j, e) \mid \mathsf{lazyMutable}(j, e). \tag{7.13}$$

The normalized sequence may also contain a simultaneous group:

$$\bar{\ell} ::= \epsilon \mid \lambda \cdot \bar{\ell} \mid \mathsf{simultaneous}(\bar{\lambda}) \cdot \bar{\ell}. \tag{7.14}$$

Empty initialization of the declaration sequence in Equation (7.14) enters the body:

$$\langle P, H, \mathcal{A} \cdot \langle a, \pi \rangle, \mathsf{initialize}(a, \epsilon, b, \tau) \rangle \longmapsto \langle P, H, \mathcal{A} \cdot \langle a, \pi \rangle, \mathsf{body}(\tau, b) \rangle. \tag{7.15}$$

An immutable or initialized mutable local evaluates its initializer in the new activation and then writes directly:

$$\frac{\lambda \in \{\mathsf{immutable}(j, e), \mathsf{mutable}(j, e)\}}{\langle P, H, \mathcal{A} \cdot \langle a, \pi \rangle, \mathsf{initialize}(a, \lambda \cdot \bar{\ell}, b, \tau) \rangle \longmapsto \langle P, H, \mathcal{A} \cdot \langle a, \pi \cdot \mathsf{init}(j, \bar{\ell}, b, \tau) \rangle, e \rangle} \tag{7.16}$$

$$\langle P, H, \mathcal{A} \cdot \langle a, \pi \cdot \mathsf{init}(j, \bar{\ell}, b, \tau) \rangle, o \rangle \longmapsto \langle P, H[a.\ell(j) \mapsto o], \mathcal{A} \cdot \langle a, \pi \rangle, \mathsf{initialize}(a, \bar{\ell}, b, \tau) \rangle. \tag{7.17}$$

An uninitialized mutable local and either lazy form write nil without evaluating the lazy initializer:

$$\frac{\lambda \in \{\mathsf{mutable}(j, \mathsf{none}), \mathsf{lazyImmutable}(j, e), \mathsf{lazyMutable}(j, e)\}}{\langle P, H, \mathcal{A} \cdot \langle a, \pi \rangle, \mathsf{initialize}(a, \lambda \cdot \bar{\ell}, b, \tau) \rangle \longmapsto \langle P, H[a.\ell(j) \mapsto \mathsf{nil}], \mathcal{A} \cdot \langle a, \pi \rangle, \mathsf{initialize}(a, \bar{\ell}, b, \tau) \rangle.} \tag{7.18}$$

Simultaneous locals use direct activation-slot operations, just as instance initialization uses the direct object-slot operations of Equation (8.27). Extend the frame grammar in Equation (7.2) with $\mathsf{localWrite}(a, j)$ and define

$$\begin{gathered} \frac{H(a_0).\ell(j) = v}{\langle P, H, \mathcal{A} \cdot \langle a, \pi \rangle, \mathsf{readLocal}(a_0, j) \rangle \longmapsto \langle P, H, \mathcal{A} \cdot \langle a, \pi \rangle, v \rangle}, \\ \langle P, H, \mathcal{A} \cdot \langle a, \pi \rangle, \mathsf{writeLocal}(a_0, j, e) \rangle \longmapsto \langle P, H, \mathcal{A} \cdot \langle a, \pi \cdot \mathsf{localWrite}(a_0, j) \rangle, e \rangle, \\ \langle P, H, \mathcal{A} \cdot \langle a, \pi \cdot \mathsf{localWrite}(a_0, j) \rangle, v \rangle \longmapsto \langle P, H[a_0.\ell(j) \mapsto v], \mathcal{A} \cdot \langle a, \pi \rangle, v \rangle. \end{gathered} \tag{7.19}$$

Let e_F be the annotated core send denoting `Past`Future` in activation a. For a declaration in a simultaneous group, define its installation term by

$$\begin{aligned} \mathsf{localInstall}_a(\mathsf{immutable}(j, e)) &= \mathsf{writeLocal}(a, j, \mathsf{send}(e_F, \mathsf{computing:}, [e])), \\ \mathsf{localInstall}_a(\mathsf{mutable}(j, e)) &= \mathsf{writeLocal}(a, j, \mathsf{send}(e_F, \mathsf{computing:}, [e])), \\ \mathsf{localInstall}_a(\mathsf{mutable}(j, \mathsf{none})) &= \mathsf{writeLocal}(a, j, \mathsf{nil}), \\ \mathsf{localInstall}_a(\mathsf{lazyImmutable}(j, e)) &= \mathsf{writeLocal}(a, j, \mathsf{nil}), \\ \mathsf{localInstall}_a(\mathsf{lazyMutable}(j, e)) &= \mathsf{writeLocal}(a, j, \mathsf{nil}). \end{aligned} \tag{7.20}$$

Suppose the declarations with eager right-hand sides in $\bar{\lambda}$, in source order, are $j_1 = e_1, \ldots, j_n = e_n$. Using the installation terms of Equation (7.20), define

$$\begin{aligned} \mathsf{localSimCode}_a(\bar{\lambda}) = \langle &\mathsf{localInstall}_a(\lambda_1), \ldots, \mathsf{localInstall}_a(\lambda_m), \\ &\mathsf{send}(\mathsf{readLocal}(a, j_1), \#\mathsf{resolve}, \epsilon), \ldots, \mathsf{send}(\mathsf{readLocal}(a, j_n), \#\mathsf{resolve}, \epsilon) \rangle. \end{aligned} \tag{7.21}$$

Using the internal sequence defined in Equation (8.33), the group rule is

$$\begin{aligned}&\mathsf{initialize}(a, \mathsf{simultaneous}(\bar{\lambda}) \cdot \bar{\ell}, b, \tau)\\ &\quad \longmapsto \mathsf{internalSeq}(\mathsf{localSimCode}_a(\bar{\lambda}), \mathsf{initialize}(a, \bar{\ell}, b, \tau)).\end{aligned} \tag{7.22}$$

Equations (7.19)–(7.22) therefore install every future before the first resolution and choose source order for resolution, exactly as the instance-slot expansion in Equation (8.31).

Statements are $\mathsf{expr}(e)$ or $\mathsf{return}(e)$. Define

$$\begin{aligned}\mathsf{empty}(\mathsf{methodBody}(o)) &= o, & \mathsf{last}(\mathsf{methodBody}(o), v) &= o,\\ \mathsf{empty}(\mathsf{closureBody}) &= \mathsf{nil}, & \mathsf{last}(\mathsf{closureBody}, v) &= v.\end{aligned} \tag{7.23}$$

For a frame stack π, define

$$\mathsf{start}(\pi, \mathsf{expr}(e)) = \langle \pi, e\rangle, \qquad \mathsf{start}(\pi, \mathsf{return}(e)) = \langle \pi \cdot \mathsf{returnFrame}, e\rangle. \tag{7.24}$$

Thus Equation (7.24) installs the return frame before its expression is evaluated. Using the result functions in Equation (7.23), the empty-body rule is

$$\mathsf{body}(\tau, \epsilon) \longmapsto \mathsf{finish}(\mathsf{empty}(\tau)). \tag{7.25}$$

A nonempty body uses Equation (7.24) to start its first statement and save the remainder:

$$\frac{\mathsf{start}(\pi \cdot \mathsf{seq}(\tau, \bar{s}), s_1) = \langle \pi', e_1\rangle}{\langle P, H, \mathcal{A} \cdot \langle a, \pi\rangle, \mathsf{body}(\tau, \langle s_1, \bar{s}\rangle)\rangle \longmapsto \langle P, H, \mathcal{A} \cdot \langle a, \pi'\rangle, e_1\rangle}. \tag{7.26}$$

On an ordinary statement value,

$$\begin{aligned}\langle \pi \cdot \mathsf{seq}(\tau, \epsilon), v\rangle &\longmapsto \langle \pi, \mathsf{finish}(\mathsf{last}(\tau, v))\rangle,\\ \mathsf{start}(\pi \cdot \mathsf{seq}(\tau, \bar{s}), s) = \langle \pi', e\rangle &\implies \langle \pi \cdot \mathsf{seq}(\tau, \langle s, \bar{s}\rangle), v\rangle \longmapsto \langle \pi', e\rangle.\end{aligned} \tag{7.27}$$

The abbreviated pairs above leave $P, H, \mathcal{A}$ and current a unchanged. The installed return frame transfers control non-locally:

$$\langle P, H, \mathcal{A}\cdot\langle a, \pi\cdot\mathsf{returnFrame}\rangle, v\rangle \longmapsto \langle P, H, \mathcal{A}\cdot\langle a, \pi\rangle, \mathsf{transfer}(\mathsf{returnTarget}_H(a), v)\rangle. \quad \text{(Return-Value)}$$

7.6 Completion and return

For a stack containing c, define

$$\mathsf{resume}(\mathcal{A}_0 \cdot \langle c, \pi_c\rangle \cdot \mathcal{A}_1, c, v) = \langle \mathcal{A}_0 \cdot \langle c, \pi_c\rangle, v\rangle. \tag{7.28}$$

It discards dynamic stack entries above the continuation but retains their heap records. Normal completion uses Equation (7.28); it first saves $c = H(a).k$, then severs the link:

$$\frac{c = H(a).k \neq \mathsf{nil} \quad \mathsf{resume}(\mathcal{A} \cdot \langle a, \pi\rangle, c, v) = \langle \mathcal{A}', v\rangle}{\langle P, H, \mathcal{A} \cdot \langle a, \pi\rangle, \mathsf{finish}(v)\rangle \longmapsto \langle P, H[a.k \mapsto \mathsf{nil}], \mathcal{A}', v\rangle} \quad \text{(Finish-Resume)}$$

If $H(a).k = \mathsf{nil}$, normal completion yields a terminal result rather than $\mathsf{cannotReturn}$:

$$\frac{H(a).k = \mathsf{nil}}{\langle P, H, \mathcal{A} \cdot \langle a, \pi\rangle, \mathsf{finish}(v)\rangle \longmapsto \langle P, H[a.k \mapsto \mathsf{nil}], \epsilon, \mathsf{halt}(v)\rangle} \quad \text{(Finish-Halt)}.$$

Let the target of an explicit return from a be

$$\mathsf{returnTarget}_H(a) = \begin{cases} a & H(a).p \in \mathit{MethodId}, \\ H(a).h & \text{otherwise.} \end{cases} \tag{7.29}$$

After the return expression evaluates to v, its frame produces $\mathsf{transfer}(\mathsf{returnTarget}_H(a), v)$ using Equation (7.29). Define $\mathsf{dependents}_H(t)$ as the least set X such that

$$t \in X \quad \wedge \quad \forall x \in \mathit{ActId}.\ (H(x).k \in X \Rightarrow x \in X). \tag{7.30}$$

Let $H' = \mathsf{markUncontinuable}(H, t)$ be H updated, for every $x \in \mathsf{dependents}_H(t)$, by

$$H'(x).k = \mathsf{nil}, \qquad H'(x).q_a = \mathsf{false}. \tag{7.31}$$

Using Equations (7.30) and (7.31), for $c = H(t).k \neq \mathsf{nil}$, explicit return resumes c with v after this marking:

$$\frac{\begin{array}{c} t{=}\mathsf{returnTarget}_H(a) \quad c{=}H(t).k{\neq}\mathsf{nil} \\ H'{=}\mathsf{markUncontinuable}(H,t) \\ \mathsf{resume}(\mathcal{A}{\cdot}\langle a,\pi\rangle,c,v){=}\langle\mathcal{A}',v\rangle \end{array}}{\begin{array}{c} \langle H; \mathcal{A} \cdot \langle a, \pi\rangle; \mathsf{transfer}(t, v)\rangle_P \\ \longmapsto \langle H'; \mathcal{A}'; v\rangle_P \end{array}} \quad (\textsc{Return-Transfer})$$

If the saved c is nil, the machine instead produces an exception after marking t uncontinuable:

$$\frac{t = \mathsf{returnTarget}_H(a) \quad H(t).k = \mathsf{nil} \quad H' = \mathsf{markUncontinuable}(H, t)}{\begin{array}{c} \langle H; \mathcal{A} \cdot \langle a, \pi\rangle; \mathsf{transfer}(t, v)\rangle_P \\ \longmapsto \langle H'; \mathcal{A} \cdot \langle a, \pi\rangle; \mathsf{throw}(\mathsf{cannotReturn})\rangle_P \end{array}} \quad (\textsc{Return-Cannot})$$

This applies both to a repeated method return and to a non-local return whose home method has already completed. Section 9.6 reifies this abrupt term without discarding its frames; the standard exception library determines whether signaling unwinds them or resumes at the failure point.

8 Class construction and initialization

This section separates three times that the surface notation intentionally interleaves: elaboration creates and shares mixins, evaluation of a class expression determines a superclass and allocates a fresh class, and a later primary-factory invocation initializes a fresh instance. In particular, the superclass expression is not re-evaluated when an instance is made, while its initializer message is not evaluated when the class is made.

8.1 Normalized class expressions

Elaboration gives every class body a descriptor

$$\chi_b = \langle D, n, u, s_f, \bar{x}, M, M_c, m_S\rangle. \tag{8.1}$$

D is the class declaration identity, n its name, u records whether the superclass was implicit, $s_f, \bar{x}$ are the primary-factory signature, M and M_c are the instance- and class-side mixins, and m_S is the deferred superclass-initializer message template. A mixin application has descriptor

$$\chi_a = \langle D, n, s_f, \bar{x}, M_c, i_A, m_S, m_M\rangle, \tag{8.2}$$

where i_A is the stable identity of its synthesized instance initializer and m_M is the deferred initializer message for the applied mixin. From Equations (8.1) and (8.2), the core forms are

$$\mathsf{classBody}(\chi_b, e_S), \qquad \mathsf{mixinApply}(\chi_a, e_S, e_M). \tag{8.3}$$

Chained surface applications elaborate to left-associated $\mathsf{mixinApply}$ nodes.

The surrounding-object annotation on D determines

$$\mathsf{classEO}_{P,H}(a, D) = \begin{cases} a & P.owner(D) = \mathsf{activationOwner}, \\ H(a).o & P.owner(D) \in \{\mathsf{classOwner}, \mathsf{objectLiteralOwner}\}, \\ \mathsf{nil} & P.owner(D) = \mathsf{topOwner}. \end{cases} \tag{8.4}$$

Thus a class expression directly in a method or closure is enclosed by its activation, while a member class is enclosed by the receiver of its synthesized getter. Top-level evaluation supplies nil without an activation.

Write $\mathsf{isClass}_H(C)$ when the corresponding record in Equation (3.10) has tag class, and define

$$\mathsf{factorySel}_H(C) = s_f \quad \text{when} \quad H(C).q = \langle i_I, s_f, \bar{x}, \kappa \rangle. \tag{8.5}$$

Write $\mathsf{hasFactory}_H(C)$ when $H(C).q$ is a factory descriptor rather than none; "factory descriptor" means the form in Equation (3.9). Using the selector projection in Equation (8.5), validation of an inheritance clause is the total function

$$\mathsf{inhErr}_H(v, m) = \begin{cases} \mathsf{classExpected} & \neg \mathsf{isClass}_H(v), \\ \mathsf{classNotInstantiable} & \mathsf{isClass}_H(v) \ \wedge \ \neg \mathsf{hasFactory}_H(v), \\ \mathsf{wrongFactory} & \mathsf{hasFactory}_H(v) \ \wedge \ \mathrm{selector}(m) \neq \mathsf{factorySel}_H(v), \\ \uparrow & \text{otherwise.} \end{cases} \tag{8.6}$$

The following allocation is defined only for fresh C, C_m, a class S, and a descriptor whose class-side mixin contains the synthesized public primary factory:

$$\begin{aligned} \mathsf{allocClass}(H, C, C_m, \chi_C, M, S, eo, M_c, q) = H[&C_m \mapsto \mathsf{class}\langle M_c, \mathsf{Class}, eo, \mathsf{Metaclass}, \mathsf{none}, \chi_C \rangle, \\ &C \mapsto \mathsf{class}\langle M, S, eo, C_m, q, \chi_C \rangle]. \end{aligned} \tag{8.7}$$

This makes every created class's metaclass fresh, a direct subclass of the platform class Class, an instance of $\mathsf{Metaclass}$, and enclosed by the same object as its class.

Extend the base frame grammar of Equation (7.2) by

$$F ::= \cdots \mid \mathsf{makeBody}(\chi_b, eo) \mid \mathsf{mixLeft}(\chi_a, e_M) \mid \mathsf{mixRight}(\chi_a, S). \tag{8.8}$$

Class-body evaluation uses Equation (8.4) to fix the enclosing object, then evaluates the superclass expression:

$$\frac{eo = \mathsf{classEO}_{P,H}(a, D)}{\begin{array}{l}\langle H; \mathcal{A} \cdot \langle a, \pi \rangle; \mathsf{classBody}(\chi_b, e_S) \rangle_P \\ \quad \longmapsto \langle H; \mathcal{A} \cdot \langle a, \pi \cdot \mathsf{makeBody}(\chi_b, eo) \rangle; e_S \rangle_P\end{array}} \quad (\textsc{Class-Super})$$

Let $\iota_P(M) = \langle i_I, s_f, \bar{x}, \bar{g}, b_I \rangle$ as in Equations (3.6) and (3.7). Once e_S has produced S, class creation uses Equation (8.7):

$$\frac{\begin{array}{c}\mathsf{inhErr}_H(S, m_S) = \uparrow \\ eo \neq \mathsf{nil} \ \vee \ (u = \mathsf{implicitSuperclass} \wedge S = \mathsf{Object}) \\ q = \langle i_I, s_f, \bar{x}, \mathsf{bodyInitializer}(m_S) \rangle \\ H' = \mathsf{allocClass}(H, C, C_m, \mathsf{expression}(D), M, S, eo, M_c, q)\end{array}}{\begin{array}{c}\langle H; \mathcal{A} \cdot \langle a, \pi \cdot \mathsf{makeBody}(\chi_b, eo) \rangle; S \rangle_P \\ \longmapsto \langle H'; \mathcal{A} \cdot \langle a, \pi \rangle; C \rangle_P\end{array}} \quad (\textsc{Class-Commit})$$

where C, C_m are fresh. A non-class or invalid factory produces $\mathsf{runtimeError}$ by the error function in Equation (8.6) and the schema below. A top-level class with an explicit superclass produces $\mathsf{runtimeError}(\mathsf{topSuperclass})$. An omitted superclass elaborates to the class Object and the message $\langle \#\mathsf{new}, \epsilon \rangle$. For any construction frame F whose associated inheritance template is m,

$$\frac{\mathsf{inhErr}_H(v, m) = \xi \neq \uparrow}{\langle P, H, \mathcal{A} \cdot \langle a, \pi \cdot F \rangle, v \rangle \longmapsto \langle P, H, \mathcal{A} \cdot \langle a, \pi \rangle, \mathsf{runtimeError}(\xi) \rangle}. \tag{8.9}$$

This schema applies to $\mathsf{makeBody}$, $\mathsf{mixLeft}$, and $\mathsf{mixRight}$. The remaining top-level error is

$$\frac{\mathsf{inhErr}_H(S, m_S) = \uparrow \quad eo = \mathsf{nil} \quad (u \neq \mathsf{implicitSuperclass} \ \vee \ S \neq \mathsf{Object})}{\langle P, H, \mathcal{A} \cdot \langle a, \pi \cdot \mathsf{makeBody}(\chi_b, eo) \rangle, S \rangle \longmapsto \langle P, H, \mathcal{A} \cdot \langle a, \pi \rangle, \mathsf{runtimeError}(\mathsf{topSuperclass}) \rangle}. \tag{8.10}$$

8.2 Mixin application

Using the class forms in Equation (8.3) and the construction frames in Equation (8.8), the superclass side is evaluated before the class supplying the mixin:

$$\langle P, H, \mathcal{A} \cdot \langle a, \pi \rangle, \mathsf{mixinApply}(\chi_a, e_S, e_M) \rangle \longmapsto \langle P, H, \mathcal{A} \cdot \langle a, \pi \cdot \mathsf{mixLeft}(\chi_a, e_M) \rangle, e_S \rangle, \tag{8.11}$$

$$\frac{\mathsf{inhErr}_H(S, m_S) = \uparrow}{\langle P, H, \mathcal{A} \cdot \langle a, \pi \cdot \mathsf{mixLeft}(\chi_a, e_M) \rangle, S \rangle \longmapsto \langle P, H, \mathcal{A} \cdot \langle a, \pi \cdot \mathsf{mixRight}(\chi_a, S) \rangle, e_M \rangle}. \tag{8.12}$$

For a class record $H(R) = \mathsf{class}\langle M_R, S_R, eo_R, C_{mR}, q_R, K_R \rangle$ of the form in Equation (3.10), commit uses Equation (8.7):

$$\frac{\begin{array}{c}\mathsf{inhErr}_H(R, m_M) = \uparrow \quad \mathsf{factorySel}_H(R) = s_f \\ \iota_P(M_R) = \langle i_M, s_f, \bar{y}, \bar{g}, b_M \rangle \\ q = \langle i_A, s_f, \bar{x}, \mathsf{appliedMixinInitializer}(m_S, m_M) \rangle \\ H' = \mathsf{allocClass}(H, I, I_m, \mathsf{expression}(D), M_R, S, eo_R, M_c, q)\end{array}}{\begin{array}{c}\langle H; \mathcal{A} \cdot \langle a, \pi \cdot \mathsf{mixRight}(\chi_a, S) \rangle; R \rangle_P \\ \longmapsto \langle H'; \mathcal{A} \cdot \langle a, \pi \rangle; I \rangle_P\end{array}} \quad (\textsc{Mixin-Commit})$$

for fresh I, I_m. Hence every application shares M_R and any later reflective edit to that mixin, but has a fresh class, metaclass, superclass link, and primary factory. Its enclosing object is copied from R, not from the activation evaluating the application. Non-class operands and either selector mismatch produce the corresponding runtime errors above.

8.3 Message templates and primary factories

A deferred message template is

$$m = \langle s, \bar{e} \rangle. \tag{8.13}$$

The control term $\mathsf{message}(m)$ evaluates only its arguments, left to right, using $\mathsf{msgArgs}(s, \bar{o}, \bar{e})$ frames analogous to the receiver and argument continuations in Equations (7.5) and (7.6):

$$\begin{array}{c}\mathsf{message}(\langle s, \epsilon \rangle) \longmapsto \langle s, \epsilon \rangle, \\ \mathsf{message}(\langle s, \langle e_1, \bar{e} \rangle \rangle) \longmapsto \langle \mathsf{msgArgs}(s, \epsilon, \bar{e}), e_1 \rangle, \\ \langle \mathsf{msgArgs}(s, \bar{o}, \langle e, \bar{e} \rangle), o' \rangle \longmapsto \langle \mathsf{msgArgs}(s, \bar{o} \cdot \langle o' \rangle, \bar{e}), e \rangle, \\ \langle \mathsf{msgArgs}(s, \bar{o}, \epsilon), o' \rangle \longmapsto \langle s, \bar{o} \cdot \langle o' \rangle \rangle.\end{array} \tag{8.14}$$

These abbreviated transitions preserve the surrounding machine components. They explain precisely why factory parameters are unavailable while e_S is evaluated but are available when m_S is evaluated.

Elaboration installs in M_c a public method with selector s_f, formals $\bar{x}$, and body equivalent to

$$\mathsf{return}\big(\mathsf{newInstanceCurrent}(\langle s_f, \bar{x}\rangle)\big), \tag{8.15}$$

where each binding occurrence in $\bar{x}$ elaborates to its activation send. Thus primary factories participate in ordinary public method lookup; newInstanceCurrent is an internal primitive occurring only in this synthesized body; it obtains the class from the factory activation's current instance. Rule Factory-New below is reached from the synthesized body in Equation (8.15).

Define the root-to-leaf physical layout by

$$\begin{aligned} \xi_s ::= (C, j), \qquad \xi_n ::= (C, N), \\ \mathsf{layout}_{P,H}(\mathsf{Top}) = \epsilon, \\ \mathsf{layout}_{P,H}(C) = \mathsf{layout}_{P,H}(\underset{H}{\mathsf{super}}(C)) \cdot \\ \langle (C, j) \mid j \in \mathsf{ownSlots}_P(\underset{H}{\mathsf{mixin}}(C))\rangle. \end{aligned} \tag{8.16}$$

Here ownSlots is the static declaration projection in Equation (3.8). The identity j identifies a declaration, while $\xi_s = (C, j)$ identifies its storage in application C. The comprehension preserves declaration order. Distinct applications have disjoint storage even when they apply the same mixin, not just when different declarations share a source name. Likewise $\xi_n = (C, N)$ identifies an application-local nested-class cache entry. For $H(C).q = \langle i_I, s_f, \bar{x}, \kappa\rangle$ and $\mu = \langle s_f, \bar{o}\rangle$, define, with fresh a_I,

$$\mathsf{newI}(H, a_I, C, o, \mu, k) = H[a_I \mapsto \mathsf{activation}\langle i_I, [\bar{x} \mapsto \bar{o}], \varnothing, o, C, k, a_I, \mathsf{true}, \{i_I \mapsto a_I\}\rangle]. \tag{8.17}$$

If the current factory activation is a and its return frame is already installed, instance allocation uses Equation (8.16) and initializer activation allocation uses Equation (8.17):

$$\frac{\begin{array}{c} H(a).o{=}C \quad H(C).q{=}\langle i_I, s_f, \bar{x}, \kappa\rangle \\ \mu{=}\langle s_f, \bar{o}\rangle \quad |\bar{x}|{=}|\bar{o}| \\ \sigma_0{=}[\mathsf{layout}_{P,H}(C){\mapsto}\mathsf{nil}] \\ H_0{=}H[o{\mapsto}\mathsf{object}\langle C, \sigma_0, \varnothing\rangle] \quad H'{=}\mathsf{newI}(H_0, a_I, C, o, \mu, a) \end{array}}{\begin{array}{l} \langle H; \mathcal{A} \cdot \langle a, \pi \cdot \mathsf{returnFrame}\rangle; \mathsf{newInstanceCurrent}(\mu)\rangle_P \\ \quad \longmapsto \langle H'; \mathcal{A} \cdot \langle a, \pi \cdot \mathsf{returnFrame}\rangle \cdot \langle a_I, \epsilon\rangle; \mathsf{initClass}(C, o, \mu)\rangle_P \end{array}} \quad \text{(Factory-New)}$$

where o, a_I are fresh. All inherited and direct physical slots therefore contain nil before any superclass initializer runs. Normal completion of a_I resumes the factory's return frame with o; the existing return rule then returns o from the primary factory. For a legal instantiable class and a call matching its factory signature, allocation must not fail because of its class chain or storage layout; in particular, repeated applications of a mixin are not an allocation error. An exception raised during factory execution, including superclass and mixin initializers, is the only permitted instantiation failure. Thus the structural allocation requirement does not assert that every factory call returns normally.

8.4 Superclass and mixin initialization

Extend the frame grammar with the initialization frames

$$\begin{aligned} F ::= \cdots \mid \mathsf{superMsg}(C, o, \mu) \mid \mathsf{afterSuper}(C, o, \mu) \\ \mid \mathsf{mixinMsg}(C, o) \mid \mathsf{afterMixin}(C, o). \end{aligned} \tag{8.18}$$

The transitions below use the frames of Equation (8.18). If $\mathrm{super}_H(C) = \mathsf{Top}$, necessarily $C = \mathsf{Object}$ and initialization proceeds directly to this mixin. Otherwise the deferred superclass message is evaluated by Equation (8.14) in the new initializer activation:

$$\frac{\begin{array}{c}\mathrm{super}_H(C) = \mathsf{Top} \\ \hline \langle P, H, \mathcal{A} \cdot \langle a, \pi\rangle, \mathsf{initClass}(C, o, \mu)\rangle \longmapsto \langle P, H, \mathcal{A} \cdot \langle a, \pi\rangle, \mathsf{ownInit}(C, o, \mu)\rangle\end{array}\text{,}}{\begin{array}{c}\mathrm{super}_H(C) = S \neq \mathsf{Top} \quad H(C).q = \langle i, s, \bar{x}, \kappa\rangle \quad \mathsf{superMsg}(\kappa) = m_S \\ \hline \langle P, H, \mathcal{A} \cdot \langle a, \pi\rangle, \mathsf{initClass}(C, o, \mu)\rangle \longmapsto \langle P, H, \mathcal{A} \cdot \langle a, \pi \cdot \mathsf{superMsg}(C, o, \mu)\rangle, \mathsf{message}(m_S)\rangle\end{array}\text{.}} \tag{8.19}$$

After that message becomes μ_S, the superclass initializer is invoked on the same object, but with current class S, using the activation constructor of Equation (8.17):

$$\frac{\begin{array}{c}S{=}\mathrm{super}_H(C) \quad \mathrm{selector}(\mu_S){=}\mathsf{factorySel}_H(S) \\ H'{=}\mathsf{newI}(H, a_S, S, o, \mu_S, a)\end{array}}{\begin{array}{l}\langle H; \mathcal{A} \cdot \langle a, \pi \cdot \mathsf{superMsg}(C, o, \mu)\rangle; \mu_S\rangle_P \\ \quad \longmapsto \langle H'; \mathcal{A} \cdot \langle a, \pi \cdot \mathsf{afterSuper}(C, o, \mu)\rangle \cdot \langle a_S, \epsilon\rangle; \mathsf{initClass}(S, o, \mu_S)\rangle_P\end{array}} \quad \text{(Init-Super)}$$

with fresh a_S. Arity mismatch produces $\mathsf{wrongFactory}$. When a_S finishes, the ordinary completion rule resumes the saved frame, and

$$\langle \pi \cdot \mathsf{afterSuper}(C, o, \mu), o\rangle \longmapsto \langle \pi, \mathsf{ownInit}(C, o, \mu)\rangle. \tag{8.20}$$

For a body class, its own initializer runs in that same activation, using the slot-group form of Equation (8.25):

$$\frac{H(C).q = \langle i, s, \bar{x}, \mathsf{bodyInitializer}(m_S)\rangle \quad \iota_P(\mathrm{mixin}_H(C)) = \langle i, s, \bar{x}, \bar{g}, b_I\rangle}{\mathsf{ownInit}(C, o, \mu) \longmapsto \mathsf{initGroups}(o, \bar{g}, b_I)}. \tag{8.21}$$

For an applied mixin, m_M is first evaluated in the coordinator activation:

$$\frac{H(C).q = \langle i_A, s, \bar{x}, \mathsf{appliedMixinInitializer}(m_S, m_M)\rangle}{\langle \pi, \mathsf{ownInit}(C, o, \mu)\rangle \longmapsto \langle \pi \cdot \mathsf{mixinMsg}(C, o), \mathsf{message}(m_M)\rangle}. \tag{8.22}$$

Suppose $M = \mathrm{mixin}_H(C)$. Using the initializer form and mixin projection in Equations (3.6) and (3.7), write its initializer as $\iota_P(M) = \langle i_M, s_M, \bar{x}_M, \bar{g}, b_M\rangle$. Define

$$\begin{array}{l}\mathsf{newMixin}(H, a_M, M, C, o, \langle s_M, \bar{o}\rangle, k) \\ \qquad = H[a_M \mapsto \mathsf{activation}\langle i_M, [\bar{x}_M \mapsto \bar{o}], \varnothing, o, C, k, a_M, \mathsf{true}, \{i_M \mapsto a_M\}\rangle].\end{array} \tag{8.23}$$

Then, using the activation allocated by Equation (8.23),

$$\frac{\begin{array}{c}\mu_M{=}\langle s_M, \bar{o}\rangle \quad |\bar{x}_M|{=}|\bar{o}| \\ H'{=}\mathsf{newMixin}(H, a_M, M, C, o, \mu_M, a)\end{array}}{\begin{array}{l}\langle H; \mathcal{A} \cdot \langle a, \pi \cdot \mathsf{mixinMsg}(C, o)\rangle; \mu_M\rangle_P \\ \quad \longmapsto \langle H'; \mathcal{A} \cdot \langle a, \pi \cdot \mathsf{afterMixin}(C, o)\rangle \cdot \langle a_M, \epsilon\rangle; \mathsf{initGroups}(o, \bar{g}, b_M)\rangle_P\end{array}} \quad \text{(Init-Applied-Mixin)}$$

for fresh a_M. Its current class is the new application C, not the class from which M was obtained. Finally,

$$\langle \pi \cdot \mathsf{afterMixin}(C, o), o\rangle \longmapsto \langle \pi, \mathsf{finish}(o)\rangle. \tag{8.24}$$

This yields the specified order: superclass message, superclass initializer, applied-mixin message if any, mixin slots and body, and factory return.

8.5 Slot groups

Normalize an initializer group as

$$\begin{aligned} g &::= \mathsf{sequential}(\bar{d}) \mid \mathsf{simultaneous}(\bar{d}), \\ d &::= \mathsf{immutable}(j, e, i_t) \mid \mathsf{mutable}(j, e, i_t) \mid \mathsf{mutable}(j, \mathsf{none}). \end{aligned} \tag{8.25}$$

For an eager declaration, i_t is the stable activation-declaration identity assigned by elaboration to the zero-argument closure used when that declaration occurs in a simultaneous group. Sequential initialization ignores i_t. Extend the frame grammar of Equation (7.2) by

$$F ::= \cdots \mid \mathsf{slotWrite}(o, \xi_s) \mid \mathsf{slotStore}(o, \xi_s, \bar{d}, \bar{g}, b) \mid \mathsf{internal}(\bar{e}, t) \mid \mathsf{lazyStore}(o, \xi_s) \mid \mathsf{nestedStore}(o, \xi_n). \tag{8.26}$$

The internal $\mathsf{readSlot}(o, \xi_s)$ and $\mathsf{writeSlot}(o, \xi_s, e)$ operations use the physical identity ξ_s of Equation (8.16) and do not send accessors. Current-slot primitives and initializer expansion qualify the declaration using the activation's selected class, not the receiver's most derived class. Frames retain that address throughout evaluation. Their rules are

$$\begin{gathered} \frac{H(o).\sigma(\xi_s) = v}{\langle P, H, \mathcal{A} \cdot \langle a, \pi \rangle, \mathsf{readSlot}(o, \xi_s) \rangle \longmapsto \langle P, H, \mathcal{A} \cdot \langle a, \pi \rangle, v \rangle}, \\ \langle P, H, \mathcal{A} \cdot \langle a, \pi \rangle, \mathsf{writeSlot}(o, \xi_s, e) \rangle \longmapsto \langle P, H, \mathcal{A} \cdot \langle a, \pi \cdot \mathsf{slotWrite}(o, \xi_s) \rangle, e \rangle, \\ \langle P, H, \mathcal{A} \cdot \langle a, \pi \cdot \mathsf{slotWrite}(o, \xi_s) \rangle, v \rangle \longmapsto \langle P, H[o.\sigma(\xi_s) \mapsto v], \mathcal{A} \cdot \langle a, \pi \rangle, v \rangle. \end{gathered} \tag{8.27}$$

Empty initialization enters the initializer body:

$$\mathsf{initGroups}(o, \epsilon, b) \longmapsto \mathsf{body}(\mathsf{methodBody}(o), b). \tag{8.28}$$

Groups are taken in source order. A sequential group is replaced by its declaration state machine:

$$\mathsf{initGroups}(o, \mathsf{sequential}(\bar{d}) \cdot \bar{g}, b) \longmapsto \mathsf{initSeq}(o, \bar{d}, \bar{g}, b). \tag{8.29}$$

An uninitialized mutable declaration advances without a write, because every slot was already set to nil by FACTORY-NEW. A declaration with an initializer evaluates it before writing:

$$\begin{gathered} \mathsf{initSeq}(o, \mathsf{mutable}(j, \mathsf{none}) \cdot \bar{d}, \bar{g}, b) \longmapsto \mathsf{initSeq}(o, \bar{d}, \bar{g}, b), \\ \frac{d \in \{\mathsf{immutable}(j, e, i_t), \mathsf{mutable}(j, e, i_t)\} \quad H(a).C = C}{\langle \pi, \mathsf{initSeq}(o, d \cdot \bar{d}, \bar{g}, b) \rangle \longmapsto \langle \pi \cdot \mathsf{slotStore}(o, (C, j), \bar{d}, \bar{g}, b), e \rangle}, \\ \langle H; \mathcal{A} \cdot \langle a, \pi \cdot \mathsf{slotStore}(o, \xi_s, \bar{d}, \bar{g}, b) \rangle; v \rangle_P \longmapsto \langle H[o.\sigma(\xi_s) \mapsto v]; \mathcal{A} \cdot \langle a, \pi \rangle; \mathsf{initSeq}(o, \bar{d}, \bar{g}, b) \rangle_P, \\ \mathsf{initSeq}(o, \epsilon, \bar{g}, b) \longmapsto \mathsf{initGroups}(o, \bar{g}, b). \end{gathered} \tag{8.30}$$

Simultaneous groups are specified by an elaboration into the ordinary NEWSPEAK library protocol. If the declarations with right-hand sides, in source order, are $j_1 = e_1 \mathbin{@} i_{t_1}, \ldots, j_n = e_n \mathbin{@} i_{t_n}$, let e_F be the annotated core send that denotes `Past`Future` in this initializer's lexical context. Using the direct slot operations of Equation (8.27), define

$$\begin{aligned} \mathsf{simCode}_{a,C}(o, \bar{d}) = \langle & \mathsf{writeSlot}(o, (C, j_1), \mathsf{send}(e_F, \mathsf{computing:}, [e_1]_{i_{t_1}})), \ldots, \\ & \mathsf{writeSlot}(o, (C, j_n), \mathsf{send}(e_F, \mathsf{computing:}, [e_n]_{i_{t_n}})), \\ & \mathsf{send}(\mathsf{readSlot}(o, (C, j_1)), \#\mathsf{resolve}, \epsilon), \ldots, \mathsf{send}(\mathsf{readSlot}(o, (C, j_n)), \#\mathsf{resolve}, \epsilon) \rangle, \end{aligned} \tag{8.31}$$

where $C = H(a).C$ at expansion, and $[e_i]_{i_{t_i}}$ denotes the zero-argument closure with declaration identity i_{t_i} created in activation a, and uninitialized mutable declarations contribute no term. Then, using the expansion of Equation (8.31),

$$\frac{H(a).C = C}{\mathsf{initGroups}(o, \mathsf{simultaneous}(\bar{d}) \cdot \bar{g}, b) \longmapsto \mathsf{internalSeq}(\mathsf{simCode}_{a,C}(o, \bar{d}), \mathsf{initGroups}(o, \bar{g}, b))}. \tag{8.32}$$

internalSeq in Equation (8.32) is not a new evaluation mechanism. It is defined by

$$\begin{aligned} \mathsf{internalSeq}(\epsilon, t) &\longmapsto t, \\ \langle \pi, \mathsf{internalSeq}(e \cdot \bar{e}, t) \rangle &\longmapsto \langle \pi \cdot \mathsf{internal}(\bar{e}, t), e \rangle, \\ \langle \pi \cdot \mathsf{internal}(\bar{e}, t), v \rangle &\longmapsto \langle \pi, \mathsf{internalSeq}(\bar{e}, t) \rangle. \end{aligned} \tag{8.33}$$

As elsewhere, the pair notation leaves $P, H, \mathcal{A}$, and the current activation unchanged. By Equation (8.33), consequently every future is installed before the first is resolved, and all evaluation, forwarding, recursive resolution, and failure behavior is that of the specified `Past`Future` class under the ordinary send rules. The draft chooses source order for the final resolve sends; whether the Specification intends this order to be observable is recorded as an explicit question in the coverage ledger.

8.6 Synthesized slot and nested-class methods

Ordinary immutable and mutable getters elaborate to $\mathsf{return}(\mathsf{currentRead}(j))$. The activation-relative primitives reduce to the direct slot operations of Equation (8.27):

$$\begin{gathered} \frac{H(a).o = o \quad H(a).C = C}{\mathsf{currentRead}(j) \longmapsto \mathsf{readSlot}(o, (C, j))}, \\ \frac{H(a).o = o \quad H(a).C = C}{\mathsf{currentWrite}(j, e) \longmapsto \mathsf{writeSlot}(o, (C, j), e)}. \end{gathered} \tag{8.34}$$

The abbreviated rules preserve the surrounding configuration. A mutable setter invokes currentWrite with slot identifier j and argument x, then returns self. Getter and setter both retain the declared access level. Thus initialization never invokes an overridable accessor.

A lazy declaration j allocates physical slot (C, j) in each application C, initially nil, and its getter body contains $\mathsf{lazyRead}(j, e)$. In a getter activation whose current instance is o, the lazyStore frame declared in Equation (8.26) completes the direct write:

$$\begin{gathered} \frac{H(a).o = o \quad H(a).C = C \quad H(o).\sigma(C, j) = v \neq \mathsf{nil}}{\langle P, H, \mathcal{A} \cdot \langle a, \pi \rangle, \mathsf{lazyRead}(j, e) \rangle \longmapsto \langle P, H, \mathcal{A} \cdot \langle a, \pi \rangle, v \rangle}, \\ \frac{H(a).o = o \quad H(a).C = C \quad H(o).\sigma(C, j) = \mathsf{nil}}{\langle P, H, \mathcal{A} \cdot \langle a, \pi \rangle, \mathsf{lazyRead}(j, e) \rangle \longmapsto \langle P, H, \mathcal{A} \cdot \langle a, \pi \cdot \mathsf{lazyStore}(o, (C, j)) \rangle, e \rangle}, \\ \langle P, H, \mathcal{A} \cdot \langle a, \pi \cdot \mathsf{lazyStore}(o, \xi_s) \rangle, v \rangle \longmapsto \langle P, H[o.\sigma(\xi_s) \mapsto v], \mathcal{A} \cdot \langle a, \pi \rangle, v \rangle. \end{gathered} \tag{8.35}$$

There is deliberately no in-progress sentinel: a recursive read while the slot is still nil starts another evaluation, and an initializer returning nil causes the next read to evaluate it again. A mutable lazy setter is the same direct setter as Equation (8.34) and may reset the slot to nil.

Finally, a nested declaration N induces a getter whose body contains $\mathsf{nestedClass}(N, E_C)$, where E_C is one of the normalized class forms in Equation (8.3). For current receiver o, the nestedStore

frame from Equation (8.26) records a newly evaluated class:

$$\begin{gathered}\frac{H(a).o = o \quad H(a).C = R \quad H(o).\eta(R,N) = C}{\langle H; \mathcal{A} \cdot \langle a, \pi\rangle; \mathsf{nestedClass}(N, E_C)\rangle_P \longmapsto \langle H; \mathcal{A} \cdot \langle a, \pi\rangle; C\rangle_P},\\ \frac{H(a).o = o \quad H(a).C = R \quad (R,N) \notin \mathrm{dom}(H(o).\eta)}{\langle H; \mathcal{A} \cdot \langle a, \pi\rangle; \mathsf{nestedClass}(N, E_C)\rangle_P}\\ \longmapsto \quad \langle H; \mathcal{A} \cdot \langle a, \pi \cdot \mathsf{nestedStore}(o, (R,N))\rangle; \mathsf{classWithEO}(E_C, o)\rangle_P\\ \langle H; \mathcal{A} \cdot \langle a, \pi \cdot \mathsf{nestedStore}(o, \xi_n)\rangle; C\rangle_P \longmapsto \langle H[o.\eta(\xi_n) \mapsto C]; \mathcal{A} \cdot \langle a, \pi\rangle; C\rangle_P.\end{gathered} \tag{8.36}$$

classWithEO used by Equation (8.36) is precisely

$$\begin{aligned}&\langle \pi, \mathsf{classWithEO}(\mathsf{classBody}(\chi_b, e_S), o)\rangle \longmapsto \langle \pi \cdot \mathsf{makeBody}(\chi_b, o), e_S\rangle,\\ &\mathsf{classWithEO}(\mathsf{mixinApply}(\chi_a, e_S, e_M), o) \longmapsto \mathsf{mixinApply}(\chi_a, e_S, e_M).\end{aligned} \tag{8.37}$$

The second clause of Equation (8.37) ignores the forced object because the enclosing object of a mixin application is, by definition, copied from its mixin-source class. The cache belongs to the receiver and selected application, so two instances obtain distinct nested classes, while repeated sends to the same declaration in the same application on one receiver return the identical class even if inputs used by its superclass expression subsequently change.

9 Remaining sequential language

This section closes the sequential surface-language layer. Its rules do not add special cases to dispatch: compound literals elaborate to sends, an object literal reuses class and factory construction, and top-level execution merely supplies a distinguished activation. The fixed platform entries introduced below are the current specified meaning of literals; they are not implicit receivers and consequently cannot be shadowed by user declarations.

9.1 Atomic and tuple literals

An atomic literal payload is

$$\omega ::= \mathsf{integer}(n) \mid \mathsf{fixed}(q) \mid \mathsf{boolean}(b) \mid \mathsf{nilAtom} \mid \mathsf{string}(u) \mid \mathsf{symbol}(u), \tag{9.1}$$

where $n \in \mathbb{Z}$, $q \in \mathbb{Q}$, b is a mathematical Boolean, and u is a Unicode-normal-form-C string. The decoder for a digit character c is $\mathsf{dig}(c) \in \{0, \ldots, 35\}$, with letters denoting 10 through 35. If the integer digits are $u_0 \ldots u_{n-1}$, the fractional digits are $f_1 \ldots f_m$, the radix is r (10 when omitted), the sign is $\varepsilon \in \{-1, 1\}$, and the decimal exponent is z (zero when omitted), define

$$\mathsf{num}(r, \varepsilon, u, f, z) = \varepsilon \left(\sum_{i=0}^{n-1} \mathsf{dig}(u_i) r^{n-1-i} + \sum_{i=1}^{m} \mathsf{dig}(f_i) r^{-i} \right) 10^z. \tag{9.2}$$

Every digit must be smaller than r. A numeral with neither a fraction nor an exponent elaborates to $\mathsf{atom}(\mathsf{integer}(\mathsf{num}(r, \varepsilon, u, \epsilon, 0)))$; every other numeral elaborates to the corresponding fixed payload. Thus the exponent is decimal even for a radix numeral, matching the specified grammar and parser behavior, and integer magnitude is unbounded.

The static map $P.atom$ of Equation (3.4) is canonical:

$$P.atom(\omega_1) = P.atom(\omega_2) \quad \Longleftrightarrow \quad \omega_1 = \omega_2. \tag{9.3}$$

The initial heap contains all referenced atomic objects with the platform classes required by their payloads. String decoding applies Unicode NFC before Equation (9.3); symbol spelling is decoded by the symbol grammar and canonicalized in the same way. Boolean and nil syntax maps to the three distinguished payloads. Specification 0.109 removes both character-literal syntax and the Character platform class; neither occurs in the semantic domains. The atomic evaluation rule is

$$\langle P, H, \mathcal{A} \cdot \langle a, \pi \rangle, \mathsf{atom}(\omega) \rangle \longmapsto \langle P, H, \mathcal{A} \cdot \langle a, \pi \rangle, P.\mathit{atom}(\omega) \rangle. \tag{Atom}$$

Numeric, Boolean, nil, string, and symbol atoms are value objects. The mapping is an implementation freedom only as to representation: it must preserve the exact payload, the specified class protocol, and the identity condition in Equation (9.3).

Tuple literals use the fixed platform bindings

$$R = P.\mathit{platform}(\mathsf{ReadOnlyTuple}), \qquad B = P.\mathit{platform}(\mathsf{Array}). \tag{9.4}$$

Let $\underline{n}$ abbreviate the integer atom for n. Using the bindings in Equation (9.4), their specified expansion is

$$\begin{aligned} \mathsf{tupleElab}_P(\epsilon) &= \mathsf{send}(R, \mathsf{new:}, \langle \underline{0} \rangle), \\ \mathsf{tupleElab}_P(\langle e_1, \ldots, e_n \rangle) &= \mathsf{send}\big(R, \mathsf{fromArray:}, \langle \mathsf{cascade}(A_n, \bar{m}_n) \rangle\big), \\ A_n &= \mathsf{send}(B, \mathsf{new:}, \langle \underline{n} \rangle), \\ \bar{m}_n &= \langle \langle \#\texttt{at:put:}, \langle \underline{1}, e_1 \rangle \rangle, \ldots, \langle \#\texttt{at:put:}, \langle \underline{n}, e_n \rangle \rangle, \langle \#\texttt{yourself}, \epsilon \rangle \rangle. \end{aligned} \tag{9.5}$$

Here cascade is exactly the single-receiver surface cascade whose closure elaboration is fixed in Section 2. Hence A_n is evaluated once and the e_i are evaluated in source order by the ordinary send rules of Section 7. Formally,

$$\mathsf{tuple}(\bar{e}) \longmapsto \mathsf{tupleElab}_P(\bar{e}). \tag{9.6}$$

The surrounding configuration is unchanged. Tuple objects are shallowly immutable; unlike the atomic literals above, the Specification does not make them deeply immutable merely by virtue of being literal tuples.

9.2 Pattern literals

Pattern literals are syntax-directed library expressions. Let

$$\mathsf{Pattern}_d = \mathsf{implicit}(\#\mathsf{Pattern}, \epsilon, d), \tag{9.7}$$

where d is the lexical annotation selected during elaboration as in Equation (2.1). For a wildcard, a literal, and a nested pattern, respectively, define

$$\begin{aligned} \mathsf{pat}_P(_, d) &= \mathsf{send}(\mathsf{Pattern}_d, \#\mathsf{wildcard}, \epsilon), \\ \mathsf{pat}_P(l, d) &= \mathsf{send}(\mathsf{Pattern}_d, \mathsf{literal:}, \langle l \rangle), \\ \mathsf{pat}_P(\langle p \rangle, d) &= \mathsf{pattern}(p, d). \end{aligned} \tag{9.8}$$

For keyword pairs $k_1 p_1 \ldots k_n p_n$, let $\bar{k}$ be the tuple of symbol atoms $\#k_1, \ldots, \#k_n$ and let $\bar{p}$ be the tuple of the corresponding component expansions from Equation (9.8). Then

$$\begin{aligned} &\mathsf{pat}_P(k_1 p_1 \ldots k_n p_n, d) = \\ &\quad \mathsf{send}\big(\mathsf{Pattern}_d, \#\texttt{keywords:patterns:}, \langle \mathsf{tuple}(\bar{k}), \mathsf{tuple}(\bar{p}) \rangle\big). \end{aligned} \tag{9.9}$$

The tuples in Equation (9.9) are ordered collections; a conforming Pattern library may copy them into its preferred list representation. Pattern evaluation is now only the expansion

$$\mathsf{pattern}(p, d) \longmapsto \mathsf{pat}_P(p, d), \tag{9.10}$$

after which ordinary lookup, access control, and argument order apply. In particular, rebinding Pattern changes the meaning of the literal, and a pattern literal is therefore not a top-level-independent literal.

The Specification gives syntax for a variable component $?x$ but gives no expression that constructs it and no required representation beyond its role in a later binding object. For a variable occurrence at site z, this draft therefore uses the explicit elaboration parameter

$$\mathsf{pat}_P(?x_z, d) = P.\mathit{patternVariable}(z). \tag{9.11}$$

Equation (9.11) is not implementation latitude for the other pattern forms: it isolates the one missing normative clause. Once that constructor protocol is specified, the map is replaced by its ordinary-send expansion. The current library's NamedPattern suggests an implementation, but does not determine language semantics.

9.3 Object literals

Elaboration assigns an object literal the descriptor

$$\chi_o = \langle L, u, M, M_c, m_S \rangle. \tag{9.12}$$

L is its stable object-literal declaration, u records whether its superclass clause was implicit, M is the shared body mixin, M_c is a synthetic class-side mixin containing a public nullary new factory, and m_S is the deferred superclass initializer message. Its own initializer has the normalized form $\iota_P(M) = \langle i_I, \#\mathsf{new}, \epsilon, \bar{g}, b_I \rangle$ from Equations (3.6) and (3.7). Thus each evaluation will allocate a class, but every evaluation of the same L applies the identical M.

The current activation itself encloses a non-top-level object literal. A top-level activation is deliberately excluded because top-level expressions have no enclosing object:

$$\mathsf{literalEO}_H(a) = \begin{cases} \mathsf{nil} & H(a).p = \mathsf{top}, \\ a & \text{otherwise.} \end{cases} \tag{9.13}$$

Extend the frame grammar in Equation (7.2) by

$$F ::= \cdots \mid \mathsf{makeObject}(\chi_o, eo). \tag{9.14}$$

Evaluation first fixes the enclosing object using Equation (9.13), then evaluates the normalized superclass expression:

$$\frac{eo = \mathsf{literalEO}_H(a)}{\begin{array}{l}\langle H; \mathcal{A} \cdot \langle a, \pi \rangle; \mathsf{objectLiteral}(\chi_o, e_S) \rangle_P \\ \quad \longmapsto \langle H; \mathcal{A} \cdot \langle a, \pi \cdot \mathsf{makeObject}(\chi_o, eo) \rangle; e_S \rangle_P\end{array}} \quad (\textsc{Object-Super})$$

If $eo = \mathsf{nil}$, omission elaborates e_S to the fixed $P.\mathit{platform}(\mathsf{Object})$. Otherwise omission elaborates e_S to the annotated implicit send Object; hence a surrounding module may rebind the default. In both cases omission supplies the deferred message $\langle \#\mathsf{new}, \epsilon \rangle$.

After a valid superclass result, class allocation uses Equation (8.7); evaluation immediately continues as an ordinary send of new to that fresh class:

$$\frac{\begin{array}{c}\mathsf{inhErr}_H(S,m_S)=\uparrow \\ eo\neq\mathsf{nil}\ \vee\ (u=\mathsf{implicitSuperclass}\wedge S=P.\mathit{platform}(\mathsf{Object})) \\ \iota_P(M)=\langle i_I,\#\mathsf{new},\epsilon,\bar{g},b_I\rangle \\ q=\langle i_I,\#\mathsf{new},\epsilon,\mathsf{bodyInitializer}(m_S)\rangle \\ H'=\mathsf{allocClass}(H,C,C_m,\mathsf{expression}(L),M,S,eo,M_c,q)\end{array}}{\begin{array}{l}\langle H;\mathcal{A}\cdot\langle a,\pi\cdot\mathsf{makeObject}(\chi_o,eo)\rangle;S\rangle_P \\ \quad\longmapsto\langle H';\mathcal{A}\cdot\langle a,\pi\rangle;\mathsf{dispatch}(\mathsf{ordinaryRequest}(C),\langle\#\mathsf{new},\epsilon\rangle)\rangle_P\end{array}}\quad(\textsc{Object-Commit})$$

where C, C_m are fresh. The ordinary dispatch reaches the synthesized factory body in Equation (8.15), so Factory-New allocates the fresh result object and all superclass and mixin initialization rules apply unchanged. Consequently the result object, its implicit class, and that class's metaclass are fresh on every evaluation, while the mixin named by L is shared.

The invalid-inheritance schema for object literals is

$$\frac{\mathsf{inhErr}_H(v,m_S)=\xi\neq\uparrow}{\langle P,H,\mathcal{A}\cdot\langle a,\pi\cdot\mathsf{makeObject}(\chi_o,eo)\rangle,v\rangle\longmapsto\langle P,H,\mathcal{A}\cdot\langle a,\pi\rangle,\mathsf{runtimeError}(\xi)\rangle}. \tag{9.15}$$

When the inheritance itself is valid but a top-level object literal has an explicit superclass, the corresponding rule is

$$\frac{\mathsf{inhErr}_H(S,m_S)=\uparrow\quad eo=\mathsf{nil}\quad(u\neq\mathsf{implicitSuperclass}\ \vee\ S\neq P.\mathit{platform}(\mathsf{Object}))}{\langle P,H,\mathcal{A}\cdot\langle a,\pi\cdot\mathsf{makeObject}(\chi_o,eo)\rangle,S\rangle\longmapsto\langle P,H,\mathcal{A}\cdot\langle a,\pi\rangle,\mathsf{runtimeError}(\mathsf{topSuperclass})\rangle}. \tag{9.16}$$

9.4 Modules and value status

The creation provenance retained by every class record in Equation (3.10) makes the module distinction stable and heap-local. Define

$$\begin{array}{rcl}\mathsf{moduleDefinition}_{P,H}(C) & \iff & \mathsf{isClass}_H(C)\ \wedge\ \mathsf{hasFactory}_H(C)\ \wedge \\ & & H(C).\chi_C=\mathsf{expression}(D)\ \wedge\ P.\mathit{owner}(D)=\mathsf{topOwner}.\end{array} \tag{9.17}$$

The factory test excludes the metaclass allocated at the same declaration. Equation (9.17) includes a top-level mixin application: its class retains the application declaration D even though it shares the source class's mixin. A module instance is exactly

$$\mathsf{module}_{P,H}(o)\quad\iff\quad\mathsf{moduleDefinition}_{P,H}(\underset{H}{\mathsf{classOf}}(o)). \tag{9.18}$$

Every module definition satisfies the Specification's primitive value-object clause, irrespective of reflective mutability. This does not make its module instances values. More generally, base-level value status is the greatest fixed point containing the specified atomic objects and module definitions and closed under the structural value conditions (initialized immutable slots, value contents and enclosing objects, inheritance from Value, and no direct identity-method override). Reflection may alter behavior without altering this base-level classification; its causal effect is deferred to the reflective layer.

9.5 Top-level programs, compilation units, and entry

The least top-level-expression predicate is

$$\begin{aligned}
\mathsf{TL}_P(\mathsf{atom}(\omega)) &\Leftarrow \mathsf{true},\\
\mathsf{TL}_P(\mathsf{closureLiteral}(q,\bar{x},\ell,b)) &\Leftarrow \mathsf{true},\\
\mathsf{TL}_P(\mathsf{tuple}(\bar{e})) &\Leftarrow \forall e_i \in \bar{e}.\ \mathsf{TL}_P(e_i),\\
\mathsf{TL}_P(\mathsf{objectLiteral}(\chi_o, e_S)) &\Leftarrow \chi_o = \langle L,u,M,M_c,m_S\rangle \ \wedge\ P.owner(L) = \mathsf{topOwner},\\
\mathsf{TL}_P(E_C) &\Leftarrow E_C \text{ is a class form induced by } D \ \wedge\ P.owner(D) = \mathsf{topOwner},\\
\mathsf{TL}_P(\mathsf{send}(e,s,\bar{e})) &\Leftarrow \mathsf{TL}_P(e) \ \wedge\ \forall e_i \in \bar{e}.\ \mathsf{TL}_P(e_i),\\
\mathsf{TL}_P((e)) &\Leftarrow \mathsf{TL}_P(e).
\end{aligned} \tag{9.19}$$

There are no other clauses. Thus implicit, self, outer, super, and eventual sends are rejected at top level. Pattern literals are also excluded because their expansion in Equation (9.7) depends on an implicit send. The ownership premises exclude a class or object literal lexically nested in another body, even if its AST is considered in isolation. Parentheses disappear after this judgment.

A top-level activation has no parameters or slots and has nil current instance, current class, continuation, and home. With fresh a_T, define

$$\mathsf{newTop}(H, a_T) = H[a_T \mapsto \mathsf{activation}\langle \mathsf{top}, \varnothing, \varnothing, \mathsf{nil}, \mathsf{nil}, \mathsf{nil}, \mathsf{nil}, \mathsf{true}, \varnothing\rangle]. \tag{9.20}$$

For a well-formed source program, execution begins at

$$\mathsf{run}(P, H, e) = \langle P, \mathsf{newTop}(H, a_T), \epsilon \cdot \langle a_T, \epsilon\rangle, e\rangle \quad \text{if } \mathsf{TL}_P(e), \tag{9.21}$$

where a_T is fresh. A value with no pending top-level frame halts:

$$\frac{H(a).p = \mathsf{top}}{\langle P, H, \epsilon \cdot \langle a, \epsilon\rangle, v\rangle \longmapsto \langle P, H[a.q_a \mapsto \mathsf{false}], \epsilon, \mathsf{halt}(v)\rangle}. \quad (\textsc{Top-Halt})$$

Closures created during Equation (9.21) capture nil current instance and class through the ordinary closure-creation rule; the top activation remains their lexical activation record.

A compilation unit is source text paired with a language id:

$$U = \langle \lambda, u\rangle \in \mathit{LanguageId} \times \mathit{SourceText}. \tag{9.22}$$

Section 13 defines PEG recognition, language-specific elaboration, compilation by reified Newspeak objects, and the resulting $\langle P, e\rangle$. The current implementation's restriction of Equation (9.22) to one top-level class is a temporary accepted-syntax restriction, not a different dynamic semantics. Serialization must preserve the stable identities in P, the referenced atomic values, and all reachable heap records.

Finally, if $p = P.\mathit{platform}(\mathsf{platform})$ is the supplied platform object and r is the host's argument-list object, application entry is the top-level expression

$$\mathsf{entry}(o, p, r) = \mathsf{send}(o, \#\texttt{main:args:}, \langle p, r\rangle). \tag{9.23}$$

It is admitted exactly when

$$\mathsf{application}_{P,H}(o) \iff \exists f, C_f.\ P, H \vdash \operatorname*{lookup}_{pub}(\#\texttt{main:args:}, \operatorname*{classOf}_{H}(o)) = \langle f, C_f\rangle \ \wedge\ |f.\bar{x}| = 2. \tag{9.24}$$

Execution of Equation (9.23) is then Equation (9.21). Linking and module instantiation performed by that method are ordinary sends.

9.6 The exception-library boundary

Newspeak has no exception declaration, throw expression, catch expression, or handler stack in its core syntax. Signaling, catching, passing, returning, and resuming are library operations implemented with ordinary objects and reflective access to activations and continuations. Adding built-in handler rules here would therefore duplicate, and potentially contradict, that library semantics.

The VM can nevertheless detect failures such as those produced by Equations (8.9), (8.10), (9.15), and the failed return in RETURN-CANNOT. The platform supplies a total reification operation

$$\mathsf{reifyError}_P(H, \xi) = \langle H', x \rangle, \tag{9.25}$$

where H' extends H without changing an existing record, x is a near exception object representing ξ, and ordinary public dispatch of $\langle \#\mathsf{signal}, \epsilon \rangle$ to x is defined. Reification may allocate a fresh exception and its immutable diagnostic data; it may not invoke user code.

Both internal control terms cross the boundary in the same way:

$$\frac{\mathsf{reifyError}_P(H, \xi) = \langle H', x \rangle}{\begin{array}{l}\langle H; \mathcal{A} \cdot \langle a, \pi \rangle; \mathsf{runtimeError}(\xi) \rangle_P \\ \quad \longmapsto \langle H'; \mathcal{A} \cdot \langle a, \pi \rangle; \mathsf{dispatch}(\mathsf{ordinaryRequest}(x), \langle \#\mathsf{signal}, \epsilon \rangle) \rangle_P\end{array}} \quad \text{(RUNTIME-ERROR-SIGNAL)}$$

$$\frac{\mathsf{reifyError}_P(H, \xi) = \langle H', x \rangle}{\begin{array}{l}\langle H; \mathcal{A} \cdot \langle a, \pi \rangle; \mathsf{throw}(\xi) \rangle_P \\ \quad \longmapsto \langle H'; \mathcal{A} \cdot \langle a, \pi \rangle; \mathsf{dispatch}(\mathsf{ordinaryRequest}(x), \langle \#\mathsf{signal}, \epsilon \rangle) \rangle_P\end{array}} \quad \text{(THROW-SIGNAL)}$$

The original frames π remain in place. If the library resumes the exception, the signal send returns a value at the failure point; if it handles, passes, or returns from it, its ordinary sends and reflective continuation operations determine the transfer. In particular, no base rule silently discards sequence frames. The activation and stack-mirror operations in Equations (11.12), (11.61), and (11.65) provide the control primitives used by the standard library; they do not add a second notion of exception handling.

10 Actors and eventual references

This section lifts the sequential relation of Section 7 to a system of isolated, single-threaded actors. The lift does not make an asynchronous send an ordinary send in disguise: evaluation returns a promise immediately, the request becomes a mailbox item, and ordinary dispatch begins only when the target actor starts a later turn.

10.1 Global domains and isolation

Let the following identity sets be pairwise disjoint and disjoint from those in Equation (3.1):

$$p \in \mathit{PromiseId}, \qquad r \in \mathit{FarRefId}, \qquad u \in \mathit{EventId}, \qquad \mathit{Ref} = \mathit{ObjRef} \uplus \mathit{PromiseId} \uplus \mathit{FarRefId}. \tag{10.1}$$

Promises and far references are Newspeak objects with platform-supplied classes, but their routing state is recorded globally below. An ordinary send to such a handle invokes its local platform protocol; only asyncSend dereferences it for remote delivery.

An actor has a private heap H_A. A platform may additionally maintain a value heap V shared by actors managed by one virtual machine. The heap visible to the sequential machine running actor A is

$$\widehat{H}_A = V \uplus H_A, \qquad A \neq B \Longrightarrow \mathrm{dom}(H_A) \cap \mathrm{dom}(H_B) = \varnothing, \qquad \mathrm{dom}(V) \cap \mathrm{dom}(H_A) = \varnothing. \tag{10.2}$$

Only objects satisfying the value predicate of Section 9.4 may enter V. A distributed implementation may instead copy their complete immutable reachable graph. We write

$$\Omega \vdash \mathsf{valTransfer}_{A \Rightarrow B}(x) \Downarrow \langle \Omega', y \rangle \tag{10.3}$$

for either permitted choice; it is defined only for a fully initialized value x, preserves its class and immutable reachable graph up to identity isomorphism, and neither aliases nor modifies mutable actor-local state. Sharing is the identity case. The reflective layer will distinguish actors in one VM, which may share values and observe common code change, from actors in different VMs, which must use the copying case.

A far-reference table and promise table have the forms

$$\begin{aligned} &\Phi(r) = \langle A, o \rangle, \\ &\Pi(p) ::= \mathsf{pending}(A, W) \mid \mathsf{fulfilled}(A, x) \mid \mathsf{broken}(A, x), \\ &\quad W ::= \epsilon \mid W \cdot \langle B, \mu, p_r, \kappa \rangle. \end{aligned} \tag{10.4}$$

$\Phi(r) = \langle A, o \rangle$ means that r designates o in H_A. The actor in a promise state is the actor relative to whose heap its result is represented. Pending sends are retained in registration order; a waiter $\langle B, \mu, p_r, \kappa \rangle$ requests that, after settlement, actor B route μ to the result and settle p_r. The causal history κ is defined below.

Remote representation is an actor-relative relation $\Omega \vdash \mathsf{rr}_{A \Rightarrow B}(x) \Downarrow \langle \Omega', y \rangle$. It is the least relation given by the following rules:

$$\frac{A = B}{\Omega \vdash \mathsf{rr}_{A \Rightarrow B}(x) \Downarrow \langle \Omega, x \rangle} \quad \text{(RR-Same)}$$

$$\frac{A \neq B \quad \mathsf{value}_{P, \widehat{H}_A}(x) \quad \Omega \vdash \mathsf{valTransfer}_{A \Rightarrow B}(x) \Downarrow \langle \Omega', y \rangle}{\Omega \vdash \mathsf{rr}_{A \Rightarrow B}(x) \Downarrow \langle \Omega', y \rangle} \quad \text{(RR-Value)}$$

$$\frac{A \neq B \quad \Phi(r) = \langle B, o \rangle}{\Omega \vdash \mathsf{rr}_{A \Rightarrow B}(r) \Downarrow \langle \Omega, o \rangle} \quad \text{(RR-Far-Home)} \qquad \frac{A \neq B \quad \Phi(r) = \langle C, o \rangle \quad C \neq B}{\Omega \vdash \mathsf{rr}_{A \Rightarrow B}(r) \Downarrow \langle \Omega, r \rangle} \quad \text{(RR-Far-Pass)}$$

$$\frac{A \neq B \quad p \in \mathrm{dom}(\Pi)}{\Omega \vdash \mathsf{rr}_{A \Rightarrow B}(p) \Downarrow \langle \Omega, p \rangle} \quad \text{(RR-Promise)}$$

$$\frac{A \neq B \quad o \in \mathrm{dom}(H_A) \quad \neg \mathsf{value}_{P, \widehat{H}_A}(o) \quad r \text{ fresh} \quad \Omega' = \Omega[\Phi(r) \mapsto \langle A, o \rangle]}{\Omega \vdash \mathsf{rr}_{A \Rightarrow B}(o) \Downarrow \langle \Omega', r \rangle} \quad \text{(RR-Near-Far)}$$

Thus a far reference passed back to its target actor becomes the near referent, and a far reference passed to a third actor does not acquire a proxy chain. Promise identities are monotonic eventual capabilities and likewise pass without a proxy layer. Applying remote representation componentwise defines message transport:

$$\frac{\Omega_0 = \Omega \quad \Omega_{i-1} \vdash \mathsf{rr}_{A \Rightarrow B}(o_i) \Downarrow \langle \Omega_i, o_i' \rangle \ (1 \leq i \leq n)}{\Omega \vdash \mathsf{msgRR}_{A \Rightarrow B}(\langle s, \langle o_1, \ldots, o_n \rangle \rangle) \Downarrow \langle \Omega_n, \langle s, \langle o_1', \ldots, o_n' \rangle \rangle \rangle}. \tag{10.5}$$

The selector is a symbol value and is therefore unchanged.

10.2 Actor configurations and causal delivery

A mailbox or network packet is

$$\begin{aligned} b ::= {} & \mathsf{app}\langle u, A, B, o, \mu, p, \kappa\rangle \\ & \mid \mathsf{settlementPacket}\langle u, A, B, p, \chi, x, \kappa\rangle \\ & \mid \mathsf{wake}\langle u, A, B, p, p_r, \chi, x, \mu, \kappa_w, \kappa\rangle, \qquad \chi \in \{\mathsf{ok}, \mathsf{error}\}. \end{aligned} \tag{10.6}$$

The first two actor fields are source and destination. Every packet has a fresh event identity u and carries the source actor's causally prior event set κ. A wake names both the settled promise p that caused the wake and the dependent result promise p_r; it also retains κ_w, the history at which its dependent send was originally issued.

An actor is idle, is executing one sequential configuration on behalf of an application packet, or is suspended at a debugger safepoint:

$$\begin{aligned} \mathcal{X}(A) ::= {} & \mathsf{idle} \mid \mathsf{runningTurn}\langle \mathcal{A}, t, p, u\rangle \mid \\ & \mathsf{pausedTurn}\langle \mathcal{A}, t, p, u, r_s, d_s\rangle. \end{aligned} \tag{10.7}$$

Here $d_s \in \mathit{StopReason}$. The ordinary actor rules have no premise for $\mathsf{pausedTurn}$; deliveries may still append to its mailbox, but the actor executes no turn step until an authorized debugger operation resumes it. Section 11.2 defines suspension, stack replacement, and resumption. The global state is

$$\Omega = \langle P, V, \mathcal{H}, \mathcal{X}, \mathcal{Q}, \mathcal{N}, \Pi, \Phi, \Gamma, \mathcal{E}, \mathcal{D}, w, n\rangle, \qquad w \in \mathit{VMId}, \quad n \in \mathbb{N}. \tag{10.8}$$

w identifies the VM cohort and n is its committed program version. Every actor in $\mathrm{dom}(\mathcal{H})$ is managed by w and observes the same P and V. Section 11 changes P and n atomically. A system of multiple VMs is a map from distinct w to configurations of this form; its program components evolve independently. Γ_Ω, Π_Ω, and analogous subscripts name a component of a particular configuration when more than one configuration occurs in a rule; the subscript is omitted otherwise. $\mathcal{Q}(A)$ is a FIFO mailbox, $\mathcal{N}$ is the multiset of undelivered cross-actor packets, and $\Gamma(A)$ is a finite downward-closed causal history. The permanent event ledger records

$$\mathcal{E}(u) = \langle A, B, \kappa\rangle, \qquad \mathsf{dst}_\Omega(u) = B, \qquad \mathcal{D}(B) \in \mathit{EventId}^*. \tag{10.9}$$

$\mathcal{D}(B)$ lists, in arrival order, events delivered to B; a failed network event is marked retired at the same position. Processed event records are retained because later causal histories may still mention them.

All application and system packets are emitted by the helper

$$\begin{aligned} & \mathsf{emit}(\Omega, A, B, \beta, \kappa) = \Omega_2, \\ & u \text{ fresh}, \quad b = \beta(u, A, B, \kappa), \quad \Omega_1 = \Omega[\mathcal{E}(u) \mapsto \langle A, B, \kappa\rangle, \Gamma(A) \mapsto \Gamma(A) \cup \{u\}], \\ & \Omega_2 = \begin{cases} \Omega_1[\mathcal{Q}(B) \mapsto \mathcal{Q}(B) \cdot b, \mathcal{D}(B) \mapsto \mathcal{D}(B) \cdot u] & A = B, \\ \Omega_1[\mathcal{N} \mapsto \mathcal{N} \uplus \{b\}] & A \neq B. \end{cases} \end{aligned} \tag{10.10}$$

Here β is a packet template supplying every field except u, A, B, κ. A same-actor asynchronous send arrives immediately at the tail of its own mailbox, but cannot start until the current turn finishes.

A network packet b with identity u, destination B, and history κ is eligible precisely when

$$\mathsf{eligible}_\Omega(b) \quad \Longleftrightarrow \quad \{u' \in \kappa \mid \mathsf{dst}_\Omega(u') = B\} \subseteq \mathsf{set}(\mathcal{D}(B)). \tag{10.11}$$

An eligible packet may arrive:

$$\frac{b \in \mathcal{N} \quad \mathsf{id}(b) = u \quad \mathsf{destination}(b) = B \quad \mathsf{eligible}_{\Omega}(b)}{\Omega \longrightarrow_{\mathsf{act}} \Omega[\mathcal{N} \mapsto \mathcal{N} \setminus \{b\}, \mathcal{Q}(B) \mapsto \mathcal{Q}(B) \cdot b, \mathcal{D}(B) \mapsto \mathcal{D}(B) \cdot u]} \quad (\textsc{Network-Deliver})$$

Because mailboxes are FIFO and an actor runs at most one turn, delivery in this order implies processing in this order.

Equation (10.11) is a constructive form of E-order. After A_1 emits u_1 to A_2, $u_1 \in \Gamma(A_1)$, so a later u_2 from A_1 to A_2 carries u_1 and cannot arrive first. If the later packet instead passes a far reference for A_2 to A_3, delivery merges its history into $\Gamma(A_3)$; a subsequent send by A_3 to that referent therefore also carries u_1 and cannot arrive at A_2 first. These are exactly the two ordering clauses in Specification §5.6, generalized by transitive causal history.

Maximal executions are required to be weakly fair:

$$\mathsf{continuouslyEnabled}(\alpha, i) \Longrightarrow \exists j \geq i.\ \mathsf{taken}(\alpha, j), \tag{10.12}$$

for packet delivery, dequeue of the head of a nonempty idle mailbox, and an enabled step of a running actor. Fairness does not rescue an actor whose current turn diverges; its later mailbox entries may consequently remain unprocessed.

10.3 Asynchronous-send evaluation and routing

Extend the request and frame grammars in Equations (7.1) and (7.2) by

$$\theta ::= \cdots \mid \mathsf{eventualRequest}(x), \qquad F ::= \cdots \mid \mathsf{asyncRecv}(s, \bar{e}). \tag{10.13}$$

The receiver is evaluated first:

$$\langle P, H, \mathcal{A} \cdot \langle a, \pi \rangle, \mathsf{asyncSend}(e, s, \bar{e}) \rangle \longmapsto \langle P, H, \mathcal{A} \cdot \langle a, \pi \cdot \mathsf{asyncRecv}(s, \bar{e}) \rangle, e \rangle \quad (\textsc{Async-Receiver}).$$

After it yields x, the existing argument frames of Equation (7.6) evaluate the arguments from left to right:

$$\begin{aligned} \langle \pi \cdot \mathsf{asyncRecv}(s, \epsilon), x \rangle &\longmapsto \langle \pi, \mathsf{dispatch}(\mathsf{eventualRequest}(x), \langle s, \epsilon \rangle) \rangle, \\ \langle \pi \cdot \mathsf{asyncRecv}(s, \langle e_1, \bar{e} \rangle), x \rangle &\longmapsto \langle \pi \cdot \mathsf{args}(\mathsf{eventualRequest}(x), s, \epsilon, \bar{e}), e_1 \rangle. \end{aligned} \tag{10.14}$$

For a non-promise receiver, define its target by

$$\mathsf{target}_{\Omega,A}(x) = \begin{cases} \langle A, x \rangle & x \in \mathrm{dom}(\widehat{H}_A), \\ \langle B, o \rangle & \Phi(x) = \langle B, o \rangle. \end{cases} \tag{10.15}$$

The routing judgment $\Omega, A \vdash \mathsf{route}(x, \mu, p_r, \kappa) \Downarrow \Omega'$ is defined by the following four cases. A known target transports the arguments and emits an application packet:

$$\frac{\begin{array}{c} \mathsf{target}_{\Omega,A}(x) = \langle B, o \rangle \\ \Omega \vdash \mathsf{msgRR}_{A \Rightarrow B}(\mu) \Downarrow \langle \Omega_1, \mu' \rangle \\ \Omega' = \mathsf{emit}(\Omega_1, A, B, \lambda u, A, B, \kappa.\mathsf{app}\langle u, A, B, o, \mu', p_r, \kappa \rangle, \kappa) \end{array}}{\Omega, A \vdash \mathsf{route}(x, \mu, p_r, \kappa) \Downarrow \Omega'} \quad (\textsc{Route-Target})$$

An unresolved promise records the send without blocking its actor:

$$\frac{\Pi(p) = \mathsf{pending}(C, W) \quad \Omega' = \Omega[\Pi(p) \mapsto \mathsf{pending}(C, W \cdot \langle A, \mu, p_r, \kappa \rangle)]}{\Omega, A \vdash \mathsf{route}(p, \mu, p_r, \kappa) \Downarrow \Omega'} \quad (\textsc{Route-Pending})$$

For a fulfilled promise, its result is first represented in the requesting actor and routing repeats:

$$\frac{\Pi(p) = \mathsf{fulfilled}(C, x) \quad \Omega \vdash \mathsf{rr}_{C \Rightarrow A}(x) \Downarrow \langle \Omega_1, y \rangle \quad \Omega_1, A \vdash \mathsf{route}(y, \mu, p_r, \kappa) \Downarrow \Omega'}{\Omega, A \vdash \mathsf{route}(p, \mu, p_r, \kappa) \Downarrow \Omega'} \quad \text{(Route-Fulfilled)}$$

For a broken promise, the result promise is broken with the same represented exception:

$$\frac{\begin{array}{c}\Pi(p) = \mathsf{broken}(C, x) \\ \Omega \vdash \mathsf{rr}_{C \Rightarrow A}(x) \Downarrow \langle \Omega_1, y \rangle \\ \Omega_1, A \vdash \mathsf{settlePromise}(p_r, \mathsf{error}, A, y, \kappa) \Downarrow \Omega'\end{array}}{\Omega, A \vdash \mathsf{route}(p, \mu, p_r, \kappa) \Downarrow \Omega'} \quad \text{(Route-Broken)}$$

The last rule makes explicit the otherwise unspecified meaning of sends to a broken promise: break propagation is the conventional eventual-send behavior, but is recorded as a language-design choice in the coverage ledger.

Routing is an actor-level action. The asynchronous expression itself allocates its result promise and immediately leaves that promise as the value of the expression:

$$\frac{\begin{array}{c}\mathcal{X}(A) = \mathsf{runningTurn}\langle \mathcal{A} \cdot \langle a, \pi \rangle, \mathsf{dispatch}(\mathsf{eventualRequest}(x), \mu), p_0, u_0 \rangle \\ p_r \text{ fresh} \qquad \Omega_0 = \Omega[\Pi(p_r) \mapsto \mathsf{pending}(A, \epsilon)] \\ \Omega_0, A \vdash \mathsf{route}(x, \mu, p_r, \Gamma(A)) \Downarrow \Omega_1\end{array}}{\begin{array}{c}\Omega \longrightarrow_{\mathsf{act}} \Omega_1[\mathcal{X}(A) \mapsto \mathsf{runningTurn}\langle \mathcal{A} \cdot \langle a, \pi \rangle, \\ p_r, p_0, u_0 \rangle]\end{array}} \quad \text{(Async-Return)}$$

The displayed run record in the conclusion abbreviates replacement of its control component by p_r; its reply promise and triggering event remain p_0, u_0.

10.4 Turns and promise settlement

When an application packet reaches the head of an idle actor's mailbox, it starts a turn. With fresh top activation a_T and $H' = \mathsf{newTop}(\widehat{H}_B, a_T)$ from Equation (9.20):

$$\frac{\mathcal{X}(B) = \mathsf{idle} \quad \mathcal{Q}(B) = \mathsf{app}\langle u, A, B, o, \mu, p, \kappa \rangle \cdot Q}{\begin{array}{c}\Omega \longrightarrow_{\mathsf{act}} \Omega[\mathcal{X}(B) \mapsto \mathsf{runningTurn}\langle \epsilon \cdot \langle a_T, \epsilon \rangle, \mathsf{dispatch}(\mathsf{ordinaryRequest}(o), \mu), p, u \rangle, \\ \mathcal{Q}(B) \mapsto Q, \Gamma(B) \mapsto \Gamma(B) \cup \kappa \cup \{u\}, \mathcal{H}(B) \mapsto H' \setminus V]\end{array}} \quad \text{(Turn-Start)}$$

While B is running, each applicable sequential step is lifted pointwise:

$$\frac{\langle P, \widehat{H}_B, \mathcal{A}, t \rangle \longmapsto \langle P, H', \mathcal{A}', t' \rangle}{\Omega \longrightarrow_{\mathsf{act}} \Omega[\mathcal{H}(B) \mapsto H' \setminus V, \mathcal{X}(B) \mapsto \mathsf{runningTurn}\langle \mathcal{A}', t', p, u \rangle]} \quad \text{(Actor-Sequential)}$$

It is inapplicable to the eventual dispatch intercepted by Async-Return. No second packet can start in B until this turn ends.

Settlement wakes every recorded dependent send. Define the ordered helper

$$\begin{aligned}\mathsf{wakeAll}(\Omega, B, p, \chi, y, \epsilon) &= \Omega, \\ \mathsf{wakeAll}(\Omega, B, p, \chi, y, \langle C, \mu, p_r, \kappa_w \rangle \cdot W) &= \mathsf{wakeAll}(\Omega_1, B, p, \chi, y, W), \\ \Omega_1 &= \mathsf{emit}(\Omega, B, C, \lambda u, B, C, \kappa.\mathsf{wake}\langle u, B, C, p, p_r, \chi, y, \mu, \kappa_w, \kappa \rangle, \Gamma(B)).\end{aligned} \tag{10.16}$$

Earlier waiters for the same actor are emitted first and therefore ordered by Equation (10.11). Promise settlement is the partial judgment

$$\frac{\begin{array}{c}\Pi(p) = \mathsf{pending}(B, W) \qquad \kappa \subseteq \Gamma(B) \\ \Omega \vdash \mathsf{rr}_{A \Rightarrow B}(x) \Downarrow \langle \Omega_0, y \rangle \\ \Omega_1 = \begin{cases}\Omega_0[\Pi(p) \mapsto \mathsf{fulfilled}(B, y)] & \chi = \mathsf{ok}, \\ \Omega_0[\Pi(p) \mapsto \mathsf{broken}(B, y)] & \chi = \mathsf{error}\end{cases} \qquad \Omega' = \mathsf{wakeAll}(\Omega_1, B, p, \chi, y, W)\end{array}}{\Omega, B \vdash \mathsf{settlePromise}(p, \chi, A, x, \kappa) \Downarrow \Omega'}. \tag{10.17}$$

There is no rule for settling an already settled promise, so fulfillment and breaking are mutually exclusive and occur at most once.

When a turn returns normally, a promise owned by the same actor settles immediately; a remote owner's settlement is sent subsequently:

$$\frac{\mathcal{X}(B) = \mathsf{runningTurn}\langle \epsilon, \mathsf{halt}(v), p, u\rangle \qquad \Pi(p) = \mathsf{pending}(B, W) \qquad \Omega, B \vdash \mathsf{settlePromise}(p, \mathsf{ok}, B, v, \Gamma(B)) \Downarrow \Omega'}{\Omega \longrightarrow_{\mathsf{act}} \Omega'[\mathcal{X}(B) \mapsto \mathsf{idle}]} \quad \text{(Turn-Fulfill-Local)}$$

$$\frac{\mathcal{X}(B) = \mathsf{runningTurn}\langle \epsilon, \mathsf{halt}(v), p, u\rangle \qquad \Pi(p) = \mathsf{pending}(A, W) \qquad A \neq B \qquad \Omega' = \mathsf{emit}(\Omega, B, A, \lambda u, B, A, \kappa.\mathsf{settlementPacket}\langle u, B, A, p, \mathsf{ok}, v, \kappa\rangle, \Gamma(B))}{\Omega \longrightarrow_{\mathsf{act}} \Omega'[\mathcal{X}(B) \mapsto \mathsf{idle}]} \quad \text{(Turn-Fulfill-Remote)}$$

If exception signaling escapes the turn's top activation, the exception-library boundary produces $\mathsf{abort}(x)$ instead of $\mathsf{halt}(v)$. A local owner's promise is broken immediately:

$$\frac{\mathcal{X}(B) = \mathsf{runningTurn}\langle \epsilon, \mathsf{abort}(x), p, u\rangle \qquad \Pi(p) = \mathsf{pending}(B, W) \qquad \Omega, B \vdash \mathsf{settlePromise}(p, \mathsf{error}, B, x, \Gamma(B)) \Downarrow \Omega'}{\Omega \longrightarrow_{\mathsf{act}} \Omega'[\mathcal{X}(B) \mapsto \mathsf{idle}]} \quad \text{(Turn-Break-Local)}$$

A remote owner receives an error settlement subsequently:

$$\frac{\mathcal{X}(B) = \mathsf{runningTurn}\langle \epsilon, \mathsf{abort}(x), p, u\rangle \qquad \Pi(p) = \mathsf{pending}(A, W) \qquad A \neq B \qquad \Omega' = \mathsf{emit}(\Omega, B, A, \lambda u, B, A, \kappa.\mathsf{settlementPacket}\langle u, B, A, p, \mathsf{error}, x, \kappa\rangle, \Gamma(B))}{\Omega \longrightarrow_{\mathsf{act}} \Omega'[\mathcal{X}(B) \mapsto \mathsf{idle}]} \quad \text{(Turn-Break-Remote)}$$

These rules are the only additional obligation placed on the resumable exception library by actors.

An idle promise owner processes a settlement packet at the head of its FIFO mailbox by merging the packet history and applying Equation (10.17):

$$\frac{\mathcal{X}(B) = \mathsf{idle} \qquad \mathcal{Q}(B) = \mathsf{settlementPacket}\langle u, A, B, p, \chi, x, \kappa\rangle \cdot Q \qquad \Omega_0 = \Omega[\mathcal{Q}(B) \mapsto Q, \Gamma(B) \mapsto \Gamma(B) \cup \kappa \cup \{u\}] \qquad \Omega_0, B \vdash \mathsf{settlePromise}(p, \chi, A, x, \Gamma_0(B)) \Downarrow \Omega'}{\Omega \longrightarrow_{\mathsf{act}} \Omega'} \quad \text{(Settlement-Packet-Dequeue)}$$

A fulfilled wake represents the result in the waiting actor and resumes the stored routing operation; a broken wake settles the dependent promise as broken:

$$\frac{\mathcal{X}(C) = \mathsf{idle} \qquad \mathcal{Q}(C) = \mathsf{wake}\langle u, B, C, p, p_r, \mathsf{ok}, x, \mu, \kappa_w, \kappa\rangle \cdot Q \qquad \Omega_0 = \Omega[\mathcal{Q}(C) \mapsto Q, \Gamma(C) \mapsto \Gamma(C) \cup \kappa \cup \kappa_w \cup \{u\}] \qquad \Omega_0 \vdash \mathsf{rr}_{B \Rightarrow C}(x) \Downarrow \langle \Omega_1, y\rangle \qquad \Omega_1, C \vdash \mathsf{route}(y, \mu, p_r, \Gamma_1(C)) \Downarrow \Omega'}{\Omega \longrightarrow_{\mathsf{act}} \Omega'} \quad \text{(Wake-Fulfilled)}$$

$$\frac{\mathcal{X}(C) = \mathsf{idle} \qquad \mathcal{Q}(C) = \mathsf{wake}\langle u, B, C, p, p_r, \mathsf{error}, x, \mu, \kappa_w, \kappa\rangle \cdot Q \qquad \Omega_0 = \Omega[\mathcal{Q}(C) \mapsto Q, \Gamma(C) \mapsto \Gamma(C) \cup \kappa \cup \kappa_w \cup \{u\}] \qquad \Omega_0, C \vdash \mathsf{settlePromise}(p_r, \mathsf{error}, B, x, \Gamma_0(C)) \Downarrow \Omega'}{\Omega \longrightarrow_{\mathsf{act}} \Omega'} \quad \text{(Wake-Broken)}$$

These system packets are handled only between user turns, so remote settlement cannot mutate an actor's heap or promise observations during an executing turn. A communication subsystem may

retire an undelivered application event and emit an error settlement instead of delivering it. Its failure object must satisfy the same remote-representation boundary, the event is entered in $\mathcal{D}$ as retired so it cannot block causal successors, and delivery and failure of the same event are mutually exclusive.

10.5 Actor creation

Actor creation is a platform primitive rather than a new surface expression. Elaboration supplies, for a top-level mixin, the class-side mixin and factory descriptor needed to apply it to Object:

$$P.\mathit{actorSeed}(M) = \langle D, M_c, q\rangle \Longrightarrow \operatorname*{decl}_P(M) = D \ \wedge \ P.\mathit{owner}(D) = \mathsf{topOwner}. \tag{10.18}$$

Let $\mathsf{base}_P(B)$ be a fresh actor's platform heap, containing the distinguished classes and canonical values but no user-created mutable object. For fresh B, C, C_m, r, actor creation is

$$\frac{\begin{array}{c} P.\mathit{actorSeed}(M) = \langle D, M_c, q\rangle \qquad H_0 = \mathsf{base}_P(B) \\ H_1 = \mathsf{allocClass}(H_0, C, C_m, \mathsf{actor}(D), M, P.\mathit{platform}(\mathsf{Object}), \mathsf{nil}, M_c, q) \\ \Omega' = \Omega[\mathcal{H}(B) \mapsto H_1, \mathcal{X}(B) \mapsto \mathsf{idle}, \mathcal{Q}(B) \mapsto \epsilon, \Gamma(B) \mapsto \varnothing, \\ \qquad\qquad \mathcal{D}(B) \mapsto \epsilon, \Phi(r) \mapsto \langle B, C\rangle] \end{array}}{\Omega, A \vdash \mathsf{spawn}(M) \Downarrow \langle \Omega', r\rangle}. \tag{10.19}$$

The result is explicitly a far reference to the new class, as required by the Specification. The $\mathsf{actor}(D)$ provenance distinguishes that class from evaluation of the top-level class expression itself, so it is not accidentally classified as a module definition by Equation (9.17). Its primary factory can then be invoked asynchronously in the ordinary way.

11 Mirrors and reflective program change

This section refines the schematic transition in Equation (1.1). It deliberately specifies a semantic mirror protocol rather than a concrete library API: the Specification requires mirror-based capability control, transactional change, object-class change, and activation access, but explicitly leaves the VM-mirror selector set open. An implementation may choose selectors freely provided that each operation below is reached only through the corresponding mirror capability.

11.1 VM cohorts, mirror objects, and authority

One actor configuration in Equation (10.8) is one VM cohort. Define its complete object store by

$$\mathsf{vmStore}(\Omega) = V \uplus \biguplus_{A \in \mathrm{dom}(\mathcal{H})} \mathcal{H}(A). \tag{11.1}$$

The disjointness condition in Equation (10.2) makes this a map. Write $\mathsf{ownerActor}_\Omega(x) = A$ exactly when $x \in \mathrm{dom}(\mathcal{H}(A))$, and write $\mathsf{ownerActor}_\Omega(x) = \mathsf{shared}$ exactly when $x \in \mathrm{dom}(V)$. These two cases are disjoint.

A mirror target and a reflective right are drawn from

$$\begin{aligned}\zeta ::= {} & \mathsf{vmTarget}(w) \mid \mathsf{programTarget} \mid \mathsf{mixinTarget}(M) \mid \mathsf{methodTarget}(i_f) \\ & \mid \mathsf{classTarget}(C) \mid \mathsf{objectTarget}(o) \mid \mathsf{activationTarget}(a) \mid \mathsf{actorTarget}(A), \\ \varrho \in \mathit{ReflectRight} = {} & \{\mathsf{inspect}, \mathsf{editCode}, \mathsf{editClass}, \mathsf{reclassifyObject}, \\ & \quad \mathsf{editObjectSlots}, \mathsf{editActivation}, \mathsf{controlContinuation}, \\ & \quad \mathsf{controlActorStack}, \\ & \quad \mathsf{primitive}, \mathsf{delegate}\}.\end{aligned} \tag{11.2}$$

Extend the heap-record grammar of Equation (3.10) by

$$H(g) = \mathsf{mirror}\langle C_g, w, \zeta, R\rangle, \qquad R \in \mathcal{P}_{\mathrm{fin}}(\mathit{ReflectRight}), \tag{11.3}$$

where C_g is a platform mirror class. Thus $\mathrm{classOf}_H(g) = C_g$. Mirror identities are fresh and unforgeable. A mirror is actor-local mutable state, never a value; ordinary cross-actor transfer therefore yields a far reference under Rules RR-Near-Far and RR-Far-Pass. Possession of such a reference can still confer authority, because the eventual request executes in the mirror's owning actor.

Target containment is the least reflexive relation $\sqsubseteq_{P,H,w}$ satisfying

$$\begin{aligned} \zeta &\sqsubseteq_{P,H,w} \mathsf{vmTarget}(w), \\ \mathsf{mixinTarget}(M) &\sqsubseteq_{P,H,w} \mathsf{programTarget}, \\ \mathsf{methodTarget}(i_f) &\sqsubseteq_{P,H,w} \mathsf{mixinTarget}(M) && \text{if } \mathsf{methodOwner}_P(i_f) = M, \\ \mathsf{classTarget}(C) &\sqsubseteq_{P,H,w} \mathsf{mixinTarget}(\underset{H}{\mathrm{mixin}}(C)), \\ \mathsf{objectTarget}(o) &\sqsubseteq_{P,H,w} \mathsf{classTarget}(\underset{H}{\mathrm{classOf}}(o)), \\ \mathsf{activationTarget}(a) &\sqsubseteq_{P,H,w} \mathsf{objectTarget}(H(a).o). \end{aligned} \tag{11.4}$$

The first, generic clause places an actor target occurring in this cohort below its VM target. An actor target does not contain the actor's whole heap: activation and object mirrors remain separate capabilities. It authorizes only the stack operations explicitly requiring controlActorStack below. The method owner used here is the partial function

$$\mathsf{methodOwner}_P(i_f) = M \iff \exists s, f.\ \underset{P}{\mathrm{methods}}(M)(s) = f \wedge \mathsf{methodId}(f) = i_f. \tag{11.5}$$

Uniqueness follows from stable method identities. VM containment includes all targets in Equation (11.2) that occur in the store of that VM; it does not include any target belonging to another VM.

The platform may create an initial VM mirror only as part of the authority explicitly supplied to an application. There is no operation that obtains it from an ambient global. A holder may attenuate, but not amplify, authority:

$$\frac{\begin{array}{c} H_A(g) = \mathsf{mirror}\langle C_g, w, \zeta, R\rangle \qquad \mathsf{delegate} \in R \\ \zeta' \sqsubseteq_{P,\widehat{H}_A,w} \zeta \qquad R' \subseteq R \qquad g' \text{ fresh} \\ H'_A = H_A[g' \mapsto \mathsf{mirror}\langle C_g, w, \zeta', R'\rangle] \end{array}}{\Omega, A, g \vdash \mathsf{deriveMirror}(\zeta', R') \Downarrow \langle \Omega[\mathcal{H}(A) \mapsto H'_A], g'\rangle} \quad (\textsc{Mirror-Derive})$$

Access modifiers play no role in this rule. A private method is visible to a method or enclosing-mixin mirror precisely because the mirror, rather than a source-level send, is the authority check.

11.2 Reified control and stack images

The sequential notation of Equation (7.3) stores the active term once, beside the activation stack. Debugging requires a uniform per-frame view. Let $b_c \in \mathit{CodeImageId}$ name an immutable source, bytecode, or native-code image and define a control location by

$$\lambda_c ::= \mathsf{sourcePoint}(z) \mid \mathsf{machinePoint}(b_c, n_c), \qquad n_c \in \mathbb{N}. \tag{11.6}$$

Introduce the internal waiting term $\mathsf{awaitResult}$ and the control-image domain

$$\kappa ::= \mathsf{activeControl}\langle \pi, t\rangle \mid \mathsf{suspendedControl}\langle \pi\rangle. \tag{11.7}$$

The current term and source-level control location of a frame are the total functions

$$\begin{aligned} \mathsf{currentTerm}(\mathsf{activeControl}\langle \pi, t\rangle) &= t, \\ \mathsf{controlLocation}(\mathsf{activeControl}\langle \pi, t\rangle) &= \mathsf{site}(\pi, t), \\ \mathsf{currentTerm}(\mathsf{suspendedControl}\langle \pi\rangle) &= \mathsf{awaitResult}, \\ \mathsf{controlLocation}(\mathsf{suspendedControl}\langle \pi\rangle) &= \mathsf{resumeSite}(\pi). \end{aligned} \tag{11.8}$$

site and $\mathsf{resumeSite}$ are the control-map functions emitted by elaboration or compilation and return a λ_c. A source evaluator normally returns $\mathsf{sourcePoint}$; a bytecode or native implementation returns $\mathsf{machinePoint}$, whose second component is its actual program counter. Materialized terms and frames are not changed by a later code install, so the map for a live or debugger-retained frame must be retained as long as that frame is resumable.

For $\mathcal{A} = \langle a_1, \pi_1\rangle \cdots \langle a_m, \pi_m\rangle$ with $m > 0$, define its canonical control image by

$$\begin{aligned} \mathsf{controlImage}(\mathcal{A}, t) = &\langle a_1, \mathsf{suspendedControl}\langle \pi_1\rangle\rangle \cdots \\ &\langle a_{m-1}, \mathsf{suspendedControl}\langle \pi_{m-1}\rangle\rangle \cdot \langle a_m, \mathsf{activeControl}\langle \pi_m, t\rangle\rangle. \end{aligned} \tag{11.9}$$

Thus every live frame has a current term: a lower frame is explicitly waiting for its callee's result at the location determined by its continuation stack. The inverse partial function is

$$\mathsf{decodeControlImage}(\mathsf{controlImage}(\mathcal{A}, t)) = \langle \mathcal{A}, t\rangle. \tag{11.10}$$

It is undefined for an empty image, for an active frame below the top, or for a suspended top frame.

A reified frame contains both the activation record and its control image:

$$\varphi = \mathsf{frameImage}\langle a, H(a), \kappa\rangle. \tag{11.11}$$

The actor-stack snapshot is the immutable value

$$\begin{aligned} \mathsf{actorStackSnapshot}_\Omega(B) \simeq \langle \mathsf{running}, \mathsf{reifyFrames}_U(\mathsf{controlImage}(\mathcal{A}, t))\rangle \\ \qquad \text{if } \mathcal{X}(B) = \mathsf{runningTurn}\langle \mathcal{A}, t, p, u\rangle, \\ \mathsf{actorStackSnapshot}_\Omega(B) \simeq \langle \mathsf{paused}(r_s, d_s), \mathsf{reifyFrames}_U(\mathsf{controlImage}(\mathcal{A}, t))\rangle \\ \qquad \text{if } \mathcal{X}(B) = \mathsf{pausedTurn}\langle \mathcal{A}, t, p, u, r_s, d_s\rangle, \end{aligned} \tag{11.12}$$

where $U = \mathsf{vmStore}(\Omega)$ and $\mathsf{reifyFrames}$ replaces every $\langle a, \kappa\rangle$ by Equation (11.11). It is undefined for an idle actor. The paused form exposes the unforgeable token that must accompany any later paused-stack mutation.

The unique live control of an activation is

$$\begin{aligned} \mathsf{liveControl}_\Omega(a) = \kappa \iff &\exists B, \bar\varphi_0, \bar\varphi_1, R. \\ &\mathsf{actorStackSnapshot}_\Omega(B) = \langle m, \bar\varphi_0 \cdot \langle \mathsf{frameImage}\langle a, R, \kappa\rangle\rangle \cdot \bar\varphi_1\rangle. \end{aligned} \tag{11.13}$$

Uniqueness follows from actor ownership and the prohibition on duplicate live activation identities. The function is undefined for a retained activation that is no longer on a stack.

Editable stack templates. A debugger constructs an immutable template before requesting an atomic install. Template-local keys and frames are

$$\eta ::= \mathsf{retain}(a) \mid \mathsf{freshFrame}(u_f), \qquad \psi = \mathsf{frameTemplate}\langle \eta, R, \kappa \rangle, \qquad \mathcal{S} = \langle \psi_1, \ldots, \psi_m \rangle. \tag{11.14}$$

R is a complete activation-record template. Its continuation, home, and lexical-environment activation references may name other keys in the same $\mathcal{S}$. A retained key preserves an existing activation identity; a fresh-frame key allocates a new identity at installation. Consequently a copied frame is never made live twice under one activation identity.

The three basic stack-editing operations have distinct names. First, promote and demote control without overloading either operation:

$$\begin{aligned} &\mathsf{promoteFrame}(\mathsf{frameTemplate}\langle \eta, R, \mathsf{suspendedControl}\langle \pi \rangle \rangle, v) \\ &\qquad = \mathsf{frameTemplate}\langle \eta, R, \mathsf{activeControl}\langle \pi, v \rangle \rangle, \\ &\mathsf{demoteFrame}(\mathsf{frameTemplate}\langle \eta, R, \mathsf{activeControl}\langle \pi, t \rangle \rangle) \\ &\qquad = \mathsf{frameTemplate}\langle \eta, R, \mathsf{suspendedControl}\langle \pi \rangle \rangle. \end{aligned} \tag{11.15}$$

Popping j frames supplies the chosen continuation value to the new top:

$$\mathsf{popStackFrames}(\mathcal{S}_0 \cdot \langle \psi \rangle \cdot \mathcal{S}_1, |\mathcal{S}_1|, v) = \mathcal{S}_0 \cdot \langle \mathsf{promoteFrame}(\psi, v) \rangle, \qquad |\mathcal{S}_1| < |\mathcal{S}|. \tag{11.16}$$

Pushing a prepared active frame makes the old top wait for the pushed frame's result:

$$\mathsf{pushStackFrame}(\mathcal{S}_0 \cdot \langle \psi \rangle, \psi') = \mathcal{S}_0 \cdot \langle \mathsf{demoteFrame}(\psi), \psi' \rangle, \quad \mathsf{currentTerm}(\psi'.\kappa) \neq \mathsf{awaitResult}. \tag{11.17}$$

Finally, copying a contiguous segment allocates fresh template keys and rewires every stack-structural activation link internal to the copied segment:

$$\begin{aligned} \mathsf{copyStackFrameSegment}(\mathcal{S}, i, j, \bar{u}) &= \mathsf{rename}_\nu(\langle \psi_i, \ldots, \psi_j \rangle), \\ \nu(\psi_k.\eta) &= \mathsf{freshFrame}(u_k) \qquad (i \le k \le j), \qquad |\bar{u}| = j - i + 1. \end{aligned} \tag{11.18}$$

Continuation, home, and lexical-environment links outside the copied segment remain references to their original retained activations. Parameter and local values are shallow-copied; an activation object stored there is not implicitly retargeted. A debugger may concatenate the returned segment at any position, edit its records or controls, and submit the complete result for validation. These pure operations therefore express push, pop, single-frame copy, and multi-frame copy without making a partially edited live stack observable.

11.3 Inspection

Mirror queries are

$$\begin{aligned} q_r ::= {} & \mathsf{programVersionQuery} \mid \mathsf{mixinDescriptorQuery}(M) \\ & \mid \mathsf{methodDescriptorQuery}(i_f) \mid \mathsf{classDescriptorQuery}(C) \\ & \mid \mathsf{objectDescriptorQuery}(o) \mid \mathsf{activationDescriptorQuery}(a) \\ & \mid \mathsf{activationControlQuery}(a) \mid \mathsf{currentActivationQuery}(A) \\ & \mid \mathsf{actorStackQuery}(A) \mid \mathsf{allInstancesOfQuery}(C) \\ & \mid \mathsf{mixinApplicationsQuery}(M). \end{aligned} \tag{11.19}$$

For a running actor, the current activation is

$$\begin{aligned}\mathsf{currentActivation}_\Omega(A) = a \quad \Longleftrightarrow \quad & \mathcal{X}(A) = \mathsf{runningTurn}\langle \mathcal{A}_0 \cdot \langle a, \pi\rangle, t, p, u\rangle \quad \vee \\ & \mathcal{X}(A) = \mathsf{pausedTurn}\langle \mathcal{A}_0 \cdot \langle a, \pi\rangle, t, p, u, r_s, d_s\rangle.\end{aligned} \tag{11.20}$$

It is undefined for an idle actor. Descriptor construction is given by the following separate functions, each of which returns an immutable value object:

$$\begin{aligned}\mathsf{describeMixin}_P(M) &\simeq \left\langle \begin{array}{c} \underset{P}{\mathrm{decl}}(M), \underset{P}{\mathrm{methods}}(M), \mathsf{ownSlots}_P(M), \\ P.mixin(M).nestedDecls, \iota_P(M) \end{array} \right\rangle, \\ \mathsf{describeMethod}_P(i_f) &\simeq \langle M, f, P.body(i_f)\rangle, \\ &\qquad \text{where } M = \mathsf{methodOwner}_P(i_f), \quad \underset{P}{\mathrm{methods}}(M)(\mathrm{selector}(f)) = f, \\ \mathsf{describeClass}_H(C) &\simeq \left\langle \begin{array}{c} \underset{H}{\mathrm{mixin}}(C), \underset{H}{\mathrm{super}}(C), H(C).eo, \\ H(C).C_m, H(C).q, H(C).\chi_C \end{array} \right\rangle, \\ \mathsf{describeObject}_H(o) &\simeq \langle \underset{H}{\mathrm{classOf}}(o), H(o).\sigma, H(o).\eta\rangle, \\ \mathsf{describeActivation}_H(a) &\simeq \left\langle \begin{array}{c} H(a).p, H(a).\rho, H(a).\ell, H(a).o, H(a).C, \\ H(a).k, H(a).h, H(a).q_a, H(a).\gamma \end{array} \right\rangle.\end{aligned} \tag{11.21}$$

The complete debugger-facing activation descriptor adds its current control when the activation is live:

$$\mathsf{describeDebugActivation}_\Omega(a) \simeq \langle \mathsf{describeActivation}_{\mathsf{vmStore}(\Omega)}(a), \mathsf{option}(\mathsf{liveControl}_\Omega(a))\rangle. \tag{11.22}$$

option is none when Equation (11.13) is undefined. Thus retained dead activations remain inspectable, while every live activation exposes both its current term and the source/PC location of Equation (11.8). The symbol $\simeq$ means an immutable, recursively reified snapshot, not a mutable alias to a metalevel record. In particular, a continuation component is the activation identity specified in Section 3, reified as an activation mirror only if the caller has authority for that target.

Fix an implementation-chosen total order $\prec$ on the object identities of one VM cohort. The exact-instance query returns a snapshot sequence, not a live view:

$$\mathsf{allInstancesOf}_\Omega(C) = \mathsf{sort}_\prec\{x \in \mathrm{dom}(\mathsf{vmStore}(\Omega)) \mid \underset{\mathsf{vmStore}(\Omega)}{\mathrm{classOf}}(x) = C\}. \tag{11.23}$$

Thus subclasses are excluded. Presence in the VM store is the abstract notion of liveness. The optional garbage-collection layer makes this precise by removing eligible identities from that store; without that layer, the query observes every exact instance allocated so far. The order is deliberately a platform choice because the mirror protocol promises no allocation order.

The applications of a mixin are the live class objects whose class records name that exact static mixin identity:

$$\mathsf{mixinApplications}_\Omega(M) = \mathsf{sort}_\prec\{C \in \mathrm{dom}(\mathsf{vmStore}(\Omega)) \mid \mathsf{isClass}_{\mathsf{vmStore}(\Omega)}(C) \wedge \underset{\mathsf{vmStore}(\Omega)}{\mathrm{mixin}}(C) = M\}. \tag{11.24}$$

This is an exact-mixin query: applications of other mixins and ordinary instances of the resulting classes are excluded. A metaclass is included when its class record applies M, so a class-side mixin is treated uniformly with an instance-side mixin. As with Equation (11.23), the result is a snapshot in the fixed order $\prec$. Working Specification 0.109 requires both population queries.

For $\Omega = \langle P, V, \mathcal{H}, \mathcal{X}, \mathcal{Q}, \mathcal{N}, \Pi, \Phi, \Gamma, \mathcal{E}, \mathcal{D}, w, n\rangle$, put $U = \mathsf{vmStore}(\Omega)$. The authority target of every query is

$$\begin{aligned}
\mathsf{queryTarget}_{P,U,w}(\mathsf{programVersionQuery}) &= \mathsf{vmTarget}(w),\\
\mathsf{queryTarget}_{P,U,w}(\mathsf{mixinDescriptorQuery}(M)) &= \mathsf{mixinTarget}(M),\\
\mathsf{queryTarget}_{P,U,w}(\mathsf{methodDescriptorQuery}(i_f)) &= \mathsf{methodTarget}(i_f),\\
\mathsf{queryTarget}_{P,U,w}(\mathsf{classDescriptorQuery}(C)) &= \mathsf{classTarget}(C),\\
\mathsf{queryTarget}_{P,U,w}(\mathsf{objectDescriptorQuery}(o)) &= \mathsf{objectTarget}(o),\\
\mathsf{queryTarget}_{P,U,w}(\mathsf{activationDescriptorQuery}(a)) &= \mathsf{activationTarget}(a),\\
\mathsf{queryTarget}_{P,U,w}(\mathsf{activationControlQuery}(a)) &= \mathsf{activationTarget}(a),\\
\mathsf{queryTarget}_{P,U,w}(\mathsf{currentActivationQuery}(A)) &= \mathsf{actorTarget}(A),\\
\mathsf{queryTarget}_{P,U,w}(\mathsf{actorStackQuery}(A)) &= \mathsf{actorTarget}(A),\\
\mathsf{queryTarget}_{P,U,w}(\mathsf{allInstancesOfQuery}(C)) &= \mathsf{classTarget}(C),\\
\mathsf{queryTarget}_{P,U,w}(\mathsf{mixinApplicationsQuery}(M)) &= \mathsf{mixinTarget}(M).
\end{aligned} \tag{11.25}$$

Its result is the corresponding immutable snapshot:

$$\begin{aligned}
\mathsf{queryResult}_{\Omega}(\mathsf{programVersionQuery}) &= n,\\
\mathsf{queryResult}_{\Omega}(\mathsf{mixinDescriptorQuery}(M)) &= \mathsf{describeMixin}_{P}(M),\\
\mathsf{queryResult}_{\Omega}(\mathsf{methodDescriptorQuery}(i_f)) &= \mathsf{describeMethod}_{P}(i_f),\\
\mathsf{queryResult}_{\Omega}(\mathsf{classDescriptorQuery}(C)) &= \mathsf{describeClass}_{U}(C),\\
\mathsf{queryResult}_{\Omega}(\mathsf{objectDescriptorQuery}(o)) &= \mathsf{describeObject}_{U}(o),\\
\mathsf{queryResult}_{\Omega}(\mathsf{activationDescriptorQuery}(a)) &= \mathsf{describeDebugActivation}_{\Omega}(a),\\
\mathsf{queryResult}_{\Omega}(\mathsf{activationControlQuery}(a)) &= \mathsf{liveControl}_{\Omega}(a),\\
\mathsf{queryResult}_{\Omega}(\mathsf{currentActivationQuery}(A)) &= \mathsf{currentActivation}_{\Omega}(A),\\
\mathsf{queryResult}_{\Omega}(\mathsf{actorStackQuery}(A)) &= \mathsf{actorStackSnapshot}_{\Omega}(A),\\
\mathsf{queryResult}_{\Omega}(\mathsf{allInstancesOfQuery}(C)) &= \mathsf{allInstancesOf}_{\Omega}(C),\\
\mathsf{queryResult}_{\Omega}(\mathsf{mixinApplicationsQuery}(M)) &= \mathsf{mixinApplications}_{\Omega}(M).
\end{aligned} \tag{11.26}$$

Inspection is the partial judgment

$$\frac{\widehat{H}_A(g) = \mathsf{mirror}\langle C_g, w, \zeta, R\rangle \qquad \mathsf{inspect} \in R \qquad \mathsf{queryTarget}_{P,U,w}(q_r) \sqsubseteq_{P,U,w} \zeta \qquad d = \mathsf{queryResult}_{\Omega}(q_r)}{\Omega, A, g \vdash \mathsf{inspectMirror}(q_r) \Downarrow d} \quad (\textsc{Mirror-Inspect})$$

There is no failed-premise fallback that reveals a target. The concrete mirror library may reify denial as an exception, but must not disclose the descriptor.

11.4 Reflective transactions

A transaction Δ is a finite sequence of commands, each explicitly carrying the mirror that authorizes it:

$$d_s ::= \mathsf{exceptionStop}(x) \mid \mathsf{breakpointStop}(\lambda_c) \mid \mathsf{requestedStop}(o). \tag{11.27}$$

$$
\begin{aligned}
\delta_{\text{code}} ::= {} & \mathsf{replaceMethodBody}(g, i_f, b) \mid \mathsf{replaceMethodDefinition}(g, M, i_f, f) \\
& \mid \mathsf{addMethodDefinition}(g, M, f) \mid \mathsf{removeMethodDefinition}(g, M, i_f) \\
& \mid \mathsf{replaceSlotDeclarations}(g, M, \delta_s) \mid \mathsf{replaceNestedDeclarations}(g, M, \delta_n) \\
& \mid \mathsf{replaceMixinInitializer}(g, M, \iota), \\
\delta_{\text{class}} ::= {} & \mathsf{changeSuperclass}(g, C, S) \mid \mathsf{changeClassEnclosingObject}(g, C, eo), \\
\delta_{\text{objectClass}} ::= {} & \mathsf{changeObjectClass}(g, o, C), \\
\delta_{\text{objectSlot}} ::= {} & \mathsf{writeObjectSlot}(g, o, \xi_s, v), \\
\delta_{\text{activation}} ::= {} & \mathsf{writeActivationParameter}(g, a, x, v) \mid \mathsf{writeActivationLocal}(g, a, j, v) \\
& \mid \mathsf{changeActivationCurrentClass}(g, a, C) \\
& \mid \mathsf{changeActivationContinuation}(g, a, k) \mid \mathsf{markActivationUncontinuable}(g, a), \\
\delta_{\text{continuation}} ::= {} & \mathsf{continueAtActivation}(g, A, a, v), \\
\delta_{\text{debug}} ::= {} & \mathsf{pauseActor}(g, B, d_s) \\
& \mid \mathsf{replaceCurrentActorStack}(g, B, \mathcal{S}) \\
& \mid \mathsf{replacePausedActorStack}(g, B, r_s, \mathcal{S}) \\
& \mid \mathsf{resumePausedActorAtFullSpeed}(g, B, r_s) \\
& \mid \mathsf{replaceAndResumePausedActorStack}(g, B, r_s, \mathcal{S}), \\
\delta_r ::= {} & \delta_{\text{code}} \mid \delta_{\text{class}} \mid \delta_{\text{objectClass}} \mid \delta_{\text{objectSlot}} \mid \delta_{\text{activation}} \mid \delta_{\text{continuation}} \mid \delta_{\text{debug}}, \\
\Delta ::= {} & \langle \delta_1, \ldots, \delta_m \rangle.
\end{aligned}
\tag{11.28}
$$

By definition, *CodeCommand* is generated by δ_{code}, and *ClassCommand* by δ_{class}. The sets *ObjectClassCommand* and *ObjectSlotCommand* are generated by $\delta_{\text{objectClass}}$ and $\delta_{\text{objectSlot}}$, respectively. Finally, *ActivationCommand* is generated by $\delta_{\text{activation}}$, and *ContinuationCommand* by $\delta_{\text{continuation}}$, and *DebuggerCommand* by δ_{debug}. These seven sets are disjoint by outer constructor and form an exhaustive partition of δ_r. In particular, a class command edits a runtime class record, an object-class command reclassifies one object, and an object-slot command changes state within that object's existing final layout. These are distinct operations with distinct rights and distinct preparation phases. The names are intentionally not overloaded: changing a whole method definition is also distinct from replacing only its body.

Define $\mathsf{commandMirror}(\delta_r)$ as the first field of every command. The required right is the total function

$$
\mathsf{requiredRight}(\delta_r) = \begin{cases}
\mathsf{editCode} & \delta_r \in \mathit{CodeCommand}, \\
\mathsf{editClass} & \delta_r \in \mathit{ClassCommand}, \\
\mathsf{reclassifyObject} & \delta_r \in \mathit{ObjectClassCommand}, \\
\mathsf{editObjectSlots} & \delta_r \in \mathit{ObjectSlotCommand}, \\
\mathsf{editActivation} & \delta_r \in \mathit{ActivationCommand}, \\
\mathsf{controlContinuation} & \delta_r \in \mathit{ContinuationCommand}, \\
\mathsf{controlActorStack} & \delta_r \in \mathit{DebuggerCommand}.
\end{cases}
\tag{11.29}
$$

The target function is

$$\mathsf{commandTarget}(\delta_r) = \begin{cases} \mathsf{methodTarget}(i_f) & \delta_r \text{ names an existing } i_f, \\ \mathsf{mixinTarget}(M) & \delta_r \text{ adds a method or edits mixin structure}, \\ \mathsf{classTarget}(C) & \delta_r \in \mathit{ClassCommand}, \\ \mathsf{objectTarget}(o) & \delta_r \in \mathit{ObjectClassCommand} \cup \mathit{ObjectSlotCommand}, \\ \mathsf{activationTarget}(a) & \delta_r \in \mathit{ActivationCommand} \cup \mathit{ContinuationCommand}, \\ \mathsf{actorTarget}(B) & \delta_r \in \mathit{DebuggerCommand} \text{ and names actor } B. \end{cases} \tag{11.30}$$

For method addition the target is its mixin; for removal, definition replacement, and body replacement the method identity is the target. The authorization predicate is

$$\begin{aligned} \mathsf{authorized}_{\Omega,A}(\delta_r) \quad \Longleftrightarrow \quad & \widehat{H}_A(g) = \mathsf{mirror}\langle C_g, w, \zeta, R\rangle \wedge \\ & g = \mathsf{commandMirror}(\delta_r) \ \wedge \ \mathsf{requiredRight}(\delta_r) \in R \wedge \\ & \mathsf{commandTarget}(\delta_r) \sqsubseteq_{P,\mathsf{vmStore}(\Omega),w} \zeta. \end{aligned} \tag{11.31}$$

Thus syntactic possession of a mirror variable is not sufficient: the unforgeable heap record, VM identity, target containment, and operation right must all agree.

Transactions are simultaneous. Let λ range over the tagged write locations below, and define the complete write-footprint function writes by

$$\mathsf{templateRecordLocations}(\mathcal{S}) = \{\mathsf{activationRecordLoc}(a) \mid \mathsf{frameTemplate}\langle \mathsf{retain}(a), R, \kappa\rangle \text{ occurs in } \mathcal{S}\}. \tag{11.32}$$

$$
\begin{aligned}
&\mathsf{writes}(\mathsf{replaceMethodBody}(g, i_f, b)) \\
&\qquad = \{\mathsf{methodBodyLoc}(i_f)\}, \\
&\mathsf{writes}(\mathsf{replaceMethodDefinition}(g, M, i_f, f)) \\
&\qquad = \{\mathsf{methodDefLoc}(i_f), \mathsf{methodBodyLoc}(i_f), \mathsf{methodDictionaryLoc}(M), \mathsf{selectorLoc}(M, \mathrm{selector}(f))\}, \\
&\mathsf{writes}(\mathsf{addMethodDefinition}(g, M, f)) \\
&\qquad = \{\mathsf{methodDefLoc}(\mathsf{methodId}(f)), \mathsf{methodBodyLoc}(\mathsf{methodId}(f)), \mathsf{methodDictionaryLoc}(M), \mathsf{selectorLoc}(M, \mathrm{selector}(f))\}, \\
&\mathsf{writes}(\mathsf{removeMethodDefinition}(g, M, i_f)) \\
&\qquad = \{\mathsf{methodDefLoc}(i_f), \mathsf{methodBodyLoc}(i_f), \mathsf{methodDictionaryLoc}(M)\}, \\
&\mathsf{writes}(\mathsf{replaceSlotDeclarations}(g, M, \delta_s)) = \{\mathsf{slotDeclsLoc}(M)\}, \\
&\mathsf{writes}(\mathsf{replaceNestedDeclarations}(g, M, \delta_n)) = \{\mathsf{nestedDeclsLoc}(M)\}, \\
&\mathsf{writes}(\mathsf{replaceMixinInitializer}(g, M, \iota)) = \{\mathsf{mixinInitializerLoc}(M)\}, \\
&\mathsf{writes}(\mathsf{changeSuperclass}(g, C, S)) = \{\mathsf{superclassLoc}(C)\}, \\
&\mathsf{writes}(\mathsf{changeClassEnclosingObject}(g, C, eo)) = \{\mathsf{classEnclosingLoc}(C)\}, \\
&\mathsf{writes}(\mathsf{changeObjectClass}(g, o, C)) = \{\mathsf{objectClassLoc}(o)\}, \\
&\mathsf{writes}(\mathsf{writeObjectSlot}(g, o, \xi_s, v)) = \{\mathsf{objectSlotLoc}(o, \xi_s)\}, \\
&\mathsf{writes}(\mathsf{writeActivationParameter}(g, a, x, v)) \\
&\qquad = \{\mathsf{activationParameterLoc}(a, x)\}, \\
&\mathsf{writes}(\mathsf{writeActivationLocal}(g, a, j, v)) = \{\mathsf{activationLocalLoc}(a, j)\}, \\
&\mathsf{writes}(\mathsf{changeActivationCurrentClass}(g, a, C)) \\
&\qquad = \{\mathsf{activationClassLoc}(a)\}, \\
&\mathsf{writes}(\mathsf{changeActivationContinuation}(g, a, k)) \\
&\qquad = \{\mathsf{activationContinuationLoc}(a)\}, \\
&\mathsf{writes}(\mathsf{markActivationUncontinuable}(g, a)) \\
&\qquad = \{\mathsf{activationGraphLoc}, \mathsf{activationContinuableLoc}(a)\}, \\
&\mathsf{writes}(\mathsf{continueAtActivation}(g, A, a, v)) \\
&\qquad = \{\mathsf{actorStackLoc}(A), \mathsf{activationGraphLoc}, \mathsf{activationContinuationLoc}(a)\}, \\
&\mathsf{writes}(\mathsf{pauseActor}(g, B, d_s)) = \{\mathsf{actorStackLoc}(B), \mathsf{pauseTokenSupplyLoc}\}, \\
&\mathsf{writes}(\mathsf{replaceCurrentActorStack}(g, B, \mathcal{S})) \\
&\qquad = \{\mathsf{actorStackLoc}(B), \mathsf{activationGraphLoc}\} \cup \mathsf{templateRecordLocations}(\mathcal{S}), \\
&\mathsf{writes}(\mathsf{replacePausedActorStack}(g, B, r_s, \mathcal{S})) \\
&\qquad = \{\mathsf{pauseTokenSupplyLoc}, \mathsf{actorStackLoc}(B), \mathsf{activationGraphLoc}\} \cup \mathsf{templateRecordLocations}(\mathcal{S}), \\
&\mathsf{writes}(\mathsf{resumePausedActorAtFullSpeed}(g, B, r_s)) = \{\mathsf{actorStackLoc}(B)\}, \\
&\mathsf{writes}(\mathsf{replaceAndResumePausedActorStack}(g, B, r_s, \mathcal{S})) \\
&\qquad = \{\mathsf{actorStackLoc}(B), \mathsf{activationGraphLoc}\} \cup \mathsf{templateRecordLocations}(\mathcal{S}).
\end{aligned}
\tag{11.33}
$$

Location overlap is the least reflexive, symmetric relation

$$\begin{aligned}
\lambda &\bowtie \lambda,\\
\mathsf{activationRecordLoc}(a) &\bowtie \mathsf{activationFieldLoc}(a,\xi),\\
\mathsf{activationFieldLoc}(a,\xi) &\bowtie \mathsf{activationRecordLoc}(a),\\
\mathsf{methodDictionaryLoc}(M) &\bowtie \mathsf{selectorLoc}(M,s),\\
\mathsf{selectorLoc}(M,s) &\bowtie \mathsf{methodDictionaryLoc}(M),\\
\mathsf{activationGraphLoc} &\bowtie \mathsf{activationRecordLoc}(a),\\
\mathsf{activationRecordLoc}(a) &\bowtie \mathsf{activationGraphLoc},\\
\mathsf{activationGraphLoc} &\bowtie \mathsf{activationFieldLoc}(a,\xi),\\
\mathsf{activationFieldLoc}(a,\xi) &\bowtie \mathsf{activationGraphLoc}.
\end{aligned} \tag{11.34}$$

Each concrete parameter, local, class, continuation, and continuability location above is the corresponding instance of $\mathsf{activationFieldLoc}(a,\xi)$. The method-dictionary location accounts for replacement and removal inspecting the old selector as well as writing the new one. The activation-graph location conservatively covers the state-dependent set of records affected by popping or retirement, and the pause-token-supply location serializes commands that mint globally fresh tokens. All other distinct tagged locations are nonoverlapping. This lets two edits to different fields of one activation coexist while making a whole-record stack installation conflict with either edit. For $\Delta = \langle \delta_1, \ldots, \delta_m \rangle$, conflict and nonconflict are therefore the predicates

$$\begin{aligned}
\mathsf{conflict}_\Delta(i,j) &\iff 1 \le i < j \le m \wedge \qquad \exists \lambda \in \mathsf{writes}(\delta_i), \lambda' \in \mathsf{writes}(\delta_j).\ \lambda \bowtie \lambda',\\
\mathsf{nonconflicting}(\Delta) &\iff \neg \exists i,j.\ \mathsf{conflict}_\Delta(i,j).
\end{aligned} \tag{11.35}$$

The multi-location cases make whole-method replacement overlap body replacement, and make continuation transfer overlap another transfer for the same actor or a direct mutation of the resumed continuation.

11.5 Source change and re-elaboration

The code-command projection is

$$\mathsf{codeCommands}(\Delta) = \mathsf{filter}_{CodeCommand}(\Delta). \tag{11.36}$$

Its direct source effect is the partial function $\mathsf{patchSource}$:

$$\begin{aligned}
\mathsf{patchSource}(Q, \mathsf{replaceMethodBody}(g, i_f, b)) &= Q[source(i_f).body \mapsto b],\\
\mathsf{patchSource}(Q, \mathsf{replaceMethodDefinition}(g, M, i_f, f)) &= Q[source(i_f) \mapsto f],\\
\mathsf{patchSource}(Q, \mathsf{addMethodDefinition}(g, M, f)) &= \mathsf{insertMethodSource}(Q, \mathrm{decl}_Q(M), f),\\
\mathsf{patchSource}(Q, \mathsf{removeMethodDefinition}(g, M, i_f)) &= \mathsf{deleteMethodSource}(Q, \mathrm{decl}_Q(M), i_f),\\
\mathsf{patchSource}(Q, \mathsf{replaceSlotDeclarations}(g, M, \delta_s)) &= Q[source(\mathrm{decl}_Q(M)).slots \mapsto \delta_s],\\
\mathsf{patchSource}(Q, \mathsf{replaceNestedDeclarations}(g, M, \delta_n)) &= Q[source(\mathrm{decl}_Q(M)).nested \mapsto \delta_n],\\
\mathsf{patchSource}(Q, \mathsf{replaceMixinInitializer}(g, M, \iota)) &= Q[source(\mathrm{decl}_Q(M)).initializer \mapsto \iota].
\end{aligned} \tag{11.37}$$

$\mathsf{insertMethodSource}$ requires a fresh method identity and absent selector; $\mathsf{deleteMethodSource}$ requires that the identity is a direct method of the named mixin. Definition and body replacement preserve the method identity; delete followed by add cannot reuse it.

Because the transaction is nonconflicting, the fold

$$\begin{aligned}\mathsf{applySourceCommands}(P,\epsilon) &= P,\\ \mathsf{applySourceCommands}(P,\delta\cdot\bar{\delta}) &= \mathsf{applySourceCommands}(\mathsf{patchSource}(P,\delta),\bar{\delta})\end{aligned} \tag{11.38}$$

is independent of the chosen canonical locus order. The patched source candidate is

$$Q = \mathsf{applySourceCommands}(P, \mathsf{codeCommands}(\Delta)). \tag{11.39}$$

The ordinary elaborator is rerun on the complete affected dependency closure:

$$\begin{aligned}&\mathsf{reelaborateProgram}(P,Q) = P' \quad \Longleftrightarrow\\ &\quad P'.\mathit{source} = Q.\mathit{source} \wedge\\ &\quad P'.\mathit{unit} = Q.\mathit{unit} \wedge\\ &\quad \forall K \in \mathrm{dom}(P.\mathit{declMixin}) \cap \mathrm{dom}(Q.\mathit{source}).\ P'.\mathit{declMixin}(K) = P.\mathit{declMixin}(K) \wedge\\ &\quad \forall f, f' \text{ denoting one surviving declaration. } \mathsf{methodId}(f') = \mathsf{methodId}(f) \wedge\\ &\quad \mathsf{coreProgramOK}(P').\end{aligned} \tag{11.40}$$

The last clause is Equation (13.83); it recomputes all derived fields of Equation (3.4), including synthesized accessors, lexical parents, scope chains, activation members, owner maps, initializer normal forms, and method-body maps. It is undefined on a parse, binding, access, arity, duplicate-selector, or other static error. This is why a transaction cannot expose a half-elaborated program.

11.6 Class graphs, object layouts, and individual objects

Let the command projections be

$$\begin{aligned}\mathsf{classCommands}(\Delta) &= \mathsf{filter}_{\mathit{ClassCommand}}(\Delta),\\ \mathsf{objectClassCommands}(\Delta) &= \mathsf{filter}_{\mathit{ObjectClassCommand}}(\Delta),\\ \mathsf{objectSlotCommands}(\Delta) &= \mathsf{filter}_{\mathit{ObjectSlotCommand}}(\Delta).\end{aligned} \tag{11.41}$$

Their heap effects are the distinct partial functions

$$\begin{aligned}\mathsf{applyClassCommand}(H, \mathsf{changeSuperclass}(g,C,S)) &= H[C.S \mapsto S],\\ \mathsf{applyClassCommand}(H, \mathsf{changeClassEnclosingObject}(g,C,eo)) &= H[C.eo \mapsto eo],\\ \mathsf{applyObjectClassCommand}(H, \mathsf{changeObjectClass}(g,o,C)) &= H[o.C \mapsto C].\end{aligned} \tag{11.42}$$

The corresponding phases are

$$\begin{aligned}\mathsf{applyClassCommands}(H,\epsilon) &= H,\\ \mathsf{applyClassCommands}(H,\delta\cdot\bar{\delta}) &= \mathsf{applyClassCommands}(\mathsf{applyClassCommand}(H,\delta),\bar{\delta}),\\ \mathsf{applyObjectClassCommands}(H,\epsilon) &= H,\\ \mathsf{applyObjectClassCommands}(H,\delta\cdot\bar{\delta}) &= \mathsf{applyObjectClassCommands}\\ &\qquad (\mathsf{applyObjectClassCommand}(H,\delta),\bar{\delta}).\end{aligned} \tag{11.43}$$

The command sequences are in canonical locus order. Neither phase changes $H(C).\chi_C$: creation provenance records how the class was created, not its current superclass or enclosing object.

The three projections in Equation (11.41) must not be conflated. A class command changes the graph or lexical context shared by all instances of C. An object-class command changes only $o.C$

and then subjects that object to layout reconciliation. An object-slot command leaves both the class graph and $o.C$ unchanged and writes one slot only after the final layout has been established.

For every ordinary object, layout reconciliation against the final program and class graph is

$$L_o = \mathsf{set}(\mathsf{layout}_{P',H_c}(\underset{H_c}{\mathsf{classOf}}(o))),$$
$$\mathsf{reconcileObjectLayout}(P', H_c, o) = H_c[o.\sigma \mapsto [\xi_s \in L_o \mapsto \begin{cases} H_c(o).\sigma(\xi_s) & \xi_s \in \mathrm{dom}(H_c(o).\sigma), \\ \mathsf{nil} & \xi_s \notin \mathrm{dom}(H_c(o).\sigma), \end{cases}]]. \tag{11.44}$$

Thus stable physical slot identities preserve retained state, newly introduced slots contain nil, and removed slots become unreachable in the same atomic commit. The operation applies to every object whose old and new layouts differ, including every instance affected transitively by a superclass or mixin-slot edit and an individual object named by changeObjectClass. If $P.\mathit{mixin}(M) = \langle K, \phi, \delta, \nu, \iota \rangle$, define

$$\mathsf{ownNestedKeys}_P(M) = \mathrm{dom}(\nu),$$
$$\mathsf{nestedKeys}_{P,H}(C) = \bigcup_{R \in \mathsf{set}(\mathsf{chain}_H(C)) \setminus \{\mathsf{Top}\}} \{(R, N) \mid N \in \mathsf{ownNestedKeys}_P(\underset{H}{\mathsf{mixin}}(R))\}. \tag{11.45}$$

Nested-class caches are reconciled separately using Equation (11.45):

$$\mathsf{reconcileNestedCache}(P', H, o) = H[o.\eta \mapsto H(o).\eta \restriction \mathsf{nestedKeys}_{P',H}(\underset{H}{\mathsf{classOf}}(o))]. \tag{11.46}$$

Removing a declaration therefore removes its memoized reachability; adding one does not eagerly create a class. Existing class objects remain ordinary live objects if referenced elsewhere.

Using the VM identity order $\prec$ fixed by Equation (11.23), define

$$\begin{aligned}
\mathsf{objectIds}(H) &= \mathsf{sort}_{\prec}\{o \in \mathrm{dom}(H) \mid H(o) = \mathsf{object}\langle C, \sigma, \eta \rangle\}, \\
\mathsf{reconcileObjectLayouts}(P', H, \epsilon) &= H, \\
\mathsf{reconcileObjectLayouts}(P', H, o \cdot \bar{O}) &= \mathsf{reconcileObjectLayouts} \\
&\quad (P', \mathsf{reconcileObjectLayout}(P', H, o), \bar{O}), \\
\mathsf{reconcileAllObjectLayouts}(P', H) &= \mathsf{reconcileObjectLayouts}(P', H, \mathsf{objectIds}(H)), \\
\mathsf{reconcileNestedCaches}(P', H, \epsilon) &= H, \\
\mathsf{reconcileNestedCaches}(P', H, o \cdot \bar{O}) &= \mathsf{reconcileNestedCaches} \\
&\quad (P', \mathsf{reconcileNestedCache}(P', H, o), \bar{O}), \\
\mathsf{reconcileAllNestedCaches}(P', H) &= \mathsf{reconcileNestedCaches}(P', H, \mathsf{objectIds}(H)).
\end{aligned} \tag{11.47}$$

After all reconciliations, an explicit mirror slot write is

$$\begin{aligned}
&\mathsf{applyObjectSlotWrite}(P', H, \mathsf{writeObjectSlot}(g, o, \xi_s, v)) = H[o.\sigma(\xi_s) \mapsto v] \\
&\qquad \text{if } \xi_s \in \mathsf{set}(\mathsf{layout}_{P',H}(\underset{H}{\mathsf{classOf}}(o))).
\end{aligned} \tag{11.48}$$

It is undefined otherwise. This operation is different from the base setter and from initializer direct write; no source-level access check or synthesized method is involved. All actor-isolation checks are nevertheless retained.

The object-slot phase is

$$\begin{aligned}
\mathsf{applyObjectSlotWrites}(P', H, \epsilon) &= H, \\
\mathsf{applyObjectSlotWrites}(P', H, \delta \cdot \bar{\delta}) &= \mathsf{applyObjectSlotWrites}(P', \mathsf{applyObjectSlotWrite}(P', H, \delta), \bar{\delta}).
\end{aligned} \tag{11.49}$$

11.7 Activations and continuations

The command projections for the remaining two phases are

$$\begin{aligned}\mathsf{activationCommands}(\Delta) &= \mathsf{filter}_{ActivationCommand}(\Delta),\\ \mathsf{continuationCommands}(\Delta) &= \mathsf{filter}_{ContinuationCommand}(\Delta).\end{aligned} \tag{11.50}$$

For an activation command other than continuation transfer, define

$$\begin{aligned}\mathsf{applyActivationCommand}(H, \mathsf{writeActivationParameter}(g,a,x,v)) &= H[a.\rho(x) \mapsto v],\\ \mathsf{applyActivationCommand}(H, \mathsf{writeActivationLocal}(g,a,j,v)) &= H[a.\ell(j) \mapsto v],\\ \mathsf{applyActivationCommand}(H, \mathsf{changeActivationCurrentClass}(g,a,C)) &= H[a.C \mapsto C],\\ \mathsf{applyActivationCommand}(H, \mathsf{changeActivationContinuation}(g,a,k)) &= H[a.k \mapsto k],\\ \mathsf{applyActivationCommand}(H, \mathsf{markActivationUncontinuable}(g,a)) &= \mathsf{markUncontinuable}(H,a).\end{aligned} \tag{11.51}$$

The first case is reflectively permitted even though parameter slots are immutable at base level. The second requires a physical local identity already present in $H(a).\ell$. A changed current class must remain on the current instance's class chain. A changed continuation is either nil or a strictly lower live stack activation, so continuation links remain acyclic. Marking uses Equations (7.30) and (7.31).

The activation phase is

$$\begin{aligned}\mathsf{applyActivationCommands}(H,\epsilon) &= H,\\ \mathsf{applyActivationCommands}(H, \delta\cdot\bar{\delta}) &= \mathsf{applyActivationCommands}(\mathsf{applyActivationCommand}(H,\delta),\bar{\delta}).\end{aligned} \tag{11.52}$$

Equation (7.28) supplies the stack operation used by reflective continuation transfer. The corresponding state transformer is

$$\begin{aligned}&\delta_c = \mathsf{continueAtActivation}(g,A,a,v),\\ &\mathsf{applyContinuationTransfer}(H,\mathcal{X},\delta_c) = \langle H',\mathcal{X}'\rangle,\\ &\mathcal{X}(A) = \mathsf{runningTurn}\langle\mathcal{A},t,p,u\rangle, \qquad H(a).q_a = \mathsf{true}, \qquad \mathsf{resume}(\mathcal{A},a,v) = \langle\mathcal{A}',v\rangle,\\ &H' = H[a.k \mapsto \mathsf{nil}], \qquad \mathcal{X}' = \mathcal{X}[A \mapsto \mathsf{runningTurn}\langle\mathcal{A}',v,p,u\rangle].\end{aligned} \tag{11.53}$$

It is undefined for an idle actor, a retained but no longer live activation, or an uncontinuable activation. At most one continuation transfer per actor may occur in a transaction. The discarded stack suffix remains in the heap, as in ordinary return, so debuggers may retain it; its continuation dependents are marked uncontinuable before commit. This gives reflection continuation power without introducing a second, incompatible continuation representation.

The continuation-transfer phase is

$$\begin{aligned}\mathsf{applyContinuationTransfers}(H,\mathcal{X},\epsilon) &= \langle H,\mathcal{X}\rangle,\\ \mathsf{applyContinuationTransfers}(H,\mathcal{X},\delta\cdot\bar{\delta}) &= \mathsf{applyContinuationTransfers}(H',\mathcal{X}',\bar{\delta})\\ &\qquad \text{where } \langle H',\mathcal{X}'\rangle = \mathsf{applyContinuationTransfer}(H,\mathcal{X},\delta).\end{aligned} \tag{11.54}$$

11.8 Debugger stack installation and full-speed resumption

The debugger-command projection is

$$\mathsf{debuggerCommands}(\Delta) = \mathsf{filter}_{DebuggerCommand}(\Delta). \tag{11.55}$$

For a template $\mathcal{S} = \langle \psi_1, \ldots, \psi_m \rangle$, key resolution is the partial function

$$\begin{aligned} &\mathsf{resolveTemplateKeys}(H, \mathcal{S}) = \nu, \\ &\qquad \nu(\mathsf{retain}(a)) = a \qquad \text{only if } H(a) \text{ is an activation record,} \\ &\qquad \nu(\mathsf{freshFrame}(u_i)) = a_i \qquad a_i \text{ is fresh,} \\ &\qquad \nu(\eta_i) \neq \nu(\eta_j) \quad i \neq j. \end{aligned} \tag{11.56}$$

Fresh choices are simultaneous. Substitution $\nu(R)$ replaces every template-key occurrence in a continuation, home, or lexical environment by its resolved activation identity.

The well-formedness predicate for a resolved stack image is

$$\begin{aligned} \mathsf{debugStackOK}(P, H, \langle a_1, \kappa_1 \rangle \cdots \langle a_m, \kappa_m \rangle) \iff\ & m > 0 \ \wedge\ \mathsf{pairwiseDistinct}(a_1, \ldots, a_m) \ \wedge \\ & \mathsf{decodeControlImage}(\langle a_1, \kappa_1 \rangle \cdots \langle a_m, \kappa_m \rangle) \downarrow \ \wedge \\ & \forall i \leq m.\ H(a_i) \text{ is an activation record } \wedge H(a_i).q_a = \mathsf{true} \ \wedge \\ & \forall i \leq m.\ \mathsf{controlLocation}(\kappa_i) \downarrow \ \wedge \mathsf{controlRefs}(\kappa_i) \subseteq \mathrm{dom}(H) \ \wedge \\ & \forall i \leq m.\ H(a_i).k = \mathsf{nil} \ \vee\ \exists j < i.\ H(a_i).k = a_j \ \wedge \\ & \forall i \leq m.\ H(a_i).h = \mathsf{nil} \ \vee\ H(a_i).h \in \mathrm{dom}(H). \end{aligned} \tag{11.57}$$

controlRefs is the instance of the structural reference extractor of Equation (11.90) for terms and intra-activation frames. The control-map definedness premise validates a source or machine location; a bytecode platform additionally requires the operand-stack and temporary layout encoded by π_i to match the stack map at the corresponding PC. This is the implementation representation of the same predicate, not a weaker alternate rule.

Removed live activations remain available to retained mirrors but cease to be continuable:

$$\mathsf{retireRemoved}(H, R) = H[\, a.k \mapsto \mathsf{nil},\ a.q_a \mapsto \mathsf{false} \mid a \in R \,]. \tag{11.58}$$

Define atomic template materialization by

$$\begin{aligned} &\mathsf{materializeStackTemplate}(P, H, \mathcal{A}, \mathcal{S}) = \langle H', \mathcal{A}', t' \rangle \iff \\ &\quad \nu = \mathsf{resolveTemplateKeys}(H, \mathcal{S}) \ \wedge \\ &\quad \mathcal{K} = \langle \nu(\psi_1.\eta), \psi_1.\kappa \rangle \cdots \langle \nu(\psi_m.\eta), \psi_m.\kappa \rangle \ \wedge \\ &\quad \mathsf{decodeControlImage}(\mathcal{K}) = \langle \mathcal{A}', t' \rangle \ \wedge \\ &\quad R = \mathsf{set}(\mathsf{activationIds}(\mathcal{A})) \setminus \mathsf{set}(\mathsf{activationIds}(\mathcal{A}')) \ \wedge \\ &\quad H_0 = \mathsf{retireRemoved}(H, R) \ \wedge \\ &\quad H' = H_0[\, \nu(\psi_i.\eta) \mapsto \nu(\psi_i.R) \mid 1 \leq i \leq m \,] \ \wedge \\ &\quad \mathsf{debugStackOK}(P, H', \mathcal{K}). \end{aligned} \tag{11.59}$$

It is undefined if any key, record, control point, reference, continuation, or stack-shape condition fails. Since only H' and the reconstructed $\langle \mathcal{A}', t' \rangle$ leave the function, neither allocation nor retirement is visible on failure.

The effect of one debugger command, issued by actor A, is the following partial function. Pausing allocates a fresh token:

$$\begin{aligned} &\mathsf{applyDebuggerCommand}(P, H, \mathcal{X}, A, \mathsf{pauseActor}(g, B, d_s)) = \langle H, \mathcal{X}' \rangle, \\ &\mathcal{X}(B) = \mathsf{runningTurn}\langle \mathcal{A}, t, p, u \rangle, \quad A \neq B, \quad \mathcal{A} \neq \epsilon, \quad r_s \text{ fresh}, \\ &\mathcal{X}' = \mathcal{X}[B \mapsto \mathsf{pausedTurn}\langle \mathcal{A}, t, p, u, r_s, d_s \rangle]. \end{aligned} \tag{11.60}$$

A debugger running in the actor that it is editing uses the nonreturning current-stack replacement:

$$\begin{aligned}&\mathsf{applyDebuggerCommand}(P,H,\mathcal{X},A,\mathsf{replaceCurrentActorStack}(g,B,\mathcal{S}))=\langle H',\mathcal{X}'\rangle,\\&A=B,\quad \mathcal{X}(B)=\mathsf{runningTurn}\langle\mathcal{A},t,p,u\rangle,\\&\mathsf{materializeStackTemplate}(P,H,\mathcal{A},\mathcal{S})=\langle H',\mathcal{A}',t'\rangle,\\&\mathcal{X}'=\mathcal{X}[B\mapsto\mathsf{runningTurn}\langle\mathcal{A}',t',p,u\rangle].\end{aligned}\tag{11.61}$$

The reflective invocation that requested this command is among the discarded or replaced frames, so it does not return. This is the operation used by an in-actor debugger after a top-level resumable-exception handler has constructed the desired continuation. For the common resumable-exception case, the derived constructor is

$$\mathsf{resumeExceptionStack}(\mathcal{S}_0\cdot\langle\psi_f\rangle\cdot\mathcal{S}_d,a_f,v)=\mathcal{S}_0\cdot\langle\mathsf{promoteFrame}(\psi_f,v)\rangle\quad\text{if }\psi_f.\eta=\mathsf{retain}(a_f).\tag{11.62}$$

It removes the handler and debugger suffix $\mathcal{S}_d$ and makes v the result of the original signal: send. Submitting this template through Equation (11.61) therefore resumes precisely at the protected failure point under the normal VM relation.

For a paused actor, replacement checks the exact pause token and issues a new one:

$$\begin{aligned}&\mathsf{applyDebuggerCommand}(P,H,\mathcal{X},A,\mathsf{replacePausedActorStack}(g,B,r_s,\mathcal{S}))=\langle H',\mathcal{X}'\rangle,\\&\mathcal{X}(B)=\mathsf{pausedTurn}\langle\mathcal{A},t,p,u,r_s,d_s\rangle,\\&\mathsf{materializeStackTemplate}(P,H,\mathcal{A},\mathcal{S})=\langle H',\mathcal{A}',t'\rangle,\quad r'_s\text{ fresh},\\&\mathcal{X}'=\mathcal{X}[B\mapsto\mathsf{pausedTurn}\langle\mathcal{A}',t',p,u,r'_s,d_s\rangle].\end{aligned}\tag{11.63}$$

Resuming an unchanged paused stack selects ordinary VM execution:

$$\begin{aligned}&\mathsf{applyDebuggerCommand}(P,H,\mathcal{X},A,\mathsf{resumePausedActorAtFullSpeed}(g,B,r_s))=\langle H,\mathcal{X}'\rangle,\\&\mathcal{X}(B)=\mathsf{pausedTurn}\langle\mathcal{A},t,p,u,r_s,d_s\rangle,\\&\mathcal{X}'=\mathcal{X}[B\mapsto\mathsf{runningTurn}\langle\mathcal{A},t,p,u\rangle].\end{aligned}\tag{11.64}$$

Replacement and full-speed resumption may instead be one atomic operation:

$$\begin{aligned}&\mathsf{applyDebuggerCommand}(P,H,\mathcal{X},A,\mathsf{replaceAndResumePausedActorStack}(g,B,r_s,\mathcal{S}))=\langle H',\mathcal{X}'\rangle,\\&\mathcal{X}(B)=\mathsf{pausedTurn}\langle\mathcal{A},t,p,u,r_s,d_s\rangle,\\&\mathsf{materializeStackTemplate}(P,H,\mathcal{A},\mathcal{S})=\langle H',\mathcal{A}',t'\rangle,\\&\mathcal{X}'=\mathcal{X}[B\mapsto\mathsf{runningTurn}\langle\mathcal{A}',t',p,u\rangle].\end{aligned}\tag{11.65}$$

"Full speed" is not a second language semantics: after either resume equation, Rule Actor-Sequential again lifts the ordinary sequential relation. An interpreter, baseline VM, or optimizing VM conforms by implementing that same relation from the installed control image.

The debugger phase is the explicit fold

$$\begin{aligned}&\mathsf{applyDebuggerCommands}(P,H,\mathcal{X},A,\epsilon)=\langle H,\mathcal{X}\rangle,\\&\mathsf{applyDebuggerCommands}(P,H,\mathcal{X},A,\delta\cdot\bar{\delta})=\mathsf{applyDebuggerCommands}(P,H',\mathcal{X}',A,\bar{\delta})\\&\qquad\text{where }\langle H',\mathcal{X}'\rangle=\\&\qquad\qquad\mathsf{applyDebuggerCommand}(P,H,\mathcal{X},A,\delta).\end{aligned}\tag{11.66}$$

Simulator correspondence. Let D be a Simulator state represented by ordinary Newspeak objects and let $D \approx_{\mathsf{sim}} \Sigma$ mean that its bytecode frames and program counters decode to the stack and control image of Equation (11.9). Write $\longmapsto^{+}_{\mathsf{Simulator}}$ for a nonempty finite Simulator execution and $\longmapsto^{*}$ for a possibly empty finite semantic execution. An admitted compiler/Simulator pair supplies the three-part certificate

$$\begin{aligned} D \approx_{\mathsf{sim}} \Sigma \ \wedge\ D \longmapsto_{\mathsf{Simulator}} D' &\Rightarrow \exists \Sigma'.\ \Sigma \longmapsto^{*} \Sigma' \wedge D' \approx_{\mathsf{sim}} \Sigma', \\ D \approx_{\mathsf{sim}} \Sigma \ \wedge\ \Sigma \longmapsto \Sigma' &\Rightarrow \exists D'.\ D \longmapsto^{+}_{\mathsf{Simulator}} D' \wedge D' \approx_{\mathsf{sim}} \Sigma', \\ D \approx_{\mathsf{sim}} \Sigma &\Rightarrow \mathsf{resumeAtFullSpeed}(D) = \Sigma. \end{aligned} \tag{11.67}$$

The first clause permits bytecode-level stuttering while proving instruction soundness; the second prevents the Simulator from omitting a semantic step; the third closes the critical boundary back to native execution without a change of represented configuration. The certificate lifts the first clause from one instruction to every finite Simulator execution. The Simulator can inspect, push, pop, and copy its ordinary frame-image values; Equations (11.61) and (11.65) cross back to normal VM execution. Proving Equation (11.67) for the actual Simulator and bytecode set is an implementation-refinement proof; the language semantics fixes the state and transfer contract it must implement.

11.9 Preparation, validation, and atomic commit

All phase ordering is semantic, not user-visible. Define the partial preparation function

$$\begin{aligned} &\mathsf{prepareReflection}(\Omega, A, \Delta) = \Omega' \\ &\quad\iff\quad P' = \mathsf{reelaborateProgram}(P, Q) \wedge \\ &H_1 = \mathsf{applyClassCommands}(\mathsf{vmStore}(\Omega), \mathsf{classCommands}(\Delta)) \wedge \\ &H_c = \mathsf{applyObjectClassCommands}(H_1, \mathsf{objectClassCommands}(\Delta)) \wedge \\ &H_l = \mathsf{reconcileAllObjectLayouts}(P', H_c) \wedge \\ &H_n = \mathsf{reconcileAllNestedCaches}(P', H_l) \wedge \\ &H_o = \mathsf{applyObjectSlotWrites}(P', H_n, \mathsf{objectSlotCommands}(\Delta)) \wedge \\ &H_a = \mathsf{applyActivationCommands}(H_o, \mathsf{activationCommands}(\Delta)) \wedge \\ &\langle H_d, \mathcal{X}_d \rangle = \mathsf{applyDebuggerCommands}(P', H_a, \mathcal{X}, A, \mathsf{debuggerCommands}(\Delta)) \wedge \\ &\langle H_t, \mathcal{X}' \rangle = \mathsf{applyContinuationTransfers}(H_d, \mathcal{X}_d, \mathsf{continuationCommands}(\Delta)) \wedge \\ &\Omega_0 = \Omega[P \mapsto P', \mathcal{X} \mapsto \mathcal{X}', n \mapsto n+1] \wedge \\ &\Omega' = \mathsf{repartitionVMStore}(\Omega_0, H_t). \end{aligned} \tag{11.68}$$

The fresh activations introduced by stack installation are

$$\mathsf{freshInstalled}_{\Omega}(\mathcal{X}', B) = \mathsf{set}(\mathsf{stackActivationIds}_{\mathcal{X}'}(B)) \setminus \mathrm{dom}(\mathsf{vmStore}(\Omega)). \tag{11.69}$$

repartitionVMStore returns every pre-existing identity to the same V or $\mathcal{H}(A)$ domain from which it came, and places each member of Equation (11.69) in $\mathcal{H}(B)$. The sets for distinct actors must be disjoint. Actor-isolation validation below rejects any installed record containing an illegal near reference; reflection creates no cross-actor near reference.

The validation predicates below are over tagged record sets, so their quantifiers have explicit domains:

$$\begin{aligned} \mathsf{classRecordIds}(H) &= \{C \in \mathrm{dom}(H) \mid H(C) = \mathsf{class}\langle M, S, eo, C_m, q, \chi_C \rangle\}, \\ \mathsf{ordinaryObjectIds}(H) &= \{o \in \mathrm{dom}(H) \mid H(o) = \mathsf{object}\langle C, \sigma, \eta \rangle\}, \\ \mathsf{activationRecordIds}(H) &= \{a \in \mathrm{dom}(H) \mid H(a) = \mathsf{activation}\langle p, \rho, \ell, o, C, k, h, q_a, \gamma \rangle\}. \end{aligned} \tag{11.70}$$

Class-chain termination is the predicate

$$\begin{aligned}\mathsf{reachesTop}_H(C) \quad \Longleftrightarrow \quad & \exists k \in \mathbb{N}, C_0, \ldots, C_k.\ C_0 = C \ \wedge\ C_k = \mathsf{Top} \ \wedge \\ & \mathsf{pairwiseDistinct}(C_0, \ldots, C_k) \ \wedge \\ & \forall i < k.\ C_i \in \mathsf{classRecordIds}(H) \ \wedge \operatorname*{super}_H(C_i) = C_{i+1}.\end{aligned} \tag{11.71}$$

Individual class-record consistency is

$$\begin{aligned}\mathsf{classRecordOK}(P, H, C) \quad \Longleftrightarrow \quad & \operatorname*{mixin}_H(C) \in \mathrm{dom}(P.\mathit{mixin}) \ \wedge \\ & H(C).C_m \in \mathsf{classRecordIds}(H) \ \wedge \\ & H(C).\mathit{eo} \in \mathrm{dom}(H) \cup \{\mathsf{nil}\} \ \wedge (C = \mathsf{Object} \ \vee \ \operatorname*{super}_H(C) \neq \mathsf{Top}).\end{aligned} \tag{11.72}$$

The complete class-graph check is

$$\begin{aligned}\mathsf{classGraphOK}(P, H) \quad \Longleftrightarrow \quad & \mathsf{Object} \in \mathsf{classRecordIds}(H) \ \wedge \operatorname*{super}_H(\mathsf{Object}) = \mathsf{Top} \ \wedge \\ & \forall C \in \mathsf{classRecordIds}(H).\ \mathsf{reachesTop}_H(C) \ \wedge \\ & \forall C \in \mathsf{classRecordIds}(H) \setminus \{\mathsf{Top}\}.\ \mathsf{classRecordOK}(P, H, C).\end{aligned} \tag{11.73}$$

The declaration retained by class provenance and the complete ordinary-object check are defined by

$$\mathsf{originDecl}(\mathsf{expression}(K)) = K, \qquad \mathsf{originDecl}(\mathsf{actor}(K)) = K. \tag{11.74}$$

The cached class for one nested declaration is valid exactly when

$$\begin{aligned}& \mathsf{cachedNestedClassOK}(H, o, (R, N)) \quad \Longleftrightarrow \\ & \quad H(o).\eta(R, N) \in \mathsf{classRecordIds}(H) \ \wedge \\ & \quad H(H(o).\eta(R, N)).\mathit{eo} = o \ \wedge \\ & \quad \mathsf{originDecl}(H(H(o).\eta(R, N)).\chi_C) = N.\end{aligned} \tag{11.75}$$

An ordinary object record is valid when

$$\begin{aligned}\mathsf{objectRecordOK}(P, H, o) \quad \Longleftrightarrow \quad & \operatorname*{classOf}_H(o) \in \mathsf{classRecordIds}(H) \ \wedge \\ & \mathrm{dom}(H(o).\sigma) = \mathsf{set}(\mathsf{layout}_{P,H}(\operatorname*{classOf}_H(o))) \ \wedge \\ & \mathrm{dom}(H(o).\eta) \subseteq \mathsf{nestedKeys}_{P,H}(\operatorname*{classOf}_H(o)) \ \wedge \\ & \forall d \in \mathrm{dom}(H(o).\eta).\ \mathsf{cachedNestedClassOK}(H, o, d).\end{aligned} \tag{11.76}$$

The store-wide object check is

$$\mathsf{objectStoreOK}(P, H) \quad \Longleftrightarrow \quad \forall o \in \mathsf{ordinaryObjectIds}(H).\ \mathsf{objectRecordOK}(P, H, o). \tag{11.77}$$

For activation validation, first extract the activation identities from each actor's live stack:

$$\begin{aligned}
\mathsf{activationIds}(\epsilon) &= \epsilon,\\
\mathsf{activationIds}(\mathcal{A}\cdot\langle a,\pi\rangle) &= \mathsf{activationIds}(\mathcal{A})\cdot\langle a\rangle,\\
\mathsf{stackActivationIds}_{\mathcal{X}}(A) &= \epsilon \quad \text{if } \mathcal{X}(A)=\mathsf{idle},\\
\mathsf{stackActivationIds}_{\mathcal{X}}(A) &= \mathsf{activationIds}(\mathcal{A}),\\
&\qquad \text{if } \mathcal{X}(A)=\mathsf{runningTurn}\langle\mathcal{A},t,p,u\rangle,\\
\mathsf{stackActivationIds}_{\mathcal{X}}(A) &= \mathsf{activationIds}(\mathcal{A}),\\
&\qquad \text{if } \mathcal{X}(A)=\mathsf{pausedTurn}\langle\mathcal{A},t,p,u,r_s,d_s\rangle,\\
\mathsf{liveActivationIds}(\mathcal{X}) &= \bigcup_{A\in\mathrm{dom}(\mathcal{X})} \mathsf{set}(\mathsf{stackActivationIds}_{\mathcal{X}}(A)),\\
\mathsf{below}_{\mathcal{X}}(A,k,a) &\iff \exists i<j.\\
\mathsf{stackActivationIds}_{\mathcal{X}}(A)[i]=k\ &\wedge \mathsf{stackActivationIds}_{\mathcal{X}}(A)[j]=a.
\end{aligned} \tag{11.78}$$

Definedness of every partial expression below uses the notation

$$e_p\downarrow \iff \exists v.\ e_p=v, \qquad e_p\uparrow \iff \neg(e_p\downarrow). \tag{11.79}$$

Surviving method declarations determine the parameter domain of a live method activation:

$$\mathsf{methodFormals}_P(i_f)=\bar{x} \iff \exists M,s,\alpha,\ell,b,K.\ \underset{P}{\mathrm{methods}}(M)(s)=\langle i_f,s,\alpha,\bar{x},\ell,b,K\rangle. \tag{11.80}$$

The validity of one live activation is

$$\begin{aligned}
\mathsf{liveActivationRecordOK}(P,H,\mathcal{X},a) \iff \exists p,\rho,\ell,o,C,k,h,q_a,\gamma.&\\
H(a)=\mathsf{activation}\langle p,\rho,\ell,o,C,k,h,q_a,\gamma\rangle\ &\wedge\\
\big((p=\mathsf{top}\ \wedge\ \rho=\ell=\varnothing\ \wedge o=C=h=\mathsf{nil})\ &\vee\\
(p\neq\mathsf{top}\ \wedge\ o\in\mathrm{dom}(H)\ \wedge C\preceq_H \underset{H}{\mathrm{classOf}}(o)\ &\wedge\\
h\in\mathsf{activationRecordIds}(H))\big)\ &\wedge\\
\big(k=\mathsf{nil}\ \vee\exists A.\ \mathsf{below}_{\mathcal{X}}(A,k,a)\big)\ &\wedge\\
\big(\mathsf{methodFormals}_P(p)\uparrow\ \vee&\\
\mathrm{dom}(\rho)=\mathsf{set}(\mathsf{methodFormals}_P(p))\big).&
\end{aligned} \tag{11.81}$$

The aggregate check is

$$\begin{aligned}
\mathsf{liveActivationsOK}(P,H,\mathcal{X}) \iff\ & \forall a\in\mathsf{liveActivationIds}(\mathcal{X}).\\
& \mathsf{liveActivationRecordOK}(P,H,\mathcal{X},a).
\end{aligned} \tag{11.82}$$

Equation (11.82) therefore permits an already materialized activation of a removed method, but requires the bound parameter domain to agree with every surviving edited signature.

Let $H_0=\mathsf{vmStore}(\Omega)$ and let $H'=\mathsf{vmStore}(\Omega')$. The command-specific activation conditions are the following total predicate:

$$\begin{aligned}
&\mathsf{activationEditOK}_{H_0,H',\mathcal{X}'}(\mathsf{writeActivationParameter}(g,a,x,v))\\
&\quad\iff a\in\mathsf{activationRecordIds}(H_0)\ \wedge x\in\mathrm{dom}(H_0(a).\rho).
\end{aligned} \tag{11.83}$$

$$\begin{aligned}
&\mathsf{activationEditOK}_{H_0,H',\mathcal{X}'}(\mathsf{writeActivationLocal}(g,a,j,v))\\
&\quad\iff a\in\mathsf{activationRecordIds}(H_0)\ \wedge j\in\mathrm{dom}(H_0(a).\ell).
\end{aligned} \tag{11.84}$$

$$\begin{aligned}&\mathsf{activationEditOK}_{H_0,H',\mathcal{X}'}(\mathsf{changeActivationCurrentClass}(g,a,C))\\&\quad\Longleftrightarrow a \in \mathsf{activationRecordIds}(H') \ \wedge C \preceq_{H'} \operatorname*{\mathsf{classOf}}_{H'}(H'(a).o).\end{aligned} \tag{11.85}$$

$$\begin{aligned}&\mathsf{activationEditOK}_{H_0,H',\mathcal{X}'}(\mathsf{changeActivationContinuation}(g,a,k))\\&\quad\Longleftrightarrow a \in \mathsf{liveActivationIds}(\mathcal{X}') \wedge\\&\qquad\qquad (k = \mathsf{nil} \ \vee \ \exists A.\ \mathsf{below}_{\mathcal{X}'}(A,k,a)).\end{aligned} \tag{11.86}$$

$$\begin{aligned}&\mathsf{activationEditOK}_{H_0,H',\mathcal{X}'}(\mathsf{markActivationUncontinuable}(g,a))\\&\quad\Longleftrightarrow a \in \mathsf{activationRecordIds}(H_0),\\&\mathsf{activationEditOK}_{H_0,H',\mathcal{X}'}(\delta) \Longleftrightarrow \mathsf{true} \quad \text{if } \delta \notin \mathit{ActivationCommand}.\end{aligned} \tag{11.87}$$

Continuation transfer is already partial in Equation (11.53); its transaction-level cardinality and the remaining activation edits are checked by

$$\mathsf{transferCount}(\Delta, B) = \left|\{\delta \in \Delta \mid \exists g, a, v.\ \delta = \mathsf{continueAtActivation}(g, B, a, v)\}\right|. \tag{11.88}$$

$$\begin{aligned}\mathsf{activationEditsOK}(\Omega,\Delta,\Omega') \quad\Longleftrightarrow\quad &\big(\forall \delta \in \Delta.\ \mathsf{activationEditOK}_{H_0,H',\mathcal{X}'}(\delta)\big) \wedge\\ &\forall B \in \mathrm{dom}(\mathcal{X}').\ \mathsf{transferCount}(\Delta, B) \le 1.\end{aligned} \tag{11.89}$$

Let refs be the structural reference extractor, defined for every runtime constructor, finite map, and sequence by

$$\begin{aligned}\mathsf{refs}(x) &= \{x\} && x \in \mathit{Ref},\\ \mathsf{refs}(z) &= \varnothing && z \text{ is atomic and } z \notin \mathit{Ref},\\ \mathsf{refs}(\mathsf{ctor}(z_1,\ldots,z_k)) &= \bigcup_{i=1}^{k} \mathsf{refs}(z_i),\\ \mathsf{refs}([z_i \mapsto z_i']_{i\in I}) &= \bigcup_{i\in I} (\mathsf{refs}(z_i) \cup \mathsf{refs}(z_i')).\end{aligned} \tag{11.90}$$

The actor-residency check is then

$$\begin{aligned}&\mathsf{resident}_\Omega(A) = \mathrm{dom}(V) \cup \mathrm{dom}(\mathcal{H}(A)) \cup \mathrm{dom}(\Pi) \cup \mathrm{dom}(\Phi),\\ \mathsf{actorIsolationOK}(\Omega) \quad\Longleftrightarrow\quad &\forall A \in \mathrm{dom}(\mathcal{H}).\ \mathsf{refs}(\mathcal{X}(A)) \subseteq \mathsf{resident}_\Omega(A) \wedge\\ &\quad \forall x \in \mathrm{dom}(\mathcal{H}(A)).\ \mathsf{refs}(\mathcal{H}(A)(x)) \subseteq \mathsf{resident}_\Omega(A) \wedge\\ &\quad \forall x \in \mathrm{dom}(V).\ \mathsf{refs}(V(x)) \subseteq \mathrm{dom}(V) \ \wedge \mathsf{value}_{P,\mathsf{vmStore}(\Omega)}(x).\end{aligned} \tag{11.91}$$

The finite validation-kind domain, its interpretation, and its fixed order are

$$\begin{aligned}&\kappa_r ::= \mathsf{classGraphCheck} \mid \mathsf{objectStoreCheck} \mid \mathsf{liveActivationCheck} \mid \mathsf{activationEditCheck} \mid \mathsf{actorIsolationCheck},\\ &\mathsf{classGraphCheck} \lhd_{\mathsf{check}} \mathsf{objectStoreCheck} \lhd_{\mathsf{check}} \mathsf{liveActivationCheck},\\ &\mathsf{liveActivationCheck} \lhd_{\mathsf{check}} \mathsf{activationEditCheck} \lhd_{\mathsf{check}} \mathsf{actorIsolationCheck},\\ &\mathsf{validationHolds}(\mathsf{classGraphCheck},\Omega,\Delta,\Omega') \Longleftrightarrow \mathsf{classGraphOK}(P',H'),\\ &\mathsf{validationHolds}(\mathsf{objectStoreCheck},\Omega,\Delta,\Omega') \Longleftrightarrow \mathsf{objectStoreOK}(P',H'),\\ &\mathsf{validationHolds}(\mathsf{liveActivationCheck},\Omega,\Delta,\Omega') \Longleftrightarrow \mathsf{liveActivationsOK}(P',H',\mathcal{X}'),\\ &\mathsf{validationHolds}(\mathsf{activationEditCheck},\Omega,\Delta,\Omega') \Longleftrightarrow \mathsf{activationEditsOK}(\Omega,\Delta,\Omega'),\\ &\mathsf{validationHolds}(\mathsf{actorIsolationCheck},\Omega,\Delta,\Omega') \Longleftrightarrow \mathsf{actorIsolationOK}(\Omega').\end{aligned} \tag{11.92}$$

Define the failed validation set by

$$\mathsf{failedValidations}(\Omega, \Delta, \Omega') = \{\kappa_r \mid \neg\mathsf{validationHolds}(\kappa_r, \Omega, \Delta, \Omega')\}. \tag{11.93}$$

For $\Delta = \langle \delta_1, \ldots, \delta_m \rangle$, also define

$$\begin{aligned} \mathsf{unauthorizedIndices}(\Omega, A, \Delta) &= \{i \mid 1 \le i \le m \ \wedge \neg\mathsf{authorized}_{\Omega,A}(\delta_i)\}, \\ \mathsf{conflictPairs}(\Delta) &= \{(i,j) \mid \mathsf{conflict}_{\Delta}(i,j)\}. \end{aligned} \tag{11.94}$$

The validity predicate now has no informal conjuncts:

$$\begin{aligned} \mathsf{validReflection}(\Omega, A, \Delta, \Omega') \iff\ & \mathsf{prepareReflection}(\Omega, A, \Delta) = \Omega' \wedge \\ & \mathsf{unauthorizedIndices}(\Omega, A, \Delta) = \varnothing \wedge \\ & \mathsf{conflictPairs}(\Delta) = \varnothing \wedge \\ & \mathsf{failedValidations}(\Omega, \Delta, \Omega') = \varnothing. \end{aligned} \tag{11.95}$$

Successful preparation commits in one VM step:

$$\frac{\mathsf{validReflection}(\Omega, A, \Delta, \Omega')}{\Omega \xrightarrow{\mathsf{commitReflection}(A,\Delta,n+1)} \Omega'} \quad (\textsc{Reflection-Commit})$$

The rejection diagnostic domain is

$$\begin{aligned} \xi_r ::= \ & \mathsf{unauthorizedCommand}(i) \mid \mathsf{commandConflict}(i,j) \mid \mathsf{sourcePatchFailure} \mid \mathsf{elaborationFailure} \\ & \mid \mathsf{statePreparationFailure} \mid \mathsf{invariantFailure}(\kappa_r). \end{aligned} \tag{11.96}$$

For compactness, define the two partial candidates

$$\begin{aligned} \mathsf{reflectionSourceCandidate}(P, \Delta) &= \mathsf{applySourceCommands}(P, \mathsf{codeCommands}(\Delta)), \\ \mathsf{reflectionProgramCandidate}(P, \Delta) &= \mathsf{reelaborateProgram}(P, \mathsf{reflectionSourceCandidate}(P, \Delta)). \end{aligned} \tag{11.97}$$

Writing $\mathcal{U}_r = \mathsf{unauthorizedIndices}(\Omega, A, \Delta)$ and $\mathcal{C}_r = \mathsf{conflictPairs}(\Delta)$, the previously implicit failure function is the following partial function:

$$\mathsf{reflectionFailure}(\Omega, A, \Delta) = \begin{cases} \mathsf{unauthorizedCommand}(\min \mathcal{U}_r) & \mathcal{U}_r \neq \varnothing, \\ \mathsf{commandConflict}(i,j) & \mathcal{U}_r = \varnothing \ \wedge \ \mathcal{C}_r \neq \varnothing \wedge \\ & \quad (i,j) = \min_{\text{lex}} \mathcal{C}_r, \\ \mathsf{sourcePatchFailure} & \mathcal{U}_r = \mathcal{C}_r = \varnothing \wedge \\ & \quad \mathsf{reflectionSourceCandidate}(P, \Delta) \uparrow, \\ \mathsf{elaborationFailure} & \mathcal{U}_r = \mathcal{C}_r = \varnothing \wedge \\ & \quad \mathsf{reflectionSourceCandidate}(P, \Delta) \downarrow \ \wedge \\ & \quad \mathsf{reflectionProgramCandidate}(P, \Delta) \uparrow, \\ \mathsf{statePreparationFailure} & \mathcal{U}_r = \mathcal{C}_r = \varnothing \wedge \\ & \quad \mathsf{reflectionProgramCandidate}(P, \Delta) \downarrow \ \wedge \\ & \quad \mathsf{prepareReflection}(\Omega, A, \Delta) \uparrow, \\ \mathsf{invariantFailure}(\min_{\lhd_{\mathsf{check}}} \mathcal{B}_r) & \mathcal{U}_r = \mathcal{C}_r = \varnothing \wedge \\ & \quad \mathsf{prepareReflection}(\Omega, A, \Delta) = \Omega' \wedge \\ & \mathcal{B}_r = \mathsf{failedValidations}(\Omega, \Delta, \Omega') \neq \varnothing. \end{cases} \tag{11.98}$$

It is undefined exactly when $\mathsf{validReflection}(\Omega, A, \Delta, \Omega')$ holds for some Ω'. Failure leaves the state identical:

$$\frac{\mathsf{reflectionFailure}(\Omega, A, \Delta) = \xi}{\Omega \xrightarrow{\mathsf{rejectReflection}(A,\Delta,\xi)} \Omega} \quad \text{(Reflection-Reject)}$$

The case order in Equation (11.98) affects only the diagnostic; it cannot expose an intermediate state. Except for a successful Equation (11.61), whose requesting frame has itself been replaced, the mirror library reifies the commit receipt or failure as the return or exception of the primitive mirror message. Current-stack replacement instead continues directly from the installed top control.

11.10 Causal connection, turns, and other VMs

A reflective transition is a VM safepoint transition: it may occur between any two actor, delivery, or sequential steps, but never inside one such step. It replaces the single P in Equation (10.8); hence every actor in the cohort observes the same version $n + 1$ immediately. The following definition fixes the boundary between old and new behavior:

computation state at commit	program observed afterward
already materialized control t and frames π	their existing terms, code image, and control map
future method resolution or invocation	P'
mailbox or network packet not yet dispatched	P' at its later turn
closure object already allocated	its captured body in $H(c)$
class or object retaining stable M, C, i_f	the updated records addressed by those identities.

(11.99)

Thus an install never splices a new expression into a currently executing small step. A method invocation that has already entered its body continues that body; the next dispatch through the same stable method identity uses its new definition. In-flight asynchronous messages carry selectors and arguments, not resolved methods, so they naturally dispatch against the version current when Rule Turn-Start begins their turn.

Let a distributed system be

$$\mathfrak{F} : \mathit{VMId} \rightharpoonup \mathit{ActorConfiguration}, \qquad \mathfrak{F}(w).w = w. \tag{11.100}$$

Rule Reflection-Commit changes exactly one component $\mathfrak{F}(w)$. No rule copies P' into $\mathfrak{F}(w')$ for $w' \neq w$. Values in V are shared by every actor of w, so their behavior changes with the cohort's code; a value transferred to another VM is copied under Equation (10.3) and thereafter uses the recipient VM's independently versioned program. A remote VM may be changed only by a separate, authorized mirror request delivered to that VM. Its delivery is ordered like any other eventual message; the resulting local commit is atomic at the recipient safepoint.

The same rule governs remote debugging. An actor mirror transferred to another actor or VM remains a far capability. Its $\mathsf{actorStackQuery}$ result is transported by the remote-representation judgment; object references in a frame become values, far references, or authorized mirrors rather than raw near references. A remote controller first applies Equation (11.60), reads the resulting paused stack snapshot, and then names its pause token in Equation (11.63), (11.64), or (11.65). Formally, a remote debugger step has the component-wise shape

$$\frac{\mathfrak{F}(w), A, g \vdash \mathsf{debugRequest}(\delta) \Downarrow \mathfrak{F}'(w)}{\mathfrak{F} \longmapsto_{\mathsf{remoteDebug}} \mathfrak{F}[w \mapsto \mathfrak{F}'(w)]}, \qquad \forall v \neq w.\ \mathfrak{F}'(v) = \mathfrak{F}(v). \tag{11.101}$$

The request is delivered using the ordinary cross-VM transport and executes the corresponding local reflective command at w's safepoint. There is no distributed atomic transaction spanning several VMs: coordinated multi-actor debugging is a protocol of individually atomic pause, inspect, install, and resume operations. Pause tokens reject delayed or replayed commands after either an intervening edit or a resume.

Finally, VM primitives are ordinary messages to a mirror whose record has target $\mathsf{vmTarget}(w)$ and right $\mathsf{primitive}$. Their platform-specific state transformers lie outside this language semantics, but the capability premise is mandatory:

$$\frac{\widehat{H}_A(g) = \mathsf{mirror}\langle C_g, w, \mathsf{vmTarget}(w), R\rangle \qquad \mathsf{primitive} \in R \qquad \mathsf{hostPrimitive}(\Omega, s, \bar{o}) = \langle \Omega', v\rangle}{\Omega, A, g \vdash \mathsf{invokeVMPrimitive}(s, \bar{o}) \Downarrow \langle \Omega', v\rangle} \quad \text{(VM-Primitive)}$$

There is no primitive-call syntax and no bypass around the mirror record.

12 Optional garbage-collection layer

This section is an optional conservative extension of the VM semantics. It is kept in this separate source file, and no preceding rule depends on one of its auxiliary definitions. Omitting the single `\input{gc-semantics}` line restores the monotonically growing heaps of Sections 3–11. Including it replaces that implementation abstraction with an explicit, nondeterministic collector. The collector introduces no Newspeak syntax and never runs inside an actor step or reflective transaction.

The construction follows the explicit-heap rewrite method of Morrisett, Felleisen, and Harper, *Abstract Models of Memory Management* (1995), and the treatment of observable collection in Donnelly, Hallett, and Kfoury, *Formal Semantics of Weak References* (2006). The distinctive Newspeak observations are Equation (11.23), whose implicit instance registry is weak, and the platform classes $\mathsf{WeakArray}$ and $\mathsf{WeakMap}$. Collection can therefore change both later population snapshots and later reads of weak containers.

12.1 The collectable store

Garbage collection applies to ordinary heap records and to the routing records of promise and far-reference handles. For $\Omega = \langle P, V, \mathcal{H}, \mathcal{X}, \mathcal{Q}, \mathcal{N}, \Pi, \Phi, \Gamma, \mathcal{E}, \mathcal{D}, w, n\rangle$, define the disjoint reference store

$$\mathsf{refStore}(\Omega) = \mathsf{vmStore}(\Omega) \uplus \Pi \uplus \Phi. \tag{12.1}$$

The union is disjoint because its three domains are respectively subsets of *ObjRef*, *PromiseId*, and *FarRefId*. Put $\mathcal{S}_\Omega = \mathsf{refStore}(\Omega)$ and expose a uniform payload projection:

$$\mathsf{payload}_\Omega(x) = \begin{cases} \mathsf{vmStore}(\Omega)(x) & x \in \mathit{ObjRef}, \\ \Pi(x) & x \in \mathit{PromiseId}, \\ \Phi(x) & x \in \mathit{FarRefId}. \end{cases} \qquad x \in \mathrm{dom}(\mathcal{S}_\Omega). \tag{12.2}$$

Thus a live promise strongly retains its settlement value and registered waiters, and a live far-reference handle strongly retains its near target. The tables Π and Φ do not retain their own keys merely by containing them; otherwise every handle and every remote target would live forever.

12.2 WeakArray and WeakMap

WeakArray and WeakMap are mutable platform objects and therefore belong to exactly one actor heap. Their concrete VM layouts are abstracted by two additional object-record forms:

$$\begin{aligned} H_A(a_w) &= \mathsf{weakArray}\langle C, \langle y_1, \ldots, y_m\rangle\rangle, \quad && a_w \in \mathit{WeakArrayId}, \\ H_A(m_w) &= \mathsf{weakMap}\langle C, E_w\rangle, && m_w \in \mathit{WeakMapId}, \end{aligned} \tag{12.3}$$

where *WeakArrayId* and *WeakMapId* are disjoint subsets of *ObjRef*, every $y_i \in \mathit{Ref}$, and $E_w : \mathit{ObjRef} \rightharpoonup \mathit{Ref}$ is finite. C and any ordinary fixed fields of the implementation are strong; the sequence elements and map keys are weak. A map value is conditionally strong exactly while its key is strongly reachable. This is the usual ephemeron interpretation of a weak-key map: a value-to-key cycle cannot keep the key alive.

Every stored weak identity initially resolves; weakness controls retention, not whether an arbitrary dangling identity may be written:

$$\begin{aligned} \mathsf{weakArrayStoreOK}(\Omega) &\iff \forall A, a_w, C, \bar{y}.\ \mathcal{H}(A)(a_w) = \mathsf{weakArray}\langle C, \bar{y}\rangle \\ &\qquad \Longrightarrow \mathsf{refs}(\bar{y}) \subseteq \mathrm{dom}(\mathcal{S}_\Omega), \\ \mathsf{weakMapStoreOK}(\Omega) &\iff \forall A, m_w, C, E_w.\ \mathcal{H}(A)(m_w) = \mathsf{weakMap}\langle C, E_w\rangle \\ &\qquad \Longrightarrow \mathrm{dom}(E_w) \cup \mathsf{refs}(\mathsf{range}(E_w)) \subseteq \mathrm{dom}(\mathcal{S}_\Omega), \\ \mathsf{weakContainersOK}(\Omega) &\iff \mathsf{weakArrayStoreOK}(\Omega) \wedge \mathsf{weakMapStoreOK}(\Omega). \end{aligned} \tag{12.4}$$

The platform method contract is given by distinct partial store functions rather than overloaded notation:

$$\begin{aligned} \mathsf{weakArrayRead}(H_A, a_w, i) &= y_i, \\ \mathsf{writeWeakArrayCell}(H_A, a_w, i, v) &= H_A[a_w \mapsto \mathsf{weakArray}\langle C, \bar{y}[i \mapsto v]\rangle], \\ &\qquad 1 \le i \le |\bar{y}|. \end{aligned} \tag{12.5}$$

$$\mathsf{weakMapLookup}(H_A, m_w, k) = \begin{cases} E_w(k) & k \in \mathrm{dom}(E_w), \\ \mathsf{nil} & k \notin \mathrm{dom}(E_w), \end{cases} \tag{12.6}$$

$$\mathsf{writeWeakMapEntry}(H_A, m_w, k, v) = H_A[m_w \mapsto \mathsf{weakMap}\langle C, E_w[k \mapsto v]\rangle].$$

Here $H_A(a_w)$ and $H_A(m_w)$ have the respective forms of Equation (12.3). Allocation creates an all-nil array of the requested nonnegative size or a map with $E_w = \varnothing$, using the ordinary global-freshness obligation of Equation (12.15). The two writes are defined only when their new key and value references resolve in $\mathcal{S}_\Omega$, and they preserve Equation (12.4). Out-of-bounds array indices are undefined and are reified by the platform exception boundary. Storing nil is permitted. Since lookup also returns nil for an absent key, the public WeakMap protocol deliberately does not distinguish absence from a stored nil value.

The state outside the collectable store is

$$\mathsf{control}(\Omega) = \langle P, \mathcal{X}, \mathcal{Q}, \mathcal{N}, \Gamma, \mathcal{E}, \mathcal{D}, w, n\rangle. \tag{12.7}$$

Let $\mathcal{R}_h \subseteq \mathrm{dom}(\mathcal{S}_\Omega)$ be the finite set of strong references temporarily retained by the embedding host, such as arguments of a platform primitive that is atomic with respect to collection. A closed Newspeak execution has $\mathcal{R}_h = \varnothing$. First erase the weak components of a payload while preserving its class and ordinary fields:

$$\begin{aligned} \mathsf{strongPayload}(\mathsf{weakArray}\langle C, \bar{y}\rangle) &= \mathsf{weakArray}\langle C, \bar{\mathsf{nil}}\rangle, \\ \mathsf{strongPayload}(\mathsf{weakMap}\langle C, E_w\rangle) &= \mathsf{weakMap}\langle C, \varnothing\rangle, \\ \mathsf{strongPayload}(r) &= r \quad \text{for every other runtime record } r. \end{aligned} \tag{12.8}$$

The direct roots, unconditional strong successors, and map-value successors enabled by a candidate live set L are

$$\begin{aligned}
\mathsf{roots}_\Omega(\mathcal{R}_h) &= \big(\mathcal{R}_h \cup \mathsf{refs}(\mathsf{control}(\Omega))\big) \cap \mathrm{dom}(\mathcal{S}_\Omega),\\
\mathsf{strongSucc}_\Omega(x) &= \mathsf{refs}(\mathsf{strongPayload}(\mathsf{payload}_\Omega(x))) \cap \mathrm{dom}(\mathcal{S}_\Omega),\\
\mathsf{enabledMapValues}_\Omega(x, L) &= \begin{cases} \bigcup\limits_{\substack{k \in \mathrm{dom}(E_w)\\ k \in L}} \mathsf{refs}(E_w(k)) \cap \mathrm{dom}(\mathcal{S}_\Omega) & \mathsf{payload}_\Omega(x) = \mathsf{weakMap}\langle C, E_w\rangle,\\ \varnothing & \text{otherwise.}\end{cases}
\end{aligned} \tag{12.9}$$

The structural extractor refs is Equation (11.90). Static platform objects and atoms are roots through P; running activations and their terms are roots through $\mathcal{X}$; queued and in-flight arguments, reply promises, and settlement values are roots through $\mathcal{Q}$ and $\mathcal{N}$. Event identities and actor identities that do not occur in $\mathrm{dom}(\mathcal{S}_\Omega)$ are discarded by the intersection.

Strong reachability is the least fixed point

$$\begin{aligned}
\mathsf{reach}^0_\Omega(\mathcal{R}_h) &= \mathsf{roots}_\Omega(\mathcal{R}_h),\\
\mathsf{reach}^{i+1}_\Omega(\mathcal{R}_h) &= \mathsf{reach}^i_\Omega(\mathcal{R}_h) \cup \bigcup_{x \in \mathsf{reach}^i_\Omega(\mathcal{R}_h)} \big(\mathsf{strongSucc}_\Omega(x) \cup \mathsf{enabledMapValues}_\Omega(x, \mathsf{reach}^i_\Omega(\mathcal{R}_h))\big),\\
\mathsf{live}_\Omega(\mathcal{R}_h) &= \bigcup_{i \in \mathbb{N}} \mathsf{reach}^i_\Omega(\mathcal{R}_h),\\
\mathsf{garbage}_\Omega(\mathcal{R}_h) &= \mathrm{dom}(\mathcal{S}_\Omega) \setminus \mathsf{live}_\Omega(\mathcal{R}_h).
\end{aligned} \tag{12.10}$$

All stores are finite, so this sequence reaches a fixed point. Cycles not reachable from a root are garbage together; reference counting is neither assumed nor implied. Weak-array elements never enter the fixed point merely because their array is live. A weak-map value enters only after both its map and its key have entered; consequently a value that points back to its otherwise unreachable key does not activate its own entry.

12.3 Admissible collection and identity history

A collector need not reclaim all eligible records. It may reclaim a nonempty set G, provided no retained record would contain a dangling strong reference:

$$\begin{aligned}
\mathsf{admissibleGC}_{\Omega,\mathcal{R}_h}(G) \quad \Longleftrightarrow \quad & \varnothing \neq G \subseteq \mathsf{garbage}_\Omega(\mathcal{R}_h) \wedge\\
& \forall x \in \mathrm{dom}(\mathcal{S}_\Omega) \setminus G.\ \mathsf{strongSucc}_\Omega(x) \cap G = \varnothing.
\end{aligned} \tag{12.11}$$

The second conjunct matters because $\mathsf{allInstancesOfQuery}$ can expose a retained but previously unreachable object. It rules out retaining such an object while collecting another object to which one of its ordinary slots still points. Taking all of $\mathsf{garbage}_\Omega(\mathcal{R}_h)$ always satisfies this condition; a generational or incremental collector may choose a smaller closed subset.

Weak edges are intentionally absent from the closure premise. If a selected record is named by a retained weak array or by an inactive weak-map entry, the collector clears that weak observation rather than rejecting the collection. An enabled weak-map value cannot belong to G because Equation (12.10) already makes it live.

Define weak-edge clearing on a retained payload by

$$\begin{aligned}
\mathsf{clearWeak}_G(\mathsf{weakArray}\langle C, \langle y_1, \ldots, y_m\rangle\rangle) &= \mathsf{weakArray}\langle C, \langle y'_1, \ldots, y'_m\rangle\rangle,\\
y'_i &= \begin{cases} \mathsf{nil} & \mathsf{refs}(y_i) \cap G \neq \varnothing,\\ y_i & \mathsf{refs}(y_i) \cap G = \varnothing, \end{cases}\\
\mathsf{clearWeak}_G(\mathsf{weakMap}\langle C, E_w\rangle) &= \mathsf{weakMap}\langle C, E'_w\rangle,\\
E'_w &= E_w|_{\{k \mid k \notin G \ \wedge\ \mathsf{refs}(E_w(k)) \cap G = \varnothing\}},\\
\mathsf{clearWeak}_G(r) &= r \quad \text{for every other runtime record } r.
\end{aligned} \tag{12.12}$$

The value-side test on E'_w permits partial collection to remove an inactive entry's value while leaving its still-unreachable key for a later collection; no retained weak record then contains a dangling identity.

For $K = \mathrm{dom}(\mathcal{S}_\Omega) \setminus G$, let

$$\mathsf{prunedActorHeap}(H_A, G, K) = \{x \mapsto \mathsf{clearWeak}_G(H_A(x)) \mid x \in \mathrm{dom}(H_A) \cap K\}. \tag{12.13}$$

Store pruning is the total operation

$$\begin{aligned}
\mathsf{prune}(\Omega, G) = \Omega[V \mapsto V|_K, \quad \mathcal{H}(A) \mapsto \mathsf{prunedActorHeap}(\mathcal{H}(A), G, K) \text{ for every } A \in \mathrm{dom}(\mathcal{H}),\\
\Pi \mapsto \Pi|_K, \quad \Phi \mapsto \Phi|_K].
\end{aligned} \tag{12.14}$$

Map restriction is by key. Every other component, including causal and event history, is unchanged.

Deleting a record must not make its identity fresh again. Rather than adding an allocation ledger to every existing configuration and rule, the optional layer wraps the VM state:

$$\Psi = \langle \Omega, \mathcal{I}\rangle, \qquad \mathrm{dom}(\mathcal{S}_\Omega) \subseteq \mathcal{I} \subseteq \mathit{Ref}, \tag{12.15}$$

where $\mathcal{I}$ is the finite set of reference identities allocated so far in this execution. Let the combined existing VM transition be the disjoint union

$$\begin{aligned}
&\Omega \Longrightarrow^{\mathsf{actorStep}} \Omega' \iff \Omega \longrightarrow_{\mathsf{act}} \Omega',\\
&\Omega \Longrightarrow^{\lambda_r} \Omega' \iff \Omega \xrightarrow{\lambda_r} \Omega', \quad \lambda_r ::= \mathsf{commitReflection}(A, \Delta, n) \mid \mathsf{rejectReflection}(A, \Delta, \xi_r).
\end{aligned} \tag{12.16}$$

For one such transition define

$$\mathsf{newRefs}(\Omega, \Omega') = \mathrm{dom}(\mathcal{S}_{\Omega'}) \setminus \mathrm{dom}(\mathcal{S}_\Omega). \tag{12.17}$$

The ordinary transition lifts only when all newly allocated identities have never been used:

$$\frac{\Omega \Longrightarrow^{\lambda} \Omega' \qquad \mathsf{newRefs}(\Omega, \Omega') \cap \mathcal{I} = \varnothing}{\mathcal{R}_h \vdash \langle \Omega, \mathcal{I}\rangle \Longrightarrow^{\lambda}_{\mathsf{gc}} \langle \Omega', \mathcal{I} \cup \mathsf{newRefs}(\Omega, \Omega')\rangle} \quad \text{(GC-Lift)}$$

This gives every existing "fresh" premise its intended global meaning while leaving the underlying rules unchanged. An initial wrapped configuration has $\mathcal{I} = \mathrm{dom}(\mathcal{S}_{\Omega_0})$.

Collection is an independently enabled VM-safepoint transition:

$$\frac{\mathsf{admissibleGC}_{\Omega, \mathcal{R}_h}(G)}{\mathcal{R}_h \vdash \langle \Omega, \mathcal{I}\rangle \longrightarrow_{\mathsf{gc}} \langle \mathsf{prune}(\Omega, G), \mathcal{I}\rangle} \quad \text{(GC-Collect)}$$

There is deliberately no fairness requirement for GC-Collect: an eligible object may remain uncollected indefinitely. The implementation chooses both when to collect and which admissible closed garbage region to reclaim.

12.4 Observable consequences

Let $\Omega' = \mathsf{prune}(\Omega, G)$. Equations (12.10)–(12.14) immediately give the safety obligations

$$\begin{aligned} \mathsf{live}_{\Omega}(\mathcal{R}_h) &\subseteq \mathrm{dom}(\mathcal{S}_{\Omega'}), \\ \forall x \in \mathrm{dom}(\mathcal{S}_{\Omega'}). \quad \mathsf{refs}(\mathsf{payload}_{\Omega'}(x)) \cap \mathit{Ref} &\subseteq \mathrm{dom}(\mathcal{S}_{\Omega'}). \end{aligned} \tag{12.18}$$

The first property says that no strongly reachable record is reclaimed. The second says that every retained encoded reference still resolves: selected weak targets have already been replaced by nil or removed from their map. Together with Equation (12.15), they state memory safety and permanent identity freshness independently of any particular tracing algorithm.

For exact-instance enumeration, collection has the intentionally observable effect

$$\mathsf{allInstancesOf}_{\Omega'}(C) = \mathsf{filter}_{x \notin G}\big(\mathsf{allInstancesOf}_{\Omega}(C)\big). \tag{12.19}$$

For mixin-application enumeration, Equation (11.24) similarly gives

$$\mathsf{mixinApplications}_{\Omega'}(M) = \mathsf{filter}_{C \notin G}\big(\mathsf{mixinApplications}_{\Omega}(M)\big). \tag{12.20}$$

Let $H_A = \mathcal{H}(A)$ and $H'_A = \mathsf{prunedActorHeap}(H_A, G, K)$. For a retained weak array, Equation (12.12) gives the observable read

$$\begin{aligned} y &= \mathsf{weakArrayRead}(H_A, a_w, i), \qquad a_w \in K, \\ \mathsf{weakArrayRead}(H'_A, a_w, i) &= \begin{cases} \mathsf{nil} & \mathsf{refs}(y) \cap G \neq \varnothing, \\ y & \text{otherwise.} \end{cases} \end{aligned} \tag{12.21}$$

For a retained weak map, an entry disappears when collection removes its key or any reference in its value:

$$\begin{aligned} v &= \mathsf{weakMapLookup}(H_A, m_w, k), \qquad m_w \in K, \\ \mathsf{weakMapLookup}(H'_A, m_w, k) &= \begin{cases} \mathsf{nil} & k \in G \ \vee \ \mathsf{refs}(v) \cap G \neq \varnothing, \\ v & \text{otherwise.} \end{cases} \end{aligned} \tag{12.22}$$

These changes occur atomically with GC-Collect; no program step can observe a cleared weak cell while the corresponding target record is still being removed. A strongly reachable key keeps its weak-map value live through Equation (12.10), while neither the map nor its value keeps an otherwise unreachable key live. Consequently an unreachable instance or class application can appear in a snapshot before a GC step and be absent after it. Mirror inspection itself is atomic with respect to GC-Collect; once a returned sequence is installed in the caller's activation or heap, its elements are ordinary strong references and are roots of the next collection through $\mathcal{X}$ or a heap record. Nothing can be resurrected after its record has been removed.

If programs have no GC-observing primitive, removing unreachable records is expected to be a stuttering refinement of the uncollected semantics. In this semantics the population queries deliberately invalidate that premise, so the timing of collection is part of the permitted observable nondeterminism. Finalizers, if later added to the Newspeak platform, must extend the collection rule here rather than modify the base evaluator.

13 Source front ends and metacircular compilation

This section closes the boundary left abstract in Equation (9.22). It has two separate purposes. First, it defines the relation from source text through PEG recognition, surface AST construction, stable source identities, and elaboration to the annotated core used by the preceding rules. Second, it makes the implementations of those stages ordinary Newspeak objects. The latter introduces no new reflective command: changing a parser, elaborator, or compiler is an ordinary change to its methods or object graph under Section 11.

13.1 Source text and PEG recognition

Let $u \in \mathit{SourceText} = \mathit{UnicodeScalar}^*$, let $0 \le i \le |u|$ be a source offset, let $N \in \mathit{Nonterminal}$, and let $X \subseteq \mathit{UnicodeScalar}$. Character ranges, optional expressions, positive closure, and the Specification's prefix ˜ notation are abbreviations for the following PEG core:

$$r ::= \epsilon \mid \mathsf{char}(c) \mid \mathsf{set}(X) \mid \mathsf{any} \mid N \mid r_1 r_2 \mid r_1 / r_2 \mid r^* \mid !r \mid \&r \mid \mathsf{capture}(n, r). \tag{13.1}$$

A grammar is $G = \langle N_0, \rho_G \rangle$, where N_0 is its start symbol and $\rho_G : \mathit{Nonterminal} \rightharpoonup \mathit{PEGExpr}$ is finite. A concrete tree is

$$t ::= \mathsf{leaf}\langle i, j \rangle \mid \mathsf{node}\langle n, i, j, \bar{t} \rangle, \qquad \mathsf{span}(t) = [i, j). \tag{13.2}$$

Uncaptured recognition returns an ordered forest. Write $\mathsf{ok}(j, \bar{t})$ for success and $\mathsf{fail}(i)$ for failure at offset i. The deterministic function $\mathsf{peg}_G(r, u, i)$ is defined by the following exhaustive equations. The failure offset is diagnostic only; ordered choice always restarts its right alternative at the original offset.

$$\begin{aligned}
\mathsf{peg}_G(\epsilon, u, i) &= \mathsf{ok}(i, \epsilon), \\
\mathsf{peg}_G(\mathsf{char}(c), u, i) &= \begin{cases} \mathsf{ok}(i+1, \langle \mathsf{leaf}\langle i, i+1 \rangle \rangle) & i < |u| \wedge u_i = c, \\ \mathsf{fail}(i) & \text{otherwise}, \end{cases} \\
\mathsf{peg}_G(\mathsf{set}(X), u, i) &= \begin{cases} \mathsf{ok}(i+1, \langle \mathsf{leaf}\langle i, i+1 \rangle \rangle) & i < |u| \wedge u_i \in X, \\ \mathsf{fail}(i) & \text{otherwise}, \end{cases} \\
\mathsf{peg}_G(\mathsf{any}, u, i) &= \begin{cases} \mathsf{ok}(i+1, \langle \mathsf{leaf}\langle i, i+1 \rangle \rangle) & i < |u|, \\ \mathsf{fail}(i) & i = |u|. \end{cases}
\end{aligned} \tag{13.3}$$

$$\begin{aligned}
\mathsf{peg}_G(r_1 r_2, u, i) &= \begin{cases} \mathsf{ok}(k, \bar{t}_1 \bar{t}_2) & \mathsf{peg}_G(r_1, u, i) = \mathsf{ok}(j, \bar{t}_1) \wedge \mathsf{peg}_G(r_2, u, j) = \mathsf{ok}(k, \bar{t}_2), \\ \mathsf{fail}(h) & \mathsf{peg}_G(r_1, u, i) = \mathsf{fail}(h), \\ \mathsf{fail}(h) & \mathsf{peg}_G(r_1, u, i) = \mathsf{ok}(j, \bar{t}_1) \wedge \mathsf{peg}_G(r_2, u, j) = \mathsf{fail}(h), \end{cases} \\
\mathsf{peg}_G(r_1 / r_2, u, i) &= \begin{cases} q & \mathsf{peg}_G(r_1, u, i) = q = \mathsf{ok}(j, \bar{t}), \\ \mathsf{peg}_G(r_2, u, i) & \mathsf{peg}_G(r_1, u, i) = \mathsf{fail}(h). \end{cases}
\end{aligned} \tag{13.4}$$

Greedy repetition and lookahead are

$$\mathsf{peg}_G(r^*,u,i) = \begin{cases} \mathsf{ok}(i,\epsilon) & \mathsf{peg}_G(r,u,i) = \mathsf{fail}(h), \\ \mathsf{ok}(k,\overline{tt'}) & \mathsf{peg}_G(r,u,i) = \mathsf{ok}(j,\bar{t}) \wedge j > i \wedge \mathsf{peg}_G(r^*,u,j) = \mathsf{ok}(k,\bar{t'}), \end{cases}$$

$$\mathsf{peg}_G(!r,u,i) = \begin{cases} \mathsf{ok}(i,\epsilon) & \mathsf{peg}_G(r,u,i) = \mathsf{fail}(h), \\ \mathsf{fail}(i) & \mathsf{peg}_G(r,u,i) = \mathsf{ok}(j,\bar{t}), \end{cases} \tag{13.5}$$

$$\mathsf{peg}_G(\&r,u,i) = \begin{cases} \mathsf{ok}(i,\epsilon) & \mathsf{peg}_G(r,u,i) = \mathsf{ok}(j,\bar{t}), \\ \mathsf{fail}(i) & \mathsf{peg}_G(r,u,i) = \mathsf{fail}(h). \end{cases}$$

Nonterminal invocation and capture are

$$\mathsf{peg}_G(N,u,i) = \mathsf{peg}_G(\rho_G(N),u,i),$$

$$\mathsf{peg}_G(\mathsf{capture}(n,r),u,i) = \begin{cases} \mathsf{ok}(j,\langle \mathsf{node}\langle n,i,j,\bar{t}\rangle\rangle) & \mathsf{peg}_G(r,u,i) = \mathsf{ok}(j,\bar{t}), \\ \mathsf{fail}(h) & \mathsf{peg}_G(r,u,i) = \mathsf{fail}(h). \end{cases} \tag{13.6}$$

Undefined nonterminals are failures. The grammar admissibility predicate is

$$\begin{aligned} \mathsf{pegOK}(G) \iff N_0 \in \mathrm{dom}(\rho_G) \ \wedge \ \mathsf{refsNT}(\mathsf{range}(\rho_G)) \subseteq \mathrm{dom}(\rho_G) \ \wedge \\ \neg\exists N.\ N\ \mathsf{lead}_G^+\ N \ \wedge \ \forall r^* \preceq \mathsf{range}(\rho_G).\ \neg\mathsf{nullable}_G(r). \end{aligned} \tag{13.7}$$

Here $\mathsf{nullable}_G$ is the least predicate obtained by applying Equations (13.3)–(13.6) to empty input, and $N\ \mathsf{lead}_G\ N'$ when N' can be invoked from $\rho_G(N)$ without consuming a character. Thus Equation (13.7) excludes left recursion and a nullable operand of repetition; the recursive calls above consequently terminate. The mechanization checks this relational predicate by a finite certificate. Let $\nu_G : N \to \mathbb{B}$ be the computed nullable upper approximation and let $r_G : N \to \mathbb{N}$ be the computed rank of leading nonterminal calls. Write $\mathsf{nullable}_\nu^+(q)$ for the structural Boolean approximation that interprets a nonterminal N as $\nu(N)$, and write $\mathsf{lead}_\nu^+(q)$ for the finite set of nonterminals reachable at the head of q under the same approximation. The executable certificate is

$$\begin{aligned} \mathsf{pegCertOK}(G) \iff N_0 \in \mathrm{dom}(\rho_G) \ \wedge \\ \forall N \in \mathrm{dom}(\rho_G).\ \rho_G(N) = b \Rightarrow \\ \quad \mathsf{refsNT}(b) \subseteq \mathrm{dom}(\rho_G) \ \wedge \\ \quad (\mathsf{nullable}_{\nu_G}^+(b) \Rightarrow \nu_G(N)) \ \wedge \\ \quad \forall q^* \preceq b.\ \neg\mathsf{nullable}_{\nu_G}^+(q) \ \wedge \\ \quad \forall N' \in \mathsf{lead}_{\nu_G}^+(b).\ r_G(N') < r_G(N). \end{aligned} \tag{13.8}$$

In the fourth line q is the operand of that particular occurrence q^*. Character-set predicates are erased only while computing this structural certificate: they cannot recognize empty input and therefore cannot affect any of its four analyses. The certificate checker and its semantic consequence are

$$\mathsf{pegCertOK}(G) = \mathsf{true} \Rightarrow \mathsf{pegOK}(G), \qquad \mathsf{pegCertOK}(G_{\textsc{Newspeak}}) = \mathsf{true}. \tag{13.9}$$

Thus the normalized grammar below is not merely assumed admissible; its finite certificate is evaluated and the checker is proved sound for Equation (13.7). Complete recognition is

$$\mathsf{parse}_G(u) = t \quad \iff \quad \mathsf{pegOK}(G) \wedge \mathsf{peg}_G(N_0,u,0) = \mathsf{ok}(|u|,\langle t\rangle). \tag{13.10}$$

The remainder of this subsection gives the distinguished grammar explicitly. For compactness, the following are definitions, not additional PEG operators:

$$\begin{gathered} r? \equiv r/\epsilon, \qquad r^{+} \equiv rr^{*}, \qquad \mathsf{lit}(c_1 \cdots c_n) \equiv \mathsf{char}(c_1) \cdots \mathsf{char}(c_n), \\ \mathsf{triv} \equiv (\mathsf{white}/\mathsf{comment})^{*}, \qquad \mathsf{tok}(w) \equiv \mathsf{triv}\,\mathsf{lit}(w), \qquad \mathsf{word}(w) \equiv \mathsf{triv}\,\mathsf{lit}(w)\,!\mathsf{idRest}. \end{gathered} \tag{13.11}$$

white recognizes one Unicode `White_Space` scalar. The finite punctuation family is

$$\begin{aligned} \rho_{\textsc{Newspeak}}(T_p) &= \mathsf{tok}(p), \qquad p \in \mathcal{P}, \\ \mathcal{P} &= \{\texttt{:}, \texttt{,}, \texttt{.}, \texttt{||}, \texttt{=}, \texttt{\^{}}, \texttt{[}, \{, \texttt{(}, \texttt{<}, \texttt{<:}, \\ &\quad \texttt{\#}, \texttt{>}, \texttt{]}, \}, \texttt{)}, \texttt{;}, \texttt{/}, \texttt{|}, \texttt{::=}, \texttt{<-:}\}. \end{aligned} \tag{13.12}$$

The longest member is selected before any of its prefixes. This resolves the otherwise implicit lexical priority of `||`, `::=`, `<:`, and `<-:`.

Let L, U, D be the ASCII lower-case, upper-case, and decimal-digit sets, $A = L \cup U$, and let Q be the Specification's finite *specialCharacter* set. The lexical part of $G_{\textsc{Newspeak}}$ is

$$\begin{aligned} \rho_{\textsc{Newspeak}}(\mathsf{idRest}) &= \mathsf{set}(A \cup D)/\mathsf{char}(_), \\ \rho_{\textsc{Newspeak}}(\mathsf{rawId}) &= (\mathsf{set}(A)/\mathsf{char}(_))\mathsf{idRest}^{*}, \\ \rho_{\textsc{Newspeak}}(\mathsf{id}) &= \mathsf{triv}\,!\mathsf{reservedWord}\mathsf{rawId}, \\ \rho_{\textsc{Newspeak}}(\mathsf{rawKeyword}) &= \mathsf{rawId}\mathsf{lit}(:), \\ \rho_{\textsc{Newspeak}}(\mathsf{keyword}) &= \mathsf{triv}\,\mathsf{rawKeyword}, \\ \rho_{\textsc{Newspeak}}(\mathsf{setterKeyword}) &= \mathsf{triv}\,\mathsf{rawKeyword}\mathsf{lit}(:), \\ \rho_{\textsc{Newspeak}}(\mathsf{keywords}) &= \mathsf{rawKeyword}^{+}, \\ \rho_{\textsc{Newspeak}}(\mathsf{rawBinary}) &= (\mathsf{set}(Q)/\mathsf{lit}(-))\mathsf{set}(Q)^{*}, \\ \rho_{\textsc{Newspeak}}(\mathsf{binarySelector}) &= \mathsf{triv}\,\mathsf{rawBinary}, \\ \rho_{\textsc{Newspeak}}(\mathsf{metadataTag}) &= \mathsf{lit}(:)\mathsf{rawId}\mathsf{lit}(:), \\ \rho_{\textsc{Newspeak}}(\mathsf{commentItem}) &= (!\mathsf{lit}(*))\,\mathsf{any})/\mathsf{comment}, \\ \rho_{\textsc{Newspeak}}(\mathsf{comment}) &= \mathsf{lit}((*)\,\mathsf{metadataTag}? \\ &\qquad \mathsf{commentItem}^{*}\mathsf{lit}(*)). \end{aligned} \tag{13.13}$$

Reserved words are recognized by the dedicated tokens selfToken, superToken, outerToken, trueToken, falseToken, and nilToken; id additionally has negative lookahead for those complete words. Thus the prose reservation is part of the grammar rather than a later implementation convention. The dedicated-word productions are

$$\begin{aligned} \rho_{\textsc{Newspeak}}(\mathsf{reservedWord}) &= \mathsf{word}(\texttt{self})/\mathsf{word}(\texttt{super})/\mathsf{word}(\texttt{outer})/ \\ &\qquad \mathsf{word}(\texttt{true})/\mathsf{word}(\texttt{false})/\mathsf{word}(\texttt{nil}), \\ \rho_{\textsc{Newspeak}}(\mathsf{selfToken}) &= \mathsf{word}(\texttt{self}), \\ \rho_{\textsc{Newspeak}}(\mathsf{superToken}) &= \mathsf{word}(\texttt{super}), \\ \rho_{\textsc{Newspeak}}(\mathsf{outerToken}) &= \mathsf{word}(\texttt{outer}), \\ \rho_{\textsc{Newspeak}}(\mathsf{trueToken}) &= \mathsf{word}(\texttt{true}), \\ \rho_{\textsc{Newspeak}}(\mathsf{falseToken}) &= \mathsf{word}(\texttt{false}), \\ \rho_{\textsc{Newspeak}}(\mathsf{nilToken}) &= \mathsf{word}(\texttt{nil}), \\ \rho_{\textsc{Newspeak}}(\mathsf{classToken}) &= \mathsf{word}(\texttt{class}), \\ \rho_{\textsc{Newspeak}}(\mathsf{lazyModifier}) &= \mathsf{word}(\texttt{lazy}). \end{aligned} \tag{13.14}$$

Numeric, string, and symbol lexemes are defined by

$$\begin{aligned}
\mathsf{digits} &= \mathsf{set}(D)^{+},\\
\mathsf{extendedDigits} &= \mathsf{set}(D \cup U)^{+},\\
\mathsf{fraction} &= T_{\cdot}\,\mathsf{digits},\\
\mathsf{extendedFraction} &= T_{\cdot}\,\mathsf{extendedDigits},\\
\mathsf{exponent} &= \mathsf{lit}(e)\mathsf{lit}(-)?\mathsf{digits},\\
\mathsf{decimalNum} &= \mathsf{lit}(-)?\mathsf{digitsfraction}?\mathsf{exponent}?,\\
\mathsf{radixNum} &= \mathsf{digitslit}(r)\mathsf{lit}(-)?\mathsf{extendedDigitsextendedFraction}?\mathsf{exponent}?,\\
\rho_{\textsc{Newspeak}}(\mathsf{number}) &= \mathsf{triv}(\mathsf{radixNum}/\mathsf{decimalNum}),\\
\rho_{\textsc{Newspeak}}(\mathsf{singleString}) &= \mathsf{lit}(')((\mathsf{lit}(''))/(!\mathsf{lit}(')\mathsf{any}))^{*}\mathsf{lit}('),\\
\rho_{\textsc{Newspeak}}(\mathsf{doubleString}) &= \mathsf{lit}(")((\mathsf{lit}(""))/(!\mathsf{lit}(")\mathsf{any}))^{*}\mathsf{lit}("),\\
\rho_{\textsc{Newspeak}}(\mathsf{string}) &= \mathsf{triv}(\mathsf{singleString}/\mathsf{doubleString}),\\
\rho_{\textsc{Newspeak}}(\mathsf{symbol}) &= \mathsf{triv}\,T_{\#}\\
&\qquad (\mathsf{string}/\mathsf{keywords}/\mathsf{rawBinary}/\mathsf{rawId}).
\end{aligned} \tag{13.15}$$

The action in Equation (13.29) below performs radix and escape decoding and NFC normalization; recognition itself loses no source spelling.

Optional type annotations affect parsing, tooling, and reflection but not the untyped dynamic kernel. Let

$$\begin{aligned}
\mathcal{W}_T = \{&\texttt{arg}, \texttt{for}, \texttt{generic}, \texttt{inheritedTypeOf}, \texttt{is}, \texttt{message}, \texttt{of},\\
&\texttt{receiverType}, \texttt{subtypeOf}, \texttt{typeArg}, \texttt{where}\}.
\end{aligned} \tag{13.16}$$

Their complete PEG is

$$\begin{aligned}
\rho_{\textsc{Newspeak}}(T_w) &= \mathsf{word}(w) \quad (w \in \mathcal{W}_T),\\
\rho_{\textsc{Newspeak}}(\mathsf{returnType}) &= T_{\text{-}}\mathsf{type},\\
\rho_{\textsc{Newspeak}}(\mathsf{type}) &= T_{<}\mathsf{typeExpr}T_{>},\\
\rho_{\textsc{Newspeak}}(\mathsf{typePrimary}) &= \mathsf{id}\mathsf{typeArguments}?,\\
\rho_{\textsc{Newspeak}}(\mathsf{typeFactor}) &= \mathsf{typePrimary}/\mathsf{blockType}/\\
&\qquad \mathsf{tupleType}/\mathsf{parenthesizedTypeExpression},\\
\rho_{\textsc{Newspeak}}(\mathsf{parenthesizedTypeExpression}) &= T_{(}\mathsf{typeExpr}T_{)},\\
\rho_{\textsc{Newspeak}}(\mathsf{typeTerm}) &= \mathsf{typeFactor}\mathsf{id}^{*},\\
\rho_{\textsc{Newspeak}}(\mathsf{typeExpr}) &= \mathsf{typeTerm}\\
&\qquad ((T_{|}/T_{;}/T_{/})\mathsf{typeExpr})?,\\
\rho_{\textsc{Newspeak}}(\mathsf{typeArguments}) &= T_{[}\mathsf{typeExpr}(T_{,}\mathsf{typeExpr})^{*}T_{]},\\
\rho_{\textsc{Newspeak}}(\mathsf{tupleType}) &= T_{\{}(\mathsf{typeExpr}(T_{.}\mathsf{typeExpr})^{*})?T_{\}},\\
\rho_{\textsc{Newspeak}}(\mathsf{blockArgType}) &= T_{:}\mathsf{typeTerm},\\
\rho_{\textsc{Newspeak}}(\mathsf{blockReturnType}) &= \mathsf{typeExpr},\\
\rho_{\textsc{Newspeak}}(\mathsf{nonEmptyBlockArgList}) &= \mathsf{blockArgType}^{+}(T_{|}\,\mathsf{blockReturnType})?,\\
\rho_{\textsc{Newspeak}}(\mathsf{blockType}) &= T_{[}(\mathsf{nonEmptyBlockArgList}/\mathsf{blockReturnType}?)T_{]}.
\end{aligned} \tag{13.17}$$

Type parameters and inference patterns are

$$\begin{aligned}
\rho_{\textsc{Newspeak}}(\mathsf{typeParameterDecl}) &= T_{\texttt{<}} T_{\texttt{[}} \mathsf{id}(T_{\texttt{,}} \mathsf{id})^* T_{\texttt{]}} T_{\texttt{>}},\\
\rho_{\textsc{Newspeak}}(\mathsf{typePattern}) &= T_{\texttt{<}} \mathsf{typeFormal}(T_{\texttt{;}} \mathsf{typeFormal})^* T_{\texttt{>}},\\
\rho_{\textsc{Newspeak}}(\mathsf{typeFormal}) &= T_{\texttt{where}} \mathsf{id} \mathsf{typeParamConstraint}? T_{\texttt{is}} \mathsf{inferenceClause},\\
\rho_{\textsc{Newspeak}}(\mathsf{typeParamConstraint}) &= T_{\texttt{<}} \mathsf{typeBoundQualifier}? \mathsf{typeExpr} T_{\texttt{>}},\\
\rho_{\textsc{Newspeak}}(\mathsf{typeBoundQualifier}) &= T_{\texttt{subtypeOf}} / T_{\texttt{inheritedTypeOf}},\\
\rho_{\textsc{Newspeak}}(\mathsf{inferenceClause}) &= T_{\texttt{receiverType}} / (\mathsf{returnType} \mathsf{returnTypeInferenceClause}) / \mathsf{typeArgInferenceClaus}\\
&\quad (T_{\texttt{arg}} \mathsf{number}(T_{\texttt{of}} \mathsf{msgSelector})?),\\
\rho_{\textsc{Newspeak}}(\mathsf{returnTypeInferenceClause}) &= T_{\texttt{of}} \mathsf{msgSelector},\\
\rho_{\textsc{Newspeak}}(\mathsf{msgSelector}) &= \mathsf{symbol} T_{\texttt{message}} T_{\texttt{of}} \mathsf{inferenceClause},\\
\rho_{\textsc{Newspeak}}(\mathsf{typeArgInferenceClause}) &= T_{\texttt{typeArg}} \mathsf{number} T_{\texttt{for}} T_{\texttt{generic}}\\
&\quad \mathsf{symbol} T_{\texttt{of}} \mathsf{inferenceClause}.
\end{aligned} \tag{13.18}$$

Let $\mathcal{T}_I$ contain every nonterminal on the left of Equation (13.17) except type, together with every left-hand nonterminal of Equation (13.18) except typeParameterDecl and typePattern. Type syntax is retained as source metadata by

$$\begin{aligned}
\mathsf{staticTypeToken}_z(n, w) &= \langle \mathsf{staticType}, \langle n, w \rangle, z \rangle,\\
\mathcal{J}_{\textsc{Newspeak}}(n, \bar{v}, w) &= \mathsf{erase}\\
&\quad (n \in \mathcal{T}_I \cup \{T_w \mid w \in \mathcal{W}_T\}),\\
\mathcal{J}_{\textsc{Newspeak}}(n, \bar{v}, w) &= \mathsf{metadata}(\mathsf{staticTypeToken}_z(n, w)) \quad (n \in \{\mathsf{type}, \mathsf{typeParameterDecl}, \mathsf{typePattern}\}).
\end{aligned} \tag{13.19}$$

The message and precedence productions are the following PEG. They include the asynchronous-send token required by Specification §5.3.7, which the printed 0.109 grammar describes in prose

but omits from its displayed productions.

$$
\begin{aligned}
\rho_{\textsc{Newspeak}}(\mathsf{unarySelector}) &= \mathsf{id},\\
\rho_{\textsc{Newspeak}}(\mathsf{binaryMessage}) &= \mathsf{binarySelector\ unaryExpression},\\
\rho_{\textsc{Newspeak}}(\mathsf{keywordMessage}) &= (\mathsf{keyword\ binaryExpression})^{+},\\
\rho_{\textsc{Newspeak}}(\mathsf{message}) &= \mathsf{keywordMessage/binaryMessage/unarySelector},\\
\rho_{\textsc{Newspeak}}(\mathsf{receiverPrimary}) &= \mathsf{outerReceiver/superToken/primary},\\
\rho_{\textsc{Newspeak}}(\mathsf{unaryExpression}) &= \mathsf{receiverPrimary\ unarySelector}^{*},\\
\rho_{\textsc{Newspeak}}(\mathsf{binaryExpression}) &= \mathsf{unaryExpression\ binaryMessage}^{*},\\
\rho_{\textsc{Newspeak}}(\mathsf{keywordExpression}) &= \mathsf{binaryExpression\ keywordMessage?},\\
\rho_{\textsc{Newspeak}}(\mathsf{eventualExpression}) &= \mathsf{keywordExpression}\ T_{\texttt{<-:}}\ \mathsf{message},\\
\rho_{\textsc{Newspeak}}(\mathsf{cascadeMessage}) &= T_{\texttt{;}}(\mathsf{keywordMessage/binaryMessage/unarySelector}),\\
\rho_{\textsc{Newspeak}}(\mathsf{implicitKeywordSend}) &= \mathsf{keywordMessage},\\
\rho_{\textsc{Newspeak}}(\mathsf{nontrivialUnaryMessages}) &= \mathsf{unarySelector}^{+}\mathsf{binaryMessage}^{*}\mathsf{keywordMessage?},\\
\rho_{\textsc{Newspeak}}(\mathsf{nontrivialBinaryMessages}) &= \mathsf{binaryMessage}^{+}\mathsf{keywordMessage?},\\
\rho_{\textsc{Newspeak}}(\mathsf{keywordMessages}) &= \mathsf{keywordMessage},\\
\rho_{\textsc{Newspeak}}(\mathsf{nonEmptyMessages}) &= \mathsf{nontrivialUnaryMessages/}\\
&\quad\ \mathsf{nontrivialBinaryMessages/keywordMessages},\\
\rho_{\textsc{Newspeak}}(\mathsf{messageCascade}) &= \mathsf{nonEmptyMessages\ cascadeMessage}^{*},\\
\rho_{\textsc{Newspeak}}(\mathsf{cascadedExpression}) &= \mathsf{primary\ messageCascade?},\\
\rho_{\textsc{Newspeak}}(\mathsf{sendExpression}) &= \mathsf{eventualExpression/implicitKeywordSend/cascadedExpression},\\
\rho_{\textsc{Newspeak}}(\mathsf{expression}) &= \mathsf{setterKeyword?\ sendExpression}.
\end{aligned}
\tag{13.20}
$$

The first selector of a receiver-less unary or keyword expression is retained as an implicit send by the actions below. Binary sends always have an explicit left operand. A bare outer or super is rejected by Equation (13.78); permitting it temporarily in receiverPrimary keeps the precedence grammar non-left-recursive.

The literal, block, pattern, statement, and primary productions are

$$
\begin{aligned}
\rho_{\textsc{Newspeak}}(\mathsf{literal}) &= \mathsf{pattern/number/symbol/string/}\\
&\quad\ \mathsf{tuple/trueToken/falseToken/nilToken},\\
\rho_{\textsc{Newspeak}}(\mathsf{tuple}) &= T_{\{}(\mathsf{expression}(T_{.}\,\mathsf{expression})^{*}T_{.}\,?)?T_{\}},\\
\rho_{\textsc{Newspeak}}(\mathsf{blockParameter}) &= T_{:}\mathsf{slotDecl},\\
\rho_{\textsc{Newspeak}}(\mathsf{blockParameters}) &= \mathsf{blockParameter}^{+}T_{|},\\
\rho_{\textsc{Newspeak}}(\mathsf{block}) &= T_{[}\mathsf{blockParameters?codeBody}T_{]},\\
\rho_{\textsc{Newspeak}}(\mathsf{pattern}) &= T_{<}\mathsf{patternLiteral}T_{>},\\
\rho_{\textsc{Newspeak}}(\mathsf{patternLiteral}) &= \mathsf{wildcardPattern/literalPattern/keywordPattern},\\
\rho_{\textsc{Newspeak}}(\mathsf{wildcardPattern}) &= \mathsf{tok}(_),\\
\rho_{\textsc{Newspeak}}(\mathsf{literalPattern}) &= \mathsf{number/symbol/string/tuple},\\
\rho_{\textsc{Newspeak}}(\mathsf{keywordPattern}) &= \mathsf{keywordPatternPair}^{+},\\
\rho_{\textsc{Newspeak}}(\mathsf{keywordPatternPair}) &= \mathsf{keyword\ keywordPatternValue?},\\
\rho_{\textsc{Newspeak}}(\mathsf{keywordPatternValue}) &= \mathsf{wildcardPattern/literalPattern/}\\
&\quad\ \mathsf{variablePattern/nestedPattern},\\
\rho_{\textsc{Newspeak}}(\mathsf{variablePattern}) &= \mathsf{trivlit}(?)\mathsf{rawId},\\
\rho_{\textsc{Newspeak}}(\mathsf{nestedPattern}) &= \mathsf{trivpattern},\\
\rho_{\textsc{Newspeak}}(\mathsf{parenthesized}) &= T_{(}\mathsf{expression}T_{)},\\
\rho_{\textsc{Newspeak}}(\mathsf{primary}) &= \mathsf{literal/block/classDeclaration/}\\
&\quad\ \mathsf{objectLiteral/parenthesized/}\\
&\quad\ \mathsf{selfToken/unarySelector},\\
\rho_{\textsc{Newspeak}}(\mathsf{returnStatement}) &= T_{\text{-}}\mathsf{expression}T_{.}\,?,\\
\rho_{\textsc{Newspeak}}(\mathsf{statements}) &= \mathsf{returnStatement/}\\
&\quad\ (\mathsf{expression}(T_{.}\,\mathsf{statements})?)/\epsilon,\\
\rho_{\textsc{Newspeak}}(\mathsf{codeBody}) &= \mathsf{slotDecls?statements}.
\end{aligned}
\tag{13.21}
$$

Finally, class, member, and compilation-unit syntax is

$$
\begin{aligned}
\rho_{\textsc{Newspeak}}(\mathsf{messagePattern}) &= (\mathsf{keywordPatternDecl}/\mathsf{binaryPatternDecl}/\mathsf{unarySelector})\mathsf{returnType}?,\\
\rho_{\textsc{Newspeak}}(\mathsf{keywordPatternDecl}) &= (\mathsf{keywordslotDecl})^{+},\\
\rho_{\textsc{Newspeak}}(\mathsf{binaryPatternDecl}) &= \mathsf{binarySelectorslotDecl},\\
\rho_{\textsc{Newspeak}}(\mathsf{accessModifier}) &= \mathsf{tok}(\texttt{private})/\mathsf{tok}(\texttt{public})/\\
&\quad\ \mathsf{tok}(\texttt{protected}),\\
\rho_{\textsc{Newspeak}}(\mathsf{methodDecl}) &= \mathsf{accessModifier}?\mathsf{messagePattern}T_{=}T_{\{}\mathsf{codeBody}T_{\}},\\
\rho_{\textsc{Newspeak}}(\mathsf{slotDecl}) &= \mathsf{idtype}?,\\
\rho_{\textsc{Newspeak}}(\mathsf{slotDef}) &= \mathsf{accessModifier}?\mathsf{slotDecl}(((T_{=}/T_{::=})\mathsf{expression}T_{.})/T_{.})?,\\
\rho_{\textsc{Newspeak}}(\mathsf{sequentialSlots}) &= T_{|}\,\mathsf{slotDef}^{*}T_{|},\\
\rho_{\textsc{Newspeak}}(\mathsf{simultaneousSlots}) &= T_{||}\,\mathsf{slotDef}^{*}T_{||},\\
\rho_{\textsc{Newspeak}}(\mathsf{slotDecls}) &= \mathsf{sequentialSlots}/\mathsf{simultaneousSlots},\\
\rho_{\textsc{Newspeak}}(\mathsf{lazySlotDecl}) &= ((\mathsf{lazyModifier}\mathsf{accessModifier}?)/\\
&\quad\ (\mathsf{accessModifier}\mathsf{lazyModifier}))\mathsf{slotDecl}(T_{=}/T_{::=})\\
&\quad\ \mathsf{expression}T_{.},\\
\rho_{\textsc{Newspeak}}(\mathsf{classHeader}) &= T_{\{}\mathsf{comment}?\mathsf{slotDecls}?\mathsf{statements}T_{\}},\\
\rho_{\textsc{Newspeak}}(\mathsf{nestedClassDecl}) &= \mathsf{accessModifier}?\mathsf{classDeclaration},\\
\rho_{\textsc{Newspeak}}(\mathsf{instanceSide}) &= T_{\{}\\
&\quad\ (\mathsf{nestedClassDecl}/\mathsf{lazySlotDecl}/\mathsf{methodDecl})^{*}T_{\}},\\
\rho_{\textsc{Newspeak}}(\mathsf{classSide}) &= T_{:}T_{\{}\mathsf{methodDecl}^{*}T_{\}},\\
\rho_{\textsc{Newspeak}}(\mathsf{classBody}) &= \mathsf{classHeader}\mathsf{instanceSide}\mathsf{classSide}?,\\
\rho_{\textsc{Newspeak}}(\mathsf{outerReceiver}) &= \mathsf{outerTokenid},\\
\rho_{\textsc{Newspeak}}(\mathsf{inheritancePrefix}) &= \mathsf{outerReceiver}/\mathsf{selfToken}/\mathsf{superToken},\\
\rho_{\textsc{Newspeak}}(\mathsf{inheritanceClause}) &= \mathsf{inheritancePrefix}?\mathsf{unarySelector}\mathsf{message}?,\\
\rho_{\textsc{Newspeak}}(\mathsf{mixinSuffix}) &= (T_{<:}\mathsf{inheritanceClause})^{+}(T_{.}/\mathsf{classBody}),\\
\rho_{\textsc{Newspeak}}(\mathsf{inheritanceAndBody}) &= \mathsf{classBody}/\\
&\quad\ (\mathsf{inheritanceClause}(\mathsf{classBody}/\mathsf{mixinSuffix})),\\
\rho_{\textsc{Newspeak}}(\mathsf{classDeclaration}) &= \mathsf{classTokenidtypeParameterDecl}?\\
&\quad\ \mathsf{messagePattern}?T_{=}\mathsf{inheritanceAndBody},\\
\rho_{\textsc{Newspeak}}(\mathsf{objectLiteral}) &= (\mathsf{idkeywordMessage}?)?\mathsf{classBody},\\
\rho_{\textsc{Newspeak}}(\mathsf{compilationUnit}) &= \mathsf{idstring}?\mathsf{classDeclaration}\,\mathsf{triv}.
\end{aligned}
\tag{13.22}
$$

$G_{\textsc{Newspeak}} = \langle \mathsf{compilationUnit}, \rho_{\textsc{Newspeak}} \rangle$ with every syntactic right-hand side in Equations (13.20)–(13.22) wrapped in a same-named capture. Equations (13.11)– (13.22) are the normalized 0.109 grammar. They make explicit four facts left to prose or typography in the printed grammar: Unicode whitespace, reserved-word exclusion, asynchronous sends, and the use of class declarations, object literals, and self as primaries. No unimplemented chain production is included.

13.2 Concrete trees, metadata, and stable AST identities

Let $z = [i, j)$ be the span of a captured node. Message clauses and patterns are not expressions; their complete surface domains are

$$\begin{aligned} c &::= \mathsf{clause}_z(s, \bar{R}), \\ p &::= \mathsf{wild}_z \mid \mathsf{literalPat}_z(R) \mid \mathsf{variablePat}_z(x) \mid \mathsf{nestedPat}_z(p) \mid \mathsf{keywordPat}_z(\langle k_i, p_i? \rangle_{i=1}^{n}). \end{aligned} \tag{13.23}$$

The complete executable surface AST is

$$\begin{aligned} R ::= {} & \mathsf{atom}_z(\omega) \mid \mathsf{identifier}_z(x) \mid \mathsf{self}_z \mid \mathsf{outer}_z(x) \mid \mathsf{super}_z \mid \mathsf{paren}_z(R) \\ & \mid \mathsf{implicit}_z(s, \bar{R}) \mid \mathsf{message}_z(R, s, \bar{R}) \mid \mathsf{eventual}_z(R, s, \bar{R}) \mid \mathsf{cascade}_z(R, \bar{c}_0, \bar{c}) \\ & \mid \mathsf{setter}_z(x, R) \mid \mathsf{tuple}_z(\bar{R}) \mid \mathsf{pattern}_z(p) \mid \mathsf{closure}_z(\bar{d}, \bar{g}, \bar{s}) \\ & \mid \mathsf{object}_z(o_h) \mid \mathsf{classExpr}_z(d) \mid \mathsf{sequence}_z(\bar{s}), \\ s_R ::= {} & \mathsf{exprStmt}_z(R) \mid \mathsf{returnStmt}_z(R). \end{aligned} \tag{13.24}$$

Declarations and class structure are

$$\begin{aligned} d ::= {} & \mathsf{formal}_z(x) \mid \mathsf{slot}_z(\alpha?, x, m, e?) \mid \mathsf{lazySlot}_z(\alpha?, x, m, e) \\ & \mid \mathsf{method}_z(\alpha?, s, \bar{d}, \bar{g}, \bar{s}) \mid \mathsf{nestedClass}_z(\alpha?, d) \mid \mathsf{classDecl}_z(x, s_f, \bar{d}, b), \\ g ::= {} & \mathsf{sequentialGroup}_z(\bar{d}) \mid \mathsf{simultaneousGroup}_z(\bar{d}), \\ h ::= {} & \mathsf{classStructure}_z(g?, \bar{s}, \bar{d}, \bar{d}_c), \\ o_h ::= {} & \mathsf{implicitObjectHeader}_z(h) \mid \mathsf{explicitObjectHeader}_z(R, c, h), \\ b ::= {} & \mathsf{defaultInheritance}_z(h) \mid \mathsf{explicitInheritance}_z(R, c, h) \mid \mathsf{mixinChain}_z(R, c, \langle c_i \rangle_{i=1}^{n}, h?), \\ U ::= {} & \mathsf{unit}_z(\lambda, \gamma?, d). \end{aligned} \tag{13.25}$$

Here $m \in \{\mathsf{immutable}, \mathsf{mutable}\}$ records `=` versus `::=`; γ is the optional category string; and absent access remains absent until elaboration supplies the protected default. Every declaration and expression constructor carries its captured source span and, after Equation (13.43), its stable identity.

The grammar carries a finite semantic-action map

$$\begin{aligned} \mathcal{J}_G : \mathit{CaptureName} \times \mathit{ActionValue}^* \times \mathit{SourceSlice} \rightharpoonup \mathit{ActionResult}, \\ a ::= \mathsf{emit}(v) \mid \mathsf{erase} \mid \mathsf{metadata}(m). \end{aligned} \tag{13.26}$$

Projection returns an ordered sequence because punctuation contributes no semantic child. Metadata likewise leaves the semantic-child sequence but is accumulated separately. Put

$$\begin{aligned} \mathsf{project}_{G,u}(\mathsf{leaf}\langle i, j \rangle) &= \langle \langle \mathsf{token}(u[i:j], [i, j)) \rangle, \epsilon \rangle, \\ \bar{v} &= \bar{v}_1 \cdots \bar{v}_r, & \bar{m} &= \bar{m}_1 \cdots \bar{m}_r, \\ \mathsf{project}_{G,u}(t_k) &= \langle \bar{v}_k, \bar{m}_k \rangle \quad (1 \le k \le r), \\ \mathsf{project}_{G,u}(\mathsf{node}\langle n, i, j, \bar{t} \rangle) &= \mathsf{emitResult}(\mathcal{J}_G(n, \bar{v}, u[i:j]), \bar{m}), \\ \mathsf{emitResult}(\mathsf{emit}(v), \bar{m}) &= \langle \langle v \rangle, \bar{m} \rangle, \\ \mathsf{emitResult}(\mathsf{erase}, \bar{m}) &= \langle \epsilon, \bar{m} \rangle, \\ \mathsf{emitResult}(\mathsf{metadata}(m), \bar{m}) &= \langle \epsilon, \bar{m} \cdot \langle m \rangle \rangle, \\ \mathsf{ast}_{G,u}(t) = \langle R, \bar{m} \rangle \quad &\iff \quad \mathsf{project}_{G,u}(t) = \langle \langle R \rangle, \bar{m} \rangle. \end{aligned} \tag{13.27}$$

The metadata sequence $\bar{m}$ is attached by Equation (13.41); it is not an executable AST child. Below, an equation $\mathcal{J}_G(\cdots) = v$ abbreviates $\mathcal{J}_G(\cdots) = \mathsf{emit}(v)$. To define $\mathcal{J}_{G_{\textsc{Newspeak}}}$ without a production-sized prose gap, first define the message folds. If $c_i = \mathsf{clause}(s_i, \bar{R}_i)$, then

$$
\begin{aligned}
\mathsf{asClause}_z(\mathsf{selector}(s)) &= \mathsf{clause}_z(s, \epsilon),\\
\mathsf{asClause}_z(c) &= c \quad (c = \mathsf{clause}_{z'}(s, \bar{R})),\\
\mathsf{flattenClauses}_z(\epsilon) &= \epsilon,\\
\mathsf{flattenClauses}_z(v \cdot \bar{v}) &= \mathsf{asClause}_z(v) \cdot \mathsf{flattenClauses}_z(\bar{v}) \quad (v \notin \mathit{Sequence}),\\
\mathsf{flattenClauses}_z(\bar{c} \cdot \bar{v}) &= \bar{c} \cdot \mathsf{flattenClauses}_z(\bar{v}) \quad (\bar{c} \in \mathit{Clause}^*),\\
\mathsf{foldExplicit}_z(R, \epsilon) &= R,\\
\mathsf{foldExplicit}_z(R, v_1 \cdot \bar{v}) &= \mathsf{foldExplicit}_z(\mathsf{message}_z(R, s_1, \bar{R}_1), \bar{v}),\\
&\qquad \mathsf{asClause}_z(v_1) = \mathsf{clause}(s_1, \bar{R}_1),\\
\mathsf{foldImplicit}_z(v_1 \cdot \bar{v}) &= \mathsf{foldExplicit}_z(\mathsf{implicit}_z(s_1, \bar{R}_1), \bar{v}),\\
&\qquad \mathsf{asClause}_z(v_1) = \mathsf{clause}(s_1, \bar{R}_1),\\
\mathsf{makeCascade}_z(R, \bar{c}_0, \epsilon) &= \mathsf{foldExplicit}_z(R, \bar{c}_0),\\
\mathsf{makeCascade}_z(R, \bar{c}_0, c_1 \cdot \bar{c}) &= \mathsf{cascade}_z(R, \bar{c}_0, c_1 \cdot \bar{c}).
\end{aligned}
\tag{13.28}
$$

The first two equations preserve unary-before-binary-before-keyword order already imposed by Equation (13.20); the third retains one receiver and every cascade clause.

Lexical actions are

$$
\begin{aligned}
\mathcal{J}_{\textsc{Newspeak}}(\mathsf{id}, \bar{v}, w) &= \mathsf{name}(\mathsf{decodeId}(w)),\\
\mathcal{J}_{\textsc{Newspeak}}(\mathsf{unarySelector}, \langle \mathsf{name}(x) \rangle, w) &= \mathsf{selector}_z(\#x),\\
\mathcal{J}_{\textsc{Newspeak}}(\mathsf{binarySelector}, \bar{v}, w) &= \mathsf{selector}_z(\mathsf{decodeBinary}(w)),\\
\mathcal{J}_{\textsc{Newspeak}}(\mathsf{keyword}, \bar{v}, w) &= \mathsf{keyword}_z(\mathsf{decodeKeyword}(w)),\\
\mathcal{J}_{\textsc{Newspeak}}(\mathsf{setterKeyword}, \bar{v}, w) &= \mathsf{name}(\mathsf{decodeSetterName}(w)),\\
\mathcal{J}_{\textsc{Newspeak}}(\mathsf{selfToken}, \bar{v}, w) &= \mathsf{self}_z,\\
\mathcal{J}_{\textsc{Newspeak}}(\mathsf{superToken}, \bar{v}, w) &= \mathsf{super}_z,\\
\mathcal{J}_{\textsc{Newspeak}}(\mathsf{outerReceiver}, \langle \mathsf{name}(x) \rangle, w) &= \mathsf{outer}_z(x),\\
\mathcal{J}_{\textsc{Newspeak}}(\mathsf{comment}, \bar{v}, w) &= \mathsf{metadata}(\mathsf{metadataToken}_z(\mathsf{tag}(\bar{v}), \mathsf{payload}(w))),\\
\mathcal{J}_{\textsc{Newspeak}}(\mathsf{number}, \bar{v}, w) &= \mathsf{atom}_z(\mathsf{decodeNumber}(w)),\\
\mathcal{J}_{\textsc{Newspeak}}(\mathsf{string}, \bar{v}, w) &= \mathsf{atom}_z(\mathsf{NFC}(\mathsf{decodeString}(w))),\\
\mathcal{J}_{\textsc{Newspeak}}(\mathsf{symbol}, \bar{v}, w) &= \mathsf{atom}_z(\mathsf{decodeSymbol}(w)),\\
\mathcal{J}_{\textsc{Newspeak}}(\mathsf{trueToken}, \bar{v}, w) &= \mathsf{atom}_z(\mathsf{true}),\\
\mathcal{J}_{\textsc{Newspeak}}(\mathsf{falseToken}, \bar{v}, w) &= \mathsf{atom}_z(\mathsf{false}),\\
\mathcal{J}_{\textsc{Newspeak}}(\mathsf{nilToken}, \bar{v}, w) &= \mathsf{atom}_z(\mathsf{nil}).
\end{aligned}
\tag{13.29}
$$

$\mathsf{decodeNumber}$ is the positional interpretation in Specification §5.1.1; it rejects a digit outside the stated radix.

Punctuation and lexical scaffolding are erased only after their enclosing capture has used its

source slice. Pure wrapper productions pass their unique semantic child through:

$$\begin{aligned}
\mathcal{E}_{\textsc{Newspeak}} &= \{T_p \mid p \in \mathcal{P}\} \cup \{\mathsf{white}, \mathsf{reservedWord}, \mathsf{idRest}, \mathsf{rawId},\\
&\qquad \mathsf{rawKeyword}, \mathsf{keywords}, \mathsf{rawBinary}, \mathsf{metadataTag}, \mathsf{commentItem},\\
&\qquad \mathsf{classToken}, \mathsf{lazyModifier}, \mathsf{outerToken}\},\\
\mathcal{I}_{\textsc{Newspeak}} &= \{\mathsf{literal}, \mathsf{receiverPrimary}, \mathsf{sendExpression},\\
&\qquad \mathsf{slotDecls}, \mathsf{blockParameter}, \mathsf{inheritancePrefix}, \mathsf{patternLiteral},\\
&\qquad \mathsf{keywordPatternValue}\},\\
\mathcal{J}_{\textsc{Newspeak}}(n, \bar{v}, w) &= \mathsf{erase} \quad (n \in \mathcal{E}_{\textsc{Newspeak}}),\\
\mathcal{J}_{\textsc{Newspeak}}(n, \langle v \rangle, w) &= v \quad (n \in \mathcal{I}_{\textsc{Newspeak}}).
\end{aligned} \tag{13.30}$$

Sequence normalization used by the expression actions is

$$\begin{aligned}
\mathsf{orEmpty}(\mathsf{none}) &= \epsilon,\\
\mathsf{orEmpty}(v) &= \langle v \rangle \quad (v \neq \mathsf{none}),\\
\mathsf{orEmptySeq}(\mathsf{none}) &= \epsilon,\\
\mathsf{orEmptySeq}(\bar{v}) &= \bar{v} \quad (\bar{v} \neq \mathsf{none}),\\
\mathsf{asStatement}(R) &= \mathsf{exprStmt}(R) \quad (R \in \mathit{SurfaceExpression}),\\
\mathsf{asStatement}(s_R) &= s_R \quad (s_R \in \mathit{SurfaceStatement}),\\
\mathsf{flattenStatements}(\epsilon) &= \epsilon,\\
\mathsf{flattenStatements}(v \cdot \bar{v}) &= \mathsf{asStatement}(v) \cdot \mathsf{flattenStatements}(\bar{v}) \quad (v \notin \mathit{Sequence}),\\
\mathsf{flattenStatements}(\mathsf{sequence}(\bar{s}) \cdot \bar{v}) &= \bar{s} \cdot \mathsf{flattenStatements}(\bar{v}),
\end{aligned} \tag{13.31}$$

The message and expression actions are exhaustive:

$$\begin{aligned}
\mathcal{J}_{\textsc{Newspeak}}(\mathsf{binaryMessage}, \langle \mathsf{selector}(s), R \rangle, w) &= \mathsf{clause}_z(s, \langle R \rangle),\\
\mathcal{J}_{\textsc{Newspeak}}(\mathsf{keywordMessage}, \langle \mathsf{keyword}(k_i), R_i \rangle_{i=1}^{n}, w) &= \mathsf{clause}_z(k_1 \cdots k_n, \langle R_i \rangle_{i=1}^{n}),\\
\mathcal{J}_{\textsc{Newspeak}}(\mathsf{message}, \langle \mathsf{selector}(s) \rangle, w) &= \mathsf{clause}_z(s, \epsilon),\\
\mathcal{J}_{\textsc{Newspeak}}(\mathsf{message}, \langle c \rangle, w) &= c \quad (c = \mathsf{clause}_{z'}(s, \bar{R})),\\
\mathcal{J}_{\textsc{Newspeak}}(\mathsf{cascadeMessage}, \langle \mathsf{selector}(s) \rangle, w) &= \mathsf{clause}_z(s, \epsilon),\\
\mathcal{J}_{\textsc{Newspeak}}(\mathsf{cascadeMessage}, \langle c \rangle, w) &= c,
\end{aligned} \tag{13.32}$$

The productions that normalize to an ordered sequence of message clauses are

$$\mathcal{N}_M = \{\mathsf{nontrivialUnaryMessages}, \mathsf{nontrivialBinaryMessages}, \mathsf{keywordMessages}, \mathsf{nonEmptyMessages}\}. \tag{13.33}$$

$$
\begin{aligned}
\mathcal{J}_{\textsc{Newspeak}}(\mathsf{primary}, \langle \mathsf{selector}_{z'}(\#x)\rangle, w) &= \mathsf{identifier}_z(x),\\
\mathcal{J}_{\textsc{Newspeak}}(\mathsf{primary}, \langle \mathsf{classDecl}_{z'}(\bar{v})\rangle, w) &= \mathsf{classExpr}_z(\mathsf{classDecl}_{z'}(\bar{v})),\\
\mathcal{J}_{\textsc{Newspeak}}(\mathsf{primary}, \langle R\rangle, w) &= R \quad \text{for every other primary},\\
\mathcal{J}_{\textsc{Newspeak}}(\mathsf{unaryExpression}, \langle R, \bar{c}\rangle, w) &= \mathsf{foldExplicit}_z(R, \bar{c}),\\
\mathcal{J}_{\textsc{Newspeak}}(\mathsf{binaryExpression}, \langle R, \bar{c}\rangle, w) &= \mathsf{foldExplicit}_z(R, \bar{c}),\\
\mathcal{J}_{\textsc{Newspeak}}(\mathsf{keywordExpression}, \langle R, c?\rangle, w) &= \mathsf{foldExplicit}_z(R, c?),\\
\mathcal{J}_{\textsc{Newspeak}}(\mathsf{eventualExpression}, \langle R, c\rangle, w) &= \mathsf{eventual}_z(R, \mathrm{selector}(c), \mathsf{arguments}(c)),\\
\mathcal{J}_{\textsc{Newspeak}}(\mathsf{implicitKeywordSend}, \langle c\rangle, w) &= \mathsf{implicit}_z(\mathrm{selector}(c), \mathsf{arguments}(c)),\\
\mathcal{J}_{\textsc{Newspeak}}(n, \bar{v}, w) &= \mathsf{flattenClauses}_z(\bar{v}) \quad (n \in \mathcal{N}_M),\\
\mathcal{J}_{\textsc{Newspeak}}(\mathsf{messageCascade}, \langle \bar{c}_0, \bar{c}\rangle, w) &= \mathsf{cascadeParts}_z(\bar{c}_0, \bar{c}),\\
\mathcal{J}_{\textsc{Newspeak}}(\mathsf{cascadedExpression}, \langle R\rangle, w) &= R,\\
\mathcal{J}_{\textsc{Newspeak}}(\mathsf{cascadedExpression}, \langle R, \mathsf{cascadeParts}(\bar{c}_0, \bar{c})\rangle, w) &= \mathsf{makeCascade}_z(R, \bar{c}_0, \bar{c}),
\end{aligned}
\tag{13.34}
$$

$$
\begin{aligned}
\mathcal{J}_{\textsc{Newspeak}}(\mathsf{expression}, \langle \mathsf{name}(x), R\rangle, w) &= \mathsf{setter}_z(x, R),\\
\mathcal{J}_{\textsc{Newspeak}}(\mathsf{expression}, \langle R\rangle, w) &= R,\\
\mathcal{J}_{\textsc{Newspeak}}(\mathsf{parenthesized}, \langle R\rangle, w) &= \mathsf{paren}_z(R),\\
\mathcal{J}_{\textsc{Newspeak}}(\mathsf{tuple}, \bar{R}, w) &= \mathsf{tuple}_z(\bar{R}),\\
\mathcal{J}_{\textsc{Newspeak}}(\mathsf{blockParameters}, \bar{d}, w) &= \bar{d},\\
\mathcal{J}_{\textsc{Newspeak}}(\mathsf{codeBody}, \langle g?, \mathsf{sequence}(\bar{s})\rangle, w) &= \mathsf{body}_z(g?, \bar{s}),\\
\mathcal{J}_{\textsc{Newspeak}}(\mathsf{block}, \langle \bar{d}?, \mathsf{body}(g?, \bar{s})\rangle, w) &= \mathsf{closure}_z(\mathsf{orEmptySeq}(\bar{d}?), \mathsf{orEmpty}(g?), \bar{s}),\\
\mathcal{J}_{\textsc{Newspeak}}(\mathsf{returnStatement}, \langle R\rangle, w) &= \mathsf{returnStmt}_z(R),\\
\mathcal{J}_{\textsc{Newspeak}}(\mathsf{statements}, \bar{v}, w) &= \mathsf{sequence}_z(\mathsf{flattenStatements}(\bar{v})).
\end{aligned}
\tag{13.35}
$$

A unary selector in primary position becomes $\mathsf{identifier}_z(x)$; Equation (13.53) later makes it an implicit nullary send. An implicit keyword message instead takes the explicit implicitKeywordSend branch. Thus neither form introduces a variable-read node.

Pattern actions are

$$
\begin{aligned}
\mathcal{J}_{\textsc{Newspeak}}(\mathsf{wildcardPattern}, \epsilon, w) &= \mathsf{wild}_z,\\
\mathcal{J}_{\textsc{Newspeak}}(\mathsf{literalPattern}, \langle R\rangle, w) &= \mathsf{literalPat}_z(R),\\
\mathcal{J}_{\textsc{Newspeak}}(\mathsf{variablePattern}, \langle \mathsf{name}(x)\rangle, w) &= \mathsf{variablePat}_z(x),\\
\mathcal{J}_{\textsc{Newspeak}}(\mathsf{nestedPattern}, \langle p\rangle, w) &= \mathsf{nestedPat}_z(p),\\
\mathcal{J}_{\textsc{Newspeak}}(\mathsf{keywordPatternPair}, \langle \mathsf{keyword}(k), p?\rangle, w) &= \langle k, p?\rangle,\\
\mathcal{J}_{\textsc{Newspeak}}(\mathsf{keywordPattern}, \bar{v}, w) &= \mathsf{keywordPat}_z(\bar{v}), \quad \bar{v} = \langle k_i, p_i?\rangle_{i=1}^{n},\\
\mathcal{J}_{\textsc{Newspeak}}(\mathsf{pattern}, \langle p\rangle, w) &= \mathsf{pattern}_z(p).
\end{aligned}
\tag{13.36}
$$

For an inheritance clause define

$$\begin{aligned}
\mathsf{inheritanceReceiver}_z(\mathsf{none}, \mathsf{selector}(s)) &= \mathsf{implicit}_z(s, \epsilon), \\
\mathsf{inheritanceReceiver}_z(R, \mathsf{selector}(s)) &= \mathsf{message}_z(R, s, \epsilon), \\
\mathsf{initializerClause}_z(\mathsf{none}) &= \mathsf{clause}_z(\#\mathsf{new}, \epsilon), \\
\mathsf{initializerClause}_z(c) &= c, \\
R_I &= \mathsf{inheritanceReceiver}_z(R?, \mathsf{selector}(s)), \\
c_I &= \mathsf{initializerClause}_z(c?), \\
\mathcal{J}_{\textsc{Newspeak}}(\mathsf{inheritanceClause}, \langle R?, \mathsf{selector}(s), c? \rangle, w) &= \mathsf{inheritanceHead}_z(R_I, c_I).
\end{aligned} \tag{13.37}$$

For compactness in the remaining action equations, define the non-overloaded abbreviation

$$\mathsf{act}_{[i,j)}(n, \bar{v}, u[i:j]) = \mathcal{J}_{\textsc{Newspeak}}(n, \bar{v}, u[i:j]). \tag{13.38}$$

The complete declaration actions are

$$
\begin{aligned}
N_I &= \mathsf{inheritanceAndBody},\\
\mathsf{sel}(\mathsf{patDecl}(s,\bar{d})) &= s,\\
\mathsf{formals}(\mathsf{patDecl}(s,\bar{d})) &= \bar{d},\\
\mathsf{factorySel}(\mathsf{none}) &= \#\mathsf{new},\\
\mathsf{factorySel}(\mathsf{patDecl}(s,\bar{d})) &= s,\\
\mathsf{factoryFormals}(\mathsf{none}) &= \epsilon,\\
\mathsf{factoryFormals}(\mathsf{patDecl}(s,\bar{d})) &= \bar{d},\\
\mathsf{act}_z(\mathsf{keywordPatternDecl}, \langle \mathsf{keyword}(k_i), d_i\rangle_{i=1}^n, w) &=\\
&\quad \mathsf{patDecl}_z(k_1 \cdots k_n, \langle d_i\rangle_{i=1}^n),\\
\mathsf{act}_z(\mathsf{binaryPatternDecl}, \langle \mathsf{selector}(s), d\rangle, w) &= \mathsf{patDecl}_z(s, \langle d\rangle),\\
\mathsf{act}_z(\mathsf{messagePattern}, \langle \mathsf{selector}(s)\rangle, w) &= \mathsf{patDecl}_z(s, \epsilon),\\
\mathsf{act}_z(\mathsf{messagePattern}, \langle p\rangle, w) &= p \quad (p = \mathsf{patDecl}_{z'}(s,\bar{d})),\\
\mathsf{act}_z(\mathsf{accessModifier}, \bar{v}, w) &= \mathsf{decodeAccess}(w),\\
\mathsf{act}_z(\mathsf{slotDecl}, \langle \mathsf{name}(x)\rangle, w) &= \mathsf{formal}_z(x),\\
\mathsf{act}_z(\mathsf{slotDef}, \langle \alpha?, d, R?\rangle, w) &=\\
&\quad \mathsf{slot}_z(\alpha?, \mathsf{name}(d), \mathsf{mutability}(w), R?),\\
\mathsf{act}_z(\mathsf{sequentialSlots}, \bar{d}, w) &= \mathsf{sequentialGroup}_z(\bar{d}),\\
\mathsf{act}_z(\mathsf{simultaneousSlots}, \bar{d}, w) &= \mathsf{simultaneousGroup}_z(\bar{d}),\\
\mathsf{act}_z(\mathsf{lazySlotDecl}, \langle \alpha?, d, R\rangle, w) &=\\
&\quad \mathsf{lazySlot}_z(\alpha?, \mathsf{name}(d), \mathsf{mutability}(w), R),\\
\mathsf{act}_z(\mathsf{methodDecl}, \langle \alpha?, p, \mathsf{body}(g?, \bar{s})\rangle, w) &=\\
&\quad \mathsf{method}_z(\alpha?, \mathsf{sel}(p), \mathsf{formals}(p), \mathsf{orEmpty}(g?), \bar{s}),\\
\mathsf{act}_z(\mathsf{nestedClassDecl}, \langle \alpha?, d\rangle, w) &= \mathsf{nestedClass}_z(\alpha?, d),\\
\mathsf{act}_z(\mathsf{classHeader}, \langle g?, \mathsf{sequence}(\bar{s})\rangle, w) &= \mathsf{header}_z(g?, \bar{s}),\\
\mathsf{act}_z(\mathsf{instanceSide}, \bar{d}, w) &= \bar{d},\\
\mathsf{act}_z(\mathsf{classSide}, \bar{d}_c, w) &= \bar{d}_c,\\
\mathsf{act}_z(\mathsf{classBody}, \langle \mathsf{header}(g?, \bar{s}), \bar{d}, \bar{d}_c?\rangle, w) &=\\
&\quad \mathsf{classStructure}_z(g?, \bar{s}, \bar{d}, \mathsf{orEmptySeq}(\bar{d}_c?)),\\
\mathsf{act}_z(\mathsf{mixinSuffix}, \langle \bar{h}, h?\rangle, w) &= \mathsf{mixinTail}_z(\bar{h}, h?),\\
\mathsf{act}_z(N_I, \langle h\rangle, w) &= \mathsf{defaultInheritance}_z(h),\\
\mathsf{act}_z(N_I, \langle \mathsf{inheritanceHead}(R, c), h\rangle, w) &= \mathsf{explicitInheritance}_z(R, c, h),\\
h_I &= \mathsf{inheritanceHead}(R, c),\\
t_I &= \mathsf{mixinTail}(\bar{c}, h?),\\
\mathsf{act}_z(N_I, \langle h_I, t_I\rangle, w) &=\\
&\quad \mathsf{mixinChain}_z(R, c, \bar{c}, h?),\\
\mathsf{act}_z(\mathsf{classDeclaration}, \langle \mathsf{name}(x), p?, b\rangle, w) &= \mathsf{classDecl}_z(x, \mathsf{factorySel}(p?), \mathsf{factoryFormals}(p?), b),\\
\mathsf{act}_z(\mathsf{objectLiteral}, \langle h\rangle, w) &= \mathsf{object}_z(\mathsf{implicitObjectHeader}_z(h)),\\
R_o &= \mathsf{identifier}_z(x),\\
c_o &= \mathsf{initializerClause}_z(c?),\\
o_h &= \mathsf{explicitObjectHeader}_z(R_o, c_o, h),\\
\mathsf{act}_z(\mathsf{objectLiteral}, \langle \mathsf{name}(x), c?, h\rangle, w) &=\\
&\quad \mathsf{object}_z(o_h),\\
\mathsf{act}_z(\mathsf{compilationUnit}, \langle \lambda, \gamma?, d\rangle, w) &=\\
&\quad \mathsf{unit}_z(\lambda, \gamma?, d).
\end{aligned}
$$

$\mathsf{mutability}(w)$ is immutable exactly when the slot definition's source slice w contains `=` rather than `::=`; either assignment operator makes a lazy slot mutable or immutable in the same way, and omission is mutable. The inheritance actions construct exactly the three b alternatives in Equation (13.25), in source order. All punctuation, trivia, and wrapper productions have either the discard action or the unique-child action. Equations (13.29)–(13.39), together with those two structural actions, define $\mathcal{J}_{G_{\text{NEWSPEAK}}}$ on every successful captured production; there is no residual action case.

A metadata token is $m = \langle \tau, w, [i, j) \rangle$, where τ is its tag and w its uninterpreted payload; an ordinary comment has tag $\mathsf{comment}$. Let $\mathsf{triviaOnly}_u(i, j)$ mean that $u[i : j]$ contains only whitespace and comments, and define

$$\mathsf{triviaBetween}_u(X, Y) \iff \mathsf{end}(X) \leq \mathsf{start}(Y) \wedge \mathsf{triviaOnly}_u(\mathsf{end}(X), \mathsf{start}(Y)). \tag{13.40}$$

The association relation is

$$\begin{aligned}
m \ \mathsf{before}_u \ R &\iff \mathsf{triviaBetween}_u(m, R) \wedge \\
&\quad R \in \mathit{Expression} \uplus \mathit{FormalDeclaration} \uplus \mathit{SlotDeclaration}, \\
m \ \mathsf{afterPattern}_u \ R &\iff \mathsf{triviaBetween}_u(\mathsf{pattern}(R), m) \wedge \\
&\quad R \in \mathit{MethodDeclaration} \uplus \mathit{FactoryDeclaration}, \\
m \ \mathsf{afterHeader}_u \ R &\iff \mathsf{triviaBetween}_u(\mathsf{header}(R), m) \wedge R \in \mathit{ClassDeclaration}, \\
m \ \mathsf{staticTypeOf}_u \ R &\iff \mathsf{tag}(m) = \mathsf{staticType} \wedge \mathsf{span}(m) \subseteq \mathsf{span}(R) \wedge \\
&\quad R = \min_{\subseteq} \{R' \mid \mathsf{span}(m) \subseteq \mathsf{span}(R') \wedge R' \in \mathit{TypedDeclaration}\}.
\end{aligned} \tag{13.41}$$

The applicable relation is selected by node kind in displayed order. Metadata attachment and its insertion into the tree are the functions

$$\begin{aligned}
\mathsf{related}_u(m, R) &= \begin{cases}
m \ \mathsf{staticTypeOf}_u \ R & \mathsf{tag}(m) = \mathsf{staticType}, \\
m \ \mathsf{afterHeader}_u \ R & R \in \mathit{ClassDeclaration}, \\
m \ \mathsf{afterPattern}_u \ R & R \in \mathit{MethodDeclaration} \uplus \mathit{FactoryDeclaration}, \\
m \ \mathsf{before}_u \ R & \text{otherwise},
\end{cases} \\
\mathsf{metadata}_{u,\bar{m}}(R) &= \mathsf{sourceOrder}(\langle m \in \bar{m} \mid \mathsf{related}_u(m, R) \rangle), \\
\mathsf{attachMetadata}_u(R, \bar{m}) &= R[\mathsf{meta}(n) \mapsto \mathsf{metadata}_{u,\bar{m}}(n) \mid n \in \mathsf{nodes}(R)].
\end{aligned} \tag{13.42}$$

Punctuation, whitespace, and metadata tokens therefore do not become executable AST children.

Parsed nodes next receive identities. Let $\kappa : \mathit{NodeKey} \rightharpoonup (\mathit{DeclId} \uplus \mathit{SiteId})$ be an optional identity-retention map, where a key contains a source-unit identity, structural path, constructor kind, and declared name when one exists. Define

$$\mathsf{identify}_\kappa(R) = \langle S, \kappa' \rangle \iff \begin{cases}
\mathsf{id}_S(n) = \kappa(\mathsf{key}_R(n)) & \mathsf{key}_R(n) \in \mathrm{dom}(\kappa), \\
\mathsf{id}_S(n) \text{ is fresh of the required sort} & \mathsf{key}_R(n) \notin \mathrm{dom}(\kappa),
\end{cases} \tag{13.43}$$

for every declaration or expression node n, with κ' extending κ by the chosen keys. The result must satisfy

$$\mathsf{idsOK}(S) \iff \mathsf{id}_S \text{ is injective } \wedge \ \mathsf{id}_S(n) \in \begin{cases}
\mathit{DeclId} & n \in \mathit{DeclarationNode}(S), \\
\mathit{SiteId} & n \in \mathit{ExpressionNode}(S).
\end{cases} \tag{13.44}$$

An initial compilation uses $\kappa = \varnothing$. A source-preserving editor may supply a retained map; mirror changes that directly edit identified AST nodes preserve identities according to Equation (11.37) and do not reparse text.

13.3 Binding, elaboration, and static well-formedness

An elaboration context is

$$\Gamma = \langle K, q, D_0, \bar{B}, r_t, h_t \rangle, \tag{13.45}$$

where K is the enclosing class-body declaration or top, q is the enclosing activation declaration or none, D_0 is the immediately enclosing named class or none, $\bar{B}$ is the innermost-first sequence of lexical binder maps, and r_t, h_t record whether local and non-local return are permitted. Lexical binding is the partial function

$$\mathsf{bind}_\Gamma(x) = d \iff j = \min\{i \mid x \in \mathrm{dom}(B_i)\} \wedge d = B_j(x), \qquad \bar{B} = \langle B_0, \ldots, B_m \rangle. \tag{13.46}$$

An absent set on the right makes the function undefined. Class-name binding uses the same definition restricted to class declarations; write $\mathsf{classBind}_\Gamma(x) = D_t$. The total annotation used by an ordinary implicit send is

$$\mathsf{bindingAnnotation}_\Gamma(x) = \begin{cases} \mathsf{bind}_\Gamma(x) & \mathsf{bind}_\Gamma(x) \neq \uparrow, \\ \mathsf{none} & \mathsf{bind}_\Gamma(x) = \uparrow. \end{cases} \tag{13.47}$$

Parentheses do not erase receiver authority. Define

$$\begin{aligned} \mathsf{receiverKind}_\Gamma((R)) &= \mathsf{receiverKind}_\Gamma(R), \\ \mathsf{receiverKind}_\Gamma(\mathsf{self}) &= \mathsf{selfReceiver}(D_0), \\ \mathsf{receiverKind}_\Gamma(\mathsf{outer}(x)) &= \mathsf{outerReceiver}(\mathsf{classBind}_\Gamma(x), D_0), \\ \mathsf{receiverKind}_\Gamma(\mathsf{super}) &= \mathsf{superReceiver}, \\ \mathsf{receiverKind}_\Gamma(R) &= \mathsf{ordinaryReceiver} \quad \text{otherwise.} \end{aligned} \tag{13.48}$$

For already elaborated arguments $\bar{e}$, put $r_k = \mathsf{receiverKind}_\Gamma(R)$. Send construction is the distinct partial function

$$\mathsf{makeSend}_\Gamma(R, s, \bar{e}) = \begin{cases} \mathsf{selfSend}(s, \bar{e}, D_0) & r_k = \mathsf{selfReceiver}(D_0), \\ \mathsf{outerSend}(s, \bar{e}, D_t, D_0) & r_k = \mathsf{outerReceiver}(D_t, D_0), \\ \mathsf{superSend}(s, \bar{e}) & r_k = \mathsf{superReceiver}, \\ \mathsf{send}(e_R, s, \bar{e}) & r_k = \mathsf{ordinaryReceiver} \wedge \mathsf{elabExpr}_\Gamma(R) = e_R. \end{cases} \tag{13.49}$$

It is undefined if $D_0 = \mathsf{none}$ in a self or outer case, if the named outer class does not bind, or if super occurs outside a method activation. Notice again that the super case receives no declaration annotation.

Use distinct functions for a single expression, an expression list, a statement sequence, a declaration, and a complete unit:

$$\begin{aligned} \mathsf{elabExpr}_\Gamma &: \mathit{SurfaceExpression} \rightharpoonup \mathit{CoreExpression}, \\ \mathsf{elabExprList}_\Gamma &: \mathit{SurfaceExpression}^* \rightharpoonup \mathit{CoreExpression}^*, \\ \mathsf{elabStatements}_\Gamma &: \mathit{SurfaceStatement}^* \rightharpoonup \mathit{CoreStatement}^*, \\ \mathsf{elabDecl}_\Gamma &: \mathit{SurfaceDeclaration} \rightharpoonup \mathit{DerivedDeclaration}, \\ \mathsf{deriveProgram} &: \mathit{IdentifiedUnit} \rightharpoonup (\mathit{Program} \times \mathit{CoreExpression}). \end{aligned} \tag{13.50}$$

The list and optional functions, selector annotations, and activation contexts are defined by

$$\begin{aligned}
\mathsf{elabExprList}_\Gamma(\epsilon) &= \epsilon,\\
\mathsf{elabExprList}_\Gamma(R \cdot \bar{R}) &= \mathsf{elabExpr}_\Gamma(R) \cdot \mathsf{elabExprList}_\Gamma(\bar{R}),\\
\mathsf{elabOptionalExpr}_\Gamma(\mathsf{none}) &= \mathsf{none},\\
\mathsf{elabOptionalExpr}_\Gamma(R) &= \mathsf{elabExpr}_\Gamma(R),\\
\mathsf{elabOptionalPattern}_\Gamma(\mathsf{none}) &= _,\\
\mathsf{elabOptionalPattern}_\Gamma(p) &= \mathsf{elabPattern}_\Gamma(p),\\
\mathsf{selectorBindingAnnotation}_\Gamma(s) &= \begin{cases} \mathsf{bindingAnnotation}_\Gamma(x) & s = \#x,\\ \mathsf{none} & \mathsf{arity}(s) > 0. \end{cases}
\end{aligned} \tag{13.51}$$

For a declaration sequence, let

$$\begin{aligned}
\mathsf{groupDecls}(\epsilon) &= \epsilon,\\
\mathsf{groupDecls}(g \cdot \bar{g}) &= \mathsf{members}(g) \cdot \mathsf{groupDecls}(\bar{g}),\\
\mathsf{binder}(\bar{d}) &= \{\mathsf{name}(d) \mapsto \mathsf{id}(d) \mid d \in \bar{d}\},\\
\mathsf{returnFlags}_\Gamma(q) &= \begin{cases} \langle \mathsf{true}, \mathsf{false}\rangle & q \in \mathit{MethodDeclId},\\ \langle \mathsf{false}, \Gamma.r_t \vee \Gamma.h_t\rangle & q \in \mathit{ClosureDeclId}, \end{cases}\\
\mathsf{activationContext}_\Gamma(q, \bar{d}, \bar{g}) &= \langle \Gamma.K, q, \Gamma.D_0, \mathsf{binder}(\bar{d} \cdot \mathsf{groupDecls}(\bar{g})) \cdot \Gamma.\bar{B}, r_q, h_q\rangle,\\
&\quad \langle r_q, h_q\rangle = \mathsf{returnFlags}_\Gamma(q).
\end{aligned} \tag{13.52}$$

Thus $\Gamma_q = \mathsf{activationContext}_\Gamma(q, \bar{d}, \bar{g})$ in every closure or method case below. The atomic and send cases are

$$\begin{aligned}
\mathsf{elabExpr}_\Gamma(\mathsf{identifier}_z(x)) &= \mathsf{implicit}(\#x, \epsilon, \mathsf{bindingAnnotation}_\Gamma(x)),\\
\mathsf{elabExpr}_\Gamma(\mathsf{implicit}_z(s, \bar{R})) &= \mathsf{implicit}(s, \mathsf{elabExprList}_\Gamma(\bar{R}), \mathsf{selectorBindingAnnotation}_\Gamma(s)),\\
\mathsf{elabExpr}_\Gamma(\mathsf{message}_z(R, s, \bar{R}')) &= \mathsf{makeSend}_\Gamma(R, s, \mathsf{elabExprList}_\Gamma(\bar{R}')),\\
\mathsf{elabExpr}_\Gamma(\mathsf{eventual}_z(R, s, \bar{R}')) &= \mathsf{asyncSend}(\mathsf{elabExpr}_\Gamma(R), s, \mathsf{elabExprList}_\Gamma(\bar{R}')),\\
\mathsf{elabExpr}_\Gamma(\mathsf{atom}_z(\omega)) &= \mathsf{atom}(\omega),\\
\mathsf{elabExpr}_\Gamma(\mathsf{self}_z) &= \mathsf{self},\\
\mathsf{elabExpr}_\Gamma(\mathsf{paren}_z(R)) &= \mathsf{elabExpr}_\Gamma(R),\\
\mathsf{elabExpr}_\Gamma(\mathsf{tuple}_z(\bar{R})) &= \mathsf{tuple}(\mathsf{elabExprList}_\Gamma(\bar{R})),\\
\mathsf{elabExpr}_\Gamma(\mathsf{pattern}_z(p)) &= \mathsf{pattern}(\mathsf{elabPattern}_\Gamma(p), \mathsf{bindingAnnotation}_\Gamma(\mathsf{Pattern})),\\
\mathsf{elabExpr}_\Gamma(\mathsf{setter}_z(x, R)) &= \mathsf{elabExpr}_\Gamma(\mathsf{setterExpansion}_z(x, R)),\\
\mathsf{elabExpr}_\Gamma(\mathsf{cascade}_z(R, \bar{c}_0, \bar{c})) &= \mathsf{elabExpr}_\Gamma(\mathsf{cascadeExpansion}_z(R, \bar{c}_0, \bar{c})).
\end{aligned} \tag{13.53}$$

Here $\mathsf{selectorBindingAnnotation}_\Gamma$ is Equation (13.47) for a unary activation member and is none otherwise. Tuple and pattern nodes are retained because their dynamic expansions are Equations (9.5) and (9.8)–(9.11).

Pattern elaboration recursively elaborates only embedded expressions:

$$\begin{aligned}
\mathsf{elabPattern}_\Gamma(\mathsf{wild}_z) &= _,\\
\mathsf{elabPattern}_\Gamma(\mathsf{literalPat}_z(R)) &= \mathsf{elabExpr}_\Gamma(R),\\
\mathsf{elabPattern}_\Gamma(\mathsf{variablePat}_z(x)) &= ?x_z,\\
\mathsf{elabPattern}_\Gamma(\mathsf{nestedPat}_z(p)) &= \langle \mathsf{elabPattern}_\Gamma(p)\rangle,\\
\mathsf{elabPattern}_\Gamma(\mathsf{keywordPat}_z(\langle k_i, p_i?\rangle_i)) &= \langle k_i, \mathsf{elabOptionalPattern}_\Gamma(p_i?)\rangle_i.
\end{aligned} \tag{13.54}$$

An absent keyword component denotes a wildcard. The resulting forms are exactly the arguments of Equations (9.8)– (9.9).

The setter expansion, with fresh $p \neq x$, is

$$\begin{aligned}
\mathsf{setterBody}_z(x,p) &= \langle \mathsf{exprStmt}(\mathsf{implicit}(\#x\mathtt{:}, \langle \mathsf{identifier}(p)\rangle)), \mathsf{exprStmt}(\mathsf{identifier}(p))\rangle, \\
\mathsf{setterExpansion}_z(x,R) &= \mathsf{message}_z\big(\mathsf{closure}_z(\langle \mathsf{formal}(p)\rangle, \epsilon, \mathsf{setterBody}_z(x,p)), \\
&\qquad\qquad \#\mathtt{value:}, \langle R\rangle\big).
\end{aligned} \tag{13.55}$$

It therefore evaluates R once, sends the setter, and returns the argument.

A cascade receiver is evaluated once. For fresh synthetic closure identity q_c, formal declaration d_c, and identifier x_c, define

$$\begin{aligned}
&\mathsf{cascadeExpansion}_z(R, \bar{c}_0, \langle \mathsf{clause}(s_1, \bar{R}_1), \ldots, \mathsf{clause}(s_m, \bar{R}_m)\rangle) \\
&\quad = \mathsf{message}_z\big(\mathsf{closure}_{q_c}(\langle \mathsf{formal}_{d_c}(x_c)\rangle, \epsilon, \\
&\qquad\qquad \langle \mathsf{exprStmt}(\mathsf{foldExplicit}_z(\mathsf{identifier}(x_c), \bar{c}_0))\rangle \cdot \\
&\qquad\qquad \langle \mathsf{exprStmt}(\mathsf{message}(\mathsf{identifier}(x_c), s_i, \bar{R}_i)) \mid 1 \le i \le m\rangle), \\
&\qquad\qquad \#\mathtt{value:}, \langle R\rangle\big).
\end{aligned} \tag{13.56}$$

The binding map inside the synthesized closure maps x_c to d_c.

The remaining expression cases and statement elaboration are preceded by normalization of local declarations:

$$\begin{aligned}
\mathsf{elabLocalDecl}_\Gamma(\mathsf{slot}_d(\alpha?, x, \mathsf{immutable}, R)) &= \mathsf{immutable}(d, \mathsf{elabExpr}_\Gamma(R)), \\
\mathsf{elabLocalDecl}_\Gamma(\mathsf{slot}_d(\alpha?, x, \mathsf{mutable}, R?)) &= \mathsf{mutable}(d, \mathsf{elabOptionalExpr}_\Gamma(R?)), \\
\mathsf{elabLocalGroup}_\Gamma(\mathsf{sequentialGroup}(\bar{d})) &= \langle \mathsf{elabLocalDecl}_\Gamma(d) \mid d \in \bar{d}\rangle, \\
\mathsf{elabLocalGroup}_\Gamma(\mathsf{simultaneousGroup}(\bar{d})) &= \langle \mathsf{simultaneous}(\langle \mathsf{elabLocalDecl}_\Gamma(d) \mid d \in \bar{d}\rangle)\rangle, \\
\mathsf{elabLocalGroups}_\Gamma(\epsilon) &= \epsilon, \\
\mathsf{elabLocalGroups}_\Gamma(g \cdot \bar{g}) &= \mathsf{elabLocalGroup}_\Gamma(g) \cdot \mathsf{elabLocalGroups}_\Gamma(\bar{g}).
\end{aligned} \tag{13.57}$$

$$\begin{aligned}
\Gamma_q &= \mathsf{activationContext}_\Gamma(q, \bar{d}, \bar{g}), \\
\ell_q &= \mathsf{elabLocalGroups}_{\Gamma_q}(\bar{g}), \\
b_q &= \mathsf{elabStatements}_{\Gamma_q}(\bar{s}), \\
\mathsf{elabExpr}_\Gamma(\mathsf{closure}_q(\bar{d}, \bar{g}, \bar{s})) &= \mathsf{closureLiteral}(q, \mathsf{names}(\bar{d}), \ell_q, b_q), \\
\mathsf{elabExpr}_\Gamma(\mathsf{object}_L(o_h)) &= \mathsf{objectLiteral}(\mathsf{objectDescriptor}_\Gamma(L, o_h), \mathsf{objectSuperclass}_\Gamma(o_h)), \\
\mathsf{elabExpr}_\Gamma(\mathsf{classExpr}_D(d)) &= \mathsf{elabClass}_\Gamma(D, d), \\
\mathsf{elabStatements}_\Gamma(\langle \mathsf{exprStmt}(R_i)\rangle_i) &= \\
&\qquad \langle \mathsf{expr}(\mathsf{elabExpr}_\Gamma(R_i))\rangle_i, \\
\mathsf{elabStatements}_\Gamma(\bar{s} \cdot \langle \mathsf{returnStmt}(R)\rangle) &= \mathsf{elabStatements}_\Gamma(\bar{s}) \cdot \langle e_r\rangle, \\
e_r &= \mathsf{return}(\mathsf{elabExpr}_\Gamma(R)).
\end{aligned} \tag{13.58}$$

Γ_q prepends the formal and local binder maps, sets the activation to q, and enables a non-local return exactly when q is a closure with an enclosing method. A return can only be the final surface statement by Equation (13.21).

Access defaulting and the declaration cases are explicit:

$$\begin{aligned}
\mathsf{accessOf}(\mathsf{none}) &= \mathsf{protected},\\
\mathsf{accessOf}(\mathsf{public}) &= \mathsf{public},\\
\mathsf{accessOf}(\mathsf{protected}) &= \mathsf{protected},\\
\mathsf{accessOf}(\mathsf{private}) &= \mathsf{private},\\
\Gamma_f &= \mathsf{activationContext}_{\Gamma}(i_f, \bar{d}, \bar{g}),\\
\ell_f &= \mathsf{elabLocalGroups}_{\Gamma_f}(\bar{g}),\\
b_f &= \mathsf{elabStatements}_{\Gamma_f}(\bar{s}),\\
a_f &= \mathsf{accessOf}(\alpha?),\\
d_f &= \mathsf{method}_{i_f}(\alpha?, s, \bar{d}, \bar{g}, \bar{s}),\\
d_s &= \mathsf{slot}_d(\alpha?, x, m, R?),\\
d_l &= \mathsf{lazySlot}_d(\alpha?, x, m, R),\\
d_n &= \mathsf{nestedClass}_d(\alpha?, D),\\
\mathsf{elabDecl}_{\Gamma}(d_f) &= \langle i_f, s, a_f, \mathsf{names}(\bar{d}), \ell_f, b_f, \Gamma.K\rangle,\\
\mathsf{elabDecl}_{\Gamma}(d_s) &=\\
&\quad \mathsf{ordinarySlotDef}(d, x, a_f, m, \mathsf{elabOptionalExpr}_{\Gamma}(R?)),\\
\mathsf{elabDecl}_{\Gamma}(d_l) &=\\
&\quad \mathsf{lazySlotDef}(d, x, a_f, m, \mathsf{elabExpr}_{\Gamma}(R)),\\
\mathsf{elabDecl}_{\Gamma}(d_n) &= \mathsf{nestedClassDef}(d, a_f, \mathsf{elabClass}_{\Gamma}(D, D)).
\end{aligned} \tag{13.59}$$

Γ_f prepends the method's formal and local declarations and enables local return. The ordinary- and lazy-slot constructors produce the physical slot and the exact getter/setter bodies of Section 8.6. The nested-class constructor produces its memoizing getter. These constructors have different domains and names.

Class and object elaboration uses the message-template domain in Equation (8.13). Define

$$\mathsf{elabMessageTemplate}_{\Gamma}(\mathsf{clause}(s, \bar{R})) = \langle s, \mathsf{elabExprList}_{\Gamma}(\bar{R})\rangle. \tag{13.60}$$

For a named class declaration D with structure h, let M_D and M_D^c be the stable instance- and class-side mixins induced by h. Its descriptor constructor is

$$\mathsf{bodyDescriptor}_{\Gamma}(D, x, u, s_f, \bar{d}, h, c) = \langle D, x, u, s_f, \mathsf{names}(\bar{d}), M_D, M_D^c, \mathsf{elabMessageTemplate}_{\Gamma}(c)\rangle. \tag{13.61}$$

This is exactly the tuple χ_b of Equation (8.1). Omitted inheritance is

$$\begin{aligned}
\mathsf{defaultSuperclass}_{\Gamma} &= \begin{cases} \mathsf{Object} & \Gamma.K = \mathsf{top},\\ \mathsf{implicit}(\#\mathsf{Object}, \epsilon, \mathsf{bindingAnnotation}_{\Gamma}(\mathsf{Object})) & \text{otherwise}, \end{cases}\\
\mathsf{defaultInitializer} &= \mathsf{clause}(\#\mathsf{new}, \epsilon).
\end{aligned} \tag{13.62}$$

The two non-application class cases are therefore

$$\begin{aligned}
&\mathsf{elabClass}_{\Gamma}(D, \mathsf{classDecl}(x, s_f, \bar{d}, \mathsf{defaultInheritance}(h)))\\
&\quad = \mathsf{classBody}(\chi_b, \mathsf{defaultSuperclass}_{\Gamma}),\\
&\qquad \chi_b = \mathsf{bodyDescriptor}_{\Gamma}(D, x, \mathsf{implicitSuperclass}, s_f, \bar{d}, h, \mathsf{defaultInitializer}),\\
&\mathsf{elabClass}_{\Gamma}(D, \mathsf{classDecl}(x, s_f, \bar{d}, \mathsf{explicitInheritance}(R, c, h)))\\
&\quad = \mathsf{classBody}(\chi_b, \mathsf{elabExpr}_{\Gamma}(R)),\\
&\qquad \chi_b = \mathsf{bodyDescriptor}_{\Gamma}(D, x, \mathsf{explicitSuperclass}, s_f, \bar{d}, h, c).
\end{aligned} \tag{13.63}$$

For a mixin chain, write its first inheritance head as $h_0 = \mathsf{inheritanceHead}(R_0, c_0)$ and its nonempty suffix as $\bar{h} = \langle h_1, \ldots, h_k \rangle$, where $h_j = \mathsf{inheritanceHead}(R_j, c_j)$. Each application uses c_{j-1} to initialize its superclass and c_j to initialize the newly applied mixin. The synthetic identities and names required for an intermediate application are deterministic injections of the enclosing declaration identity and its one-based position:

$$\begin{aligned} D_{A,j} &= \mathsf{applicationDeclId}(D, j), \quad i_{A,j} = \mathsf{applicationInitId}(D, j), \\ n_{A,j} &= \mathsf{applicationName}(D, j), \quad \bar{x}_{A,j} = \mathsf{applicationFormals}(D, j, \mathrm{selector}(c_j)). \end{aligned} \tag{13.64}$$

The names in $\bar{x}_{A,j}$ are pairwise fresh, and their number is the arity of $\mathrm{selector}(c_j)$. Let $M^c_{A,j}$ be the stable class-side mixin containing only the synthesized public primary factory for that signature. The complete descriptor of an intermediate application is

$$\begin{aligned} \mathsf{anonAppDesc}_\Gamma(D, j, c_{j-1}, c_j) = \langle & D_{A,j}, n_{A,j}, \mathrm{selector}(c_j), \bar{x}_{A,j}, \\ & M^c_{A,j}, i_{A,j}, m_{j-1}, m_j \rangle, \\ m_{j-1} &= \mathsf{elabMessageTemplate}_\Gamma(c_{j-1}), \\ m_j &= \mathsf{elabMessageTemplate}_\Gamma(c_j). \end{aligned} \tag{13.65}$$

For the last application of a body-less named class, the declaration's own identity, name, factory formals, and class-side mixin replace those synthetic components:

$$\begin{aligned} &\mathsf{namedAppDesc}_\Gamma(D, x, s_f, \bar{d}, c_{k-1}, c_k) = \\ &\quad \langle D, x, s_f, \mathsf{names}(\bar{d}), M^c_D, i_{A,k}, \mathsf{elabMessageTemplate}_\Gamma(c_{k-1}), \mathsf{elabMessageTemplate}_\Gamma(c_k) \rangle. \end{aligned} \tag{13.66}$$

The anonymous prefix fold returns both the resulting class expression and the last initializer clause; retaining the latter makes the next superclass initializer explicit:

$$\begin{aligned} \mathsf{foldAnonymous}_{\Gamma,D}(E, c_0, \epsilon) &= \langle E, c_0 \rangle, \\ \mathsf{foldAnonymous}_{\Gamma,D}(E, c_{j-1}, \langle h_j \rangle \cdot \bar{h}) &= \mathsf{foldAnonymous}_{\Gamma,D}(E_j, c_j, \bar{h}), \\ E_j &= \mathsf{mixinApply}(\chi_{A,j}, E, \mathsf{elabExpr}_\Gamma(R_j)), \\ \chi_{A,j} &= \mathsf{anonAppDesc}_\Gamma(D, j, c_{j-1}, c_j). \end{aligned} \tag{13.67}$$

A body-less chain reserves its last application for the named descriptor. Put

$$\mathsf{foldAnonymous}_{\Gamma,D}(\mathsf{elabExpr}_\Gamma(R_0), c_0, \langle h_1, \ldots, h_{k-1} \rangle) = \langle E_{k-1}, c_{k-1} \rangle. \tag{13.68}$$

Then

$$\begin{aligned} &\mathsf{elabClass}_\Gamma(D, \mathsf{classDecl}(x, s_f, \bar{d}, \mathsf{mixinChain}(R_0, c_0, \bar{h}, \mathsf{none}))) = \\ &\quad \mathsf{mixinApply}(\mathsf{namedAppDesc}_\Gamma(D, x, s_f, \bar{d}, c_{k-1}, c_k), E_{k-1}, \mathsf{elabExpr}_\Gamma(R_k)). \end{aligned} \tag{13.69}$$

This equation is defined only when $s_f = \mathrm{selector}(c_k)$ and $k \geq 1$, as the grammar requires. If the chain ends in a body, put

$$\mathsf{foldAnonymous}_{\Gamma,D}(\mathsf{elabExpr}_\Gamma(R_0), c_0, \bar{h}) = \langle E_k, c_k \rangle. \tag{13.70}$$

Then

$$\begin{aligned} &\mathsf{elabClass}_\Gamma(D, \mathsf{classDecl}(x, s_f, \bar{d}, \mathsf{mixinChain}(R_0, c_0, \bar{h}, h))) = \\ &\quad \mathsf{classBody}(\mathsf{bodyDescriptor}_\Gamma(D, x, \mathsf{explicitSuperclass}, s_f, \bar{d}, h, c_k), E_k). \end{aligned} \tag{13.71}$$

Equations (13.65) and (13.66) are instances of the complete application tuple χ_a in Equation (8.2); no descriptor component is supplied informally.

An object literal uses the same body derivation with an object-literal identity L, the public nullary synthetic class-side factory, and the optional header's inheritance receiver and initializer clause:

$$\begin{aligned}
m_0 &= \mathsf{elabMessageTemplate}_\Gamma(\mathsf{defaultInitializer}),\\
m_1 &= \mathsf{elabMessageTemplate}_\Gamma(c),\\
\mathsf{objectDescriptor}_\Gamma(L, \mathsf{implicitObjectHeader}(h)) &= \langle L, \mathsf{implicitSuperclass}, M_{L,h}, M^c_{L,h}, m_0\rangle,\\
\mathsf{objectDescriptor}_\Gamma(L, \mathsf{explicitObjectHeader}(R,c,h)) &= \langle L, \mathsf{explicitSuperclass}, M_{L,h}, M^c_{L,h}, m_1\rangle,\\
\mathsf{objectSuperclass}_\Gamma(\mathsf{implicitObjectHeader}(h)) &= \mathsf{defaultSuperclass}_\Gamma,\\
\mathsf{objectSuperclass}_\Gamma(\mathsf{explicitObjectHeader}(R,c,h)) &= \mathsf{elabExpr}_\Gamma(R).
\end{aligned} \tag{13.72}$$

Equation (13.72) is exactly the descriptor χ_o in Equation (9.12).

For an identified unit S, the maps consumed by P are now defined directly. Let $\mathsf{ancestors}_S(n)$ be the strict lexical-ancestor sequence of node n, nearest first. Let M_K be the stable mixin identity derived from class-body identity K. Then

$$\begin{aligned}
\mathsf{sourceMap}_S(d) &= \mathsf{subtree}_S(d),\\
\mathsf{declMixinMap}_S(K) &= M_K,\\
\mathsf{lexParentMap}_S(D) &= \mathsf{firstClass}(\mathsf{ancestors}_S(D)),\\
\mathsf{bodyMap}_S(i_f) &= \mathsf{elabStatements}_{\Gamma_{i_f}}(\mathsf{surfaceBody}_S(i_f)),\\
\mathsf{scopeChainMap}_S(z) &= \langle d \in \mathsf{ancestors}_S(z) \mid d \in \mathit{ClassBodyDeclId} \uplus \mathit{ActDeclId}\rangle,\\
\mathsf{activationMemberMap}_S(q) &= \{\#x \mid x \text{ is a formal or local declared directly by } q\},\\
\mathsf{ownerMap}_S(K) &= \begin{cases}
\mathsf{activationOwner} & \mathsf{firstOwner}_S(K) \in \mathit{ActDeclId},\\
\mathsf{classOwner} & \mathsf{firstOwner}_S(K) \in \mathit{ClassDeclId},\\
\mathsf{objectLiteralOwner} & \mathsf{firstOwner}_S(K) \in \mathit{ObjLitDeclId},\\
\mathsf{topOwner} & \mathsf{firstOwner}_S(K) = \mathsf{none}.
\end{cases}
\end{aligned} \tag{13.73}$$

The mixin entry itself is

$$\begin{aligned}
\phi_K &= \mathsf{explicitMethods}_S(K) \uplus \mathsf{synthMethods}_S(K),\\
\delta_K &= \mathsf{physicalSlots}_S(K), & \nu_K &= \mathsf{nestedDecls}_S(K),\\
\iota_K &= \mathsf{initializer}_S(K), & \mathsf{mixinMap}_S(M_K) &= \langle K, \phi_K, \delta_K, \nu_K, \iota_K\rangle.
\end{aligned} \tag{13.74}$$

The disjoint union is defined only when selectors are unique. $\mathsf{initializer}_S(K)$ is the tuple in Equation (3.6): its factory selector and formals come from the class header, its source-ordered groups are transformed by Equation (13.59), and its body by Equation (13.58).

Top-level actor seeds and the one underspecified pattern form are

$$\begin{aligned}
\mathsf{actorSeedMap}_S(M_K) &= \langle K, M_K, \mathsf{factoryDescriptor}_S(K)\rangle \quad \mathsf{ownerMap}_S(K) = \mathsf{topOwner},\\
\mathsf{patternVariableMap}_S(z) &= \mathsf{patternVariableParameter}_S(z) \quad \mathsf{variablePat}_z(x) \in S.
\end{aligned} \tag{13.75}$$

The latter is precisely the explicit parameter isolated by Equation (9.11); it is not used for any other surface form.

Collecting Equations (13.73)– (13.75), define

$$\begin{aligned}\mathsf{derivedProgram}(S) = \langle P, e\rangle \Longleftrightarrow\ & P.unit = S \wedge \\ & P.source = \mathsf{sourceMap}_S \wedge \\ & P.declMixin = \mathsf{declMixinMap}_S \wedge \\ & P.mixin = \mathsf{mixinMap}_S \wedge \\ & P.lexParent = \mathsf{lexParentMap}_S \wedge \\ & P.body = \mathsf{bodyMap}_S \wedge \\ & P.scopeChain = \mathsf{scopeChainMap}_S \wedge \\ & P.activationMembers = \mathsf{activationMemberMap}_S \wedge \\ & P.owner = \mathsf{ownerMap}_S \wedge \\ & P.actorSeed = \mathsf{actorSeedMap}_S \wedge \\ & P.patternVariable = \mathsf{patternVariableMap}_S \wedge \\ & e = \mathsf{elabTop}_S(S).\end{aligned} \tag{13.76}$$

The unchanged atom and platform maps are supplied as parameters to derivedProgram when a program is installed.

The static predicates are defined without an informal residual clause:

$$\begin{aligned}\mathsf{bindingsOK}(S) &\Longleftrightarrow \big(\forall \Gamma, B_i \in \Gamma.\bar{B}.\ \mathsf{declaredNames}(B_i) \text{ are pairwise distinct}\big) \wedge \\ &\quad \big(\forall \mathsf{outer}_z(x) \in S.\ \mathsf{classBind}_{\Gamma_z}(x) \neq \uparrow \wedge\, \Gamma_z.D_0 \neq \mathsf{none}\big) \wedge \\ &\quad \big(\forall \mathsf{self}_z \text{ used as receiver in } S.\ \Gamma_z.D_0 \neq \mathsf{none}\big), \\ \mathsf{membersOK}(S) &\Longleftrightarrow \forall K, s.\ |\{f \mid f \text{ is induced directly in } K \wedge \mathrm{selector}(f) = s\}| \leq 1, \\ \mathsf{arityOK}(S) &\Longleftrightarrow \forall f.\ |f.\bar{x}| = \mathsf{arity}(\mathrm{selector}(f)), \\ \mathsf{returnsOK}(S) &\Longleftrightarrow \forall R \in \mathit{ReturnNode}(S).\ (\mathsf{local}(R) \Rightarrow r_t(R)) \wedge (\mathsf{nonlocal}(R) \Rightarrow h_t(R)), \\ \mathsf{contextsOK}(S) &\Longleftrightarrow \mathsf{bindingsOK}(S) \wedge \mathsf{returnsOK}(S) \wedge \forall R \in \mathit{SuperSendNode}(S).\ q(R) \neq \mathsf{none}, \\ \mathsf{surfaceOK}(S) &\Longleftrightarrow \mathsf{idsOK}(S) \wedge \mathsf{membersOK}(S) \wedge \mathsf{arityOK}(S) \wedge \mathsf{contextsOK}(S) \wedge \mathsf{shapesOK}(S).\end{aligned} \tag{13.77}$$

The surface-shape condition is

$$\begin{aligned}\mathsf{lastClause}(\langle h_1, \ldots, h_k\rangle) = c_k \quad & \text{when } h_k = \mathsf{inheritanceHead}(R_k, c_k), \\ \mathsf{shapesOK}(S) \Longleftrightarrow\ & \text{every } \mathsf{outer}_z(x) \text{ or } \mathsf{super}_z \text{ is the receiver of a message} \wedge \\ & \text{every return is final in its statement sequence} \wedge \\ & \text{every immutable slot has an initializer} \wedge \\ & \text{every lazy slot has an initializer} \wedge \\ & \forall \mathsf{classDecl}(x, s_f, \bar{d}, \mathsf{mixinChain}(R_0, c_0, \bar{h}, \mathsf{none})) \in S.\ s_f = \mathrm{selector}(\mathsf{lastClause}(\bar{h})).\end{aligned} \tag{13.78}$$

Ordinary unbound implicit sends are permitted by Equation (13.47); named outer targets and duplicate declarations in one binder map are rejected by the first predicate.

The source-to-core judgment is

$$G, \kappa, u \vdash \mathsf{frontEnd} \Downarrow \langle P, e, \kappa' \rangle \iff \mathsf{parse}_G(u) = t, \quad \mathsf{ast}_{G,u}(t) = \langle R_0, \bar{m} \rangle, \quad R = \mathsf{attachMetadata}_u(R_0, \bar{m}), \quad \mathsf{identify}_\kappa(R) = \langle S, \kappa' \rangle, \quad \mathsf{surfaceOK}(S), \quad \mathsf{deriveProgram}(S) = \langle P, e \rangle, \quad \mathsf{TL}_P(e). \tag{13.79}$$

Core validity must cover expressions stored throughout P, not only the top-level expression. Let $\mathsf{coreRoots}(P, e)$ be the least finite set containing e and every expression occurring in method and closure bodies, their local initializers, mixin initializer groups and bodies, pattern-variable expansions, and the deferred superclass or mixin messages stored in class, mixin-application, object-literal, and actor-seed descriptors:

$$\begin{aligned} \mathsf{coreRoots}(P, e) = \mathsf{sub}\big(\{e\} &\cup P.\mathit{methodBodies} \cup P.\mathit{methodLocals} \\ &\cup P.\mathit{closureBodies} \cup P.\mathit{closureLocals} \\ &\cup P.\mathit{mixinInitializers} \cup P.\mathit{patternExpansions} \\ &\cup P.\mathit{classMessages} \cup P.\mathit{mixinMessages} \\ &\cup P.\mathit{objectMessages} \cup P.\mathit{actorMessages}\big). \end{aligned} \tag{13.80}$$

Here sub recursively includes tuple, pattern, cascade, send, lazy, class, mixin-application, object-literal, and nested-class subexpressions. For Equation (13.80), $P.\mathit{mixinInitializers}$ projects the groups and bodies of all mixin initializers, $P.\mathit{patternExpansions}$ is the range of the finite pattern-variable table, and the four *Messages* projections collect the message templates in the corresponding finite descriptor tables. These are abbreviations for existing Program fields, not new semantic stores. For fuel k, $\mathsf{annOK}^k_P(r, b)$ checks every declaration and scope annotation in r, where b says whether a super send is permitted. In particular, it checks that named class and closure declarations occur in the appropriate finite Program table, that an annotated implicit send is declared by its recorded scope, and that a super send occurs only in installed class code. The executable gate is

$$\begin{aligned} \mathsf{coreCheck}_k(P, e) = \mathsf{true} \iff\ & P.\mathit{declMixin} \text{ is injective} \wedge \\ & \forall r \in \mathsf{coreRoots}(P, e) \setminus \{e\}.\ \mathsf{annOK}^k_P(r, \mathsf{true}) \wedge \\ & \mathsf{annOK}^k_P(e, \mathsf{false}) \ \wedge\ \mathsf{TL}_P(e). \end{aligned} \tag{13.81}$$

Core validity, used by loaders and reflective validation, is therefore the independently checkable predicate

$$\begin{aligned} \mathsf{coreOK}(P, e) \iff \exists k > 0.\ \mathsf{coreCheck}_k(P, e) = \mathsf{true} \wedge \\ \mathsf{deriveProgram}(P.\mathit{unit}) = \langle P, e \rangle \ \wedge \mathsf{surfaceOK}(P.\mathit{unit}). \end{aligned} \tag{13.82}$$

For reflective validation, which replaces a program but does not itself start its top-level expression, define the projection

$$\mathsf{coreProgramOK}(P) \iff \exists e.\ \mathsf{deriveProgram}(P.\mathit{unit}) = \langle P, e \rangle \wedge \mathsf{coreOK}(P, e). \tag{13.83}$$

Equations (13.79) and (13.82) are the promised static connection from the Specification's PEG AST to every declaration and send annotation consumed by the dynamic semantics. Moreover, determinism of PEG recognition and the functional equations above give the front-end uniqueness property

$$\frac{G_{\text{NEWSPEAK}}, \kappa, u \vdash \mathsf{frontEnd} \Downarrow X \qquad G_{\text{NEWSPEAK}}, \kappa, u \vdash \mathsf{frontEnd} \Downarrow Y}{X = Y}. \tag{13.84}$$

Totality holds on exactly the inputs that parse completely and satisfy $\mathsf{surfaceOK}$; failure at any earlier partial function is a static error, never a stuck core term.

13.4 Reified language services

The host registry already introduced in Equation (9.22) now has the concrete type

$$\mathcal{L} : \mathit{LanguageId} \rightharpoonup \mathit{ObjRef}. \tag{13.85}$$

$\mathcal{L}(\lambda)$ is an ordinary Newspeak language object. It answers the public message `frontEndImage` with an immutable value

$$I_L = \mathsf{frontEndImage}\langle o_p, o_e, o_c, o_s, G, v_L\rangle, \tag{13.86}$$

whose components are parser, elaborator, compiler, and Simulator objects, the declarative PEG value, and a language-defined revision value. This record is a protocol value, not a new privileged heap-record kind. Its component objects have ordinary classes, methods, enclosing objects, and slots.

For a caller actor A, write

$$\Omega, A \vdash \mathsf{call}(o, s, \bar{v}) \Downarrow \langle \Omega', v\rangle \tag{13.87}$$

iff the ordinary public send of $\langle s, \bar{v}\rangle$ to o starts in A, follows the actor and sequential transition relations already defined, and returns v to its caller. A signaled exception or broken reply produces the corresponding failure rather than a successful judgment. Thus Equation (13.87) is notation for the reflexive transitive closure of existing rules, not a second evaluator.

Surface ASTs and annotated-core images are ordinary immutable Newspeak value graphs. Executable artifacts are immutable values as well. The VM boundary supplies injective partial decoders

$$\begin{aligned} \mathsf{decodeSurface} &: \mathit{ObjRef} \rightharpoonup \mathit{SurfaceAST}, \\ \mathsf{decodeCore} &: \mathit{ObjRef} \rightharpoonup (\mathit{Program} \times \mathit{CoreExpression}), \\ \mathsf{decodeArtifact} &: \mathit{ObjRef} \rightharpoonup \mathit{Artifact}. \end{aligned} \tag{13.88}$$

They inspect representation and allocate no object, invoke no user method, and accept only finite immutable graphs. Their inverses are ordinary constructors provided by the language implementation.

Admission is attached to the values obtained at a use boundary, not to the permanent identity of a component object. A front-end image is admitted when all four component references are live ordinary Newspeak objects, its grammar passes Equation (13.8), and successful elaboration satisfies the complete checker:

$$\begin{aligned} \mathsf{frontEndAdmitted}_\Omega(I_L) \Longleftrightarrow\ & I_L = \mathsf{frontEndImage}\langle o_p, o_e, o_c, o_s, G, v_L\rangle \ \wedge \\ & \bigwedge_{o \in \{o_p, o_e, o_c, o_s\}} \mathsf{liveOrdinary}_\Omega(o) \ \wedge\ \mathsf{pegCertOK}(G) = \mathsf{true} \ \wedge \\ & \forall S, P, e.\ \mathsf{surfaceOK}(S) \wedge \mathsf{deriveProgram}(S) = \langle P, e\rangle \\ & \qquad \Rightarrow \exists k > 0.\ \mathsf{coreCheck}_k(P, e) = \mathsf{true}. \end{aligned} \tag{13.89}$$

Here $\mathsf{liveOrdinary}_\Omega(o)$ means that o is present in the ordinary-object component of the appropriate heap in Ω. The compiler's product is separately admitted by Equation (13.92); the Simulator's code/PC representation is separately admitted by the certificate in Equation (11.67). Replacing a component therefore grants no authority to bypass the fixed core boundary.

Capture one front-end image and run its three stages by ordinary sends:

$$\begin{aligned}
&\Omega_0, A, \lambda, u \vdash \mathsf{compileUnit} \Downarrow \langle \Omega_4, B, P, e, I_L \rangle \quad \Longleftrightarrow \\
&\quad o_L = \mathcal{L}(\lambda) \ \wedge \ \Omega_0, A \vdash \mathsf{call}(o_L, \#\texttt{frontEndImage}, \epsilon) \Downarrow \langle \Omega_1, I_L \rangle \ \wedge \\
&\quad I_L = \mathsf{frontEndImage}\langle o_p, o_e, o_c, o_s, G, v_L \rangle \ \wedge \\
&\quad \mathsf{frontEndAdmitted}_{\Omega_1}(I_L) \ \wedge \ \Omega_1, A \vdash \mathsf{call}(o_p, \#\texttt{parse:}, \langle u \rangle) \Downarrow \langle \Omega_2, v_s \rangle \ \wedge \\
&\quad \mathsf{decodeSurface}(v_s) = S \ \wedge \mathsf{idsOK}(S) \ \wedge \ \Omega_2, A \vdash \mathsf{call}(o_e, \#\texttt{elaborate:}, \langle v_s \rangle) \Downarrow \langle \Omega_3, v_k \rangle \ \wedge \\
&\quad \mathsf{decodeCore}(v_k) = \langle P, e \rangle \ \wedge \mathsf{coreOK}(P, e) \ \wedge \ \Omega_3, A \vdash \mathsf{call}(o_c, \#\texttt{compile:}, \langle v_k \rangle) \Downarrow \langle \Omega_4, v_b \rangle \ \wedge \\
&\quad \mathsf{decodeArtifact}(v_b) = B \ \wedge \mathsf{artifactOK}(B, P, e).
\end{aligned} \tag{13.90}$$

The source string u is represented by the platform's immutable String value; the notation passes its contents rather than granting the compiler a raw host pointer. A language implementation may combine stages internally, but its observable result must refine Equation (13.90).

An artifact contains the core image, concrete code images, relocation data, source/control maps, and the captured language revision:

$$B = \langle P, e, \mathcal{B}, \mathsf{reloc}, \mathsf{controlMap}, v_L \rangle. \tag{13.91}$$

Its validity is

$$\begin{aligned}
\mathsf{artifactOK}(B, P, e) \Longleftrightarrow\ & \mathsf{coreOK}(P, e) \wedge B.P = P \wedge B.e = e \wedge \\
& \mathrm{dom}(B.\mathsf{controlMap}) = \mathsf{pcs}(B.\mathcal{B}) \wedge \\
& \mathsf{targetsOK}(B.\mathcal{B}, B.\mathsf{reloc}) \wedge \mathsf{codeRefinesCore}(B.\mathcal{B}, P, e).
\end{aligned} \tag{13.92}$$

The final conjunct of Equation (13.92) is the compiler-correctness obligation. Concrete execution, observed at the source/control map, must weakly simulate the sequential and actor semantics. A changed compiler may accept new surface forms or omit old conveniences by changing parsing and elaboration, but an artifact admitted by the VM must still satisfy the fixed core contract. Changing the instruction set or primitive meanings requires a VM refinement, not merely a language front-end change.

The earlier compilation-unit judgment is therefore the projection

$$\mathcal{L}, \Omega, A \vdash \langle \lambda, u \rangle \mathsf{compilesTo} \langle P, e, B \rangle \quad \Longleftrightarrow \quad \exists \Omega', I_L.\ \Omega, A, \lambda, u \vdash \mathsf{compileUnit} \Downarrow \langle \Omega', B, P, e, I_L \rangle. \tag{13.93}$$

13.5 Ordinary change, phase behavior, and bootstrap

Let $\mathsf{components}_\Omega(\lambda)$ be the value obtained by the first call in Equation (13.90). A successful existing reflective commit may change methods of o_L, o_p, o_e, o_c, o_s, or may change ordinary slots from which `frontEndImage` constructs its result. Its effect on the next compilation is simply

$$\Omega \xrightarrow{\mathsf{commitReflection}(A, \Delta, n+1)} \Omega' \ \wedge \ \mathsf{components}_\Omega(\lambda) \neq \mathsf{components}_{\Omega'}(\lambda). \tag{13.94}$$

Equation (13.94) is a consequence that an ordinary reflection transition may have; it is not a new transition rule and there is no installParser or installCompiler command. Likewise, assigning a new component object through the language object's ordinary mutable protocol needs no reflection at all when the caller already has that authority.

The phase behavior follows from existing activation and dispatch rules:

$$\begin{aligned} \mathsf{capturedImage}(c) &= I_L && \text{for a compilation } c \text{ already past its first call,} \\ \mathsf{currentTerm}_{\Omega'}(a) &= \mathsf{currentTerm}_{\Omega}(a) && \text{for every materialized stage activation } a, \\ \mathsf{dispatchProgram}(a') &= P' && \text{for a later send dispatched after the commit.} \end{aligned} \tag{13.95}$$

Thus self-replacement needs no circular premise. Source defining a replacement parser is parsed and compiled by the currently executing language objects; an ordinary commit changes their code or references; subsequent compilation sees the replacement. A replacement not expressible in the old accepted language must enter as an artifact already satisfying Equation (13.92).

Finally, metacircularity has a finite base. An initial image supplies

$$\mathsf{bootstrap} = \langle \Omega_0, \mathcal{L}_0, \mathsf{decodeSurface}, \mathsf{decodeCore}, \mathsf{decodeArtifact}, \mathsf{loadCode}, \mathsf{primitiveStep} \rangle. \tag{13.96}$$

After bootstrap, every object reachable through $\mathcal{L}_0$ is an ordinary Newspeak object and can be changed by the mechanisms above. The decoders, artifact loader, concrete instruction dispatch, allocation, and host I/O remain VM parameters unless a particular implementation supplies a Newspeak refinement of them. This boundary states what is not metacircular without preventing a more completely bootstrapped VM from discharging those parameters.

14 Established invariants and mechanization status

The following are semantic invariants, admissibility conditions for reflective transactions, and the principal claims tracked by the machine-checked companion:

1. every ordinary object has exactly one live class;
2. every base class chain is finite and ends in Top;
3. each class retains the identity of its applied mixin and enclosing object;
4. each class retains a tagged creation origin and its inducing source declaration;
5. each created class has a fresh metaclass, while repeated applications may deliberately share an instance-side mixin;
6. each mixin retains its inducing declaration identity;
7. an object's physical layout contains exactly the disjoint root-to-leaf slot layouts of its class chain, all initialized to nil before any instance initializer executes;
8. each application has its own cells for the declarations of its mixin, including when that mixin occurs repeatedly in one class chain;
9. a nested-class cache is per receiver and application and contains at most one class for each pair of application and inducing declaration identity;
10. equal atomic payloads denote the same canonical literal object;
11. each object-literal evaluation creates a fresh object, class, and metaclass but reuses the literal declaration's mixin;

12. a module definition is precisely an instantiable class created by a top-level class expression and is a value object;

13. direct method selectors are unique within a mixin after elaboration;

14. a dispatch result determines both the method and the activation's current class—method bodies never recompute the latter from the receiver alone;

15. private methods participate only in lexical private selection and unrestricted system lookup, never inherited public/protected lookup;

16. failed user dispatch invokes DNU exactly once for the original message; failure inside the chosen DNU body is an ordinary new computation;

17. mutable objects and active computations belong to exactly one actor; cross-actor arguments contain only values, far references, or promise identities;

18. no actor observes a far reference into its own heap, because transfer back to the target actor normalizes it to the near referent;

19. every actor executes at most one turn, takes mailbox entries in FIFO order, and receives causally related packets in E-order;

20. each promise is settled at most once, by fulfillment or breaking, and every dependent eventual send is resumed only after settlement;

21. reflective source-structure changes and all required re-elaboration are committed atomically.

22. every reflective read or change is authorized by an unforgeable mirror whose target and rights cover that exact operation;

23. a successful reflective transaction advances exactly one VM program version, and a failed transaction changes no program, heap, activation, continuation, or version component;

24. reflective layout migration preserves retained physical slot identities, initializes new slots to nil, and makes removed slots unreachable;

25. every actor in one VM cohort observes the same committed program version, while a distinct VM changes only through its own authorized transaction;

26. already materialized control is not rewritten by a code install, whereas later dispatch—including dispatch of an in-flight message—uses the version current when that dispatch occurs;

27. every running or paused nonempty actor stack has exactly one active control image at its top; each lower activation has an explicit suspended term and resumable control location;

28. every copied live frame has a fresh activation identity, while retained frames preserve identity and stack-structural links internal to a copied segment are consistently renamed;

29. a paused-stack mutation succeeds only for its current unforgeable pause token, and every successful mutation invalidates that token;

30. stack replacement is atomic, makes discarded activations uncontinuable, and exposes no partially materialized frame;

31. full-speed resumption continues the ordinary sequential relation from the installed control image rather than introducing a second execution semantics.

32. a weak array does not retain its elements; collection atomically replaces every cell whose target is reclaimed by nil;

33. a weak map does not retain its keys, while the value of an entry is conditionally strong exactly when both the map and its key are strongly reachable; a value-to-key cycle cannot retain that key;

34. every retained weak-container reference resolves after collection, because entries or cells naming a selected identity are cleared in the same safepoint transition.

35. every accepted compilation unit has a successful PEG parse, injective source identities, a statically well-formed surface AST, and an independently checkable annotated-core image;

36. parser, elaborator, compiler, and Simulator components are ordinary Newspeak objects: existing reflection and mutation change them, while every executable artifact admitted by the VM refines the fixed core semantics.

14.1 Machine-checked status

The companion development in `formal-semantics/lean` formalizes the domains and transition relations above in Lean. Its `Newspeak.Properties` module exports only established results, and its `THEOREMS.md` ledger records every item below as checked. The development contains no `sorry`, `admit`, or additional axiom. At the language-semantics boundary it establishes:

- uniqueness of class chains; determinism of unrestricted, public, and protected lookup; equivalence of the executable and named-rule lookup formulations; and both the static and dynamic forms of the elaborated/ECOOP-style implicit-binding equivalence;

- determinism and well-formedness preservation of the complete sequential relation through evaluation order, DNU, invocation, closures, local and non-local return, slots, class and instance construction, initialization, object literals, nested classes, exception reification, tuples, patterns, and top-level execution;

- exact finite heap and environment domains, exact duplicate-free allInstancesOf and mixin-application enumeration, unique physical layouts, stable provenance, and the object/module invariants listed above;

- deterministic selected actor turns, isolation and transfer normalization, FIFO dequeue, causal delivery constraints, terminal promise persistence, at-most-once settlement, and notification only after terminal commitment, while leaving actor and network scheduling relational;

- authorization, atomic success/failure, exact version advancement, validation preservation, materialized-control stability, debugger stack imaging and replacement, full-speed resumption, and commutation of pairwise independent reflective commands;

- exact strong and conditional reachability, stabilization and least fixed-point results for weak-map ephemerons, atomic WeakArray and WeakMap clearing, post-collection validity, well-formedness preservation, and a finite stuttering refinement of the base semantics; and

- determinism of PEG recognition, projection, stable identification, and elaboration; soundness of the finite grammar certificate; complete derivation and validation of the annotated core; and finite-execution refinement from an admitted compiled artifact to annotated-core execution.

14.2 Auditable relational bridge

The companion directory `formal-semantics/prolog` provides a proof-facing relational rendering between this mathematical presentation and Lean. Its authoritative `newspeak.pl` source is deliberately confined to a small declarative Prolog subset: definite clauses and the declarative `dif/2` constraint, with absence represented explicitly as data. It does not use cut, negation as failure, dynamic clauses, meta-calls, instantiation tests, side effects, or dependence on the Prolog search order. Every clause carries the label of its inducing rule or equation and the name of its Lean counterpart. A source checker rejects constructs outside this subset, verifies both references, and emits a normalized, hashed traceability manifest. The companion checked equation-disposition ledger makes coverage total rather than silently treating an absent clause as irrelevant. Encoded status is inferred only from an actual Horn clause. Every other numbered equation must be classified, with a rationale, as a direct term representation, an abstract contract, a theorem-only proposition, or deferred relational work. The checker rejects missing, duplicate, unknown, or contradictory classifications. At this writing the 361 numbered equations comprise 291 encoded equations, 51 term representations, 10 abstract contracts, and 9 theorem-only propositions; no equation remains deferred. The 86 named natural-deduction rules are all represented as well. These counts are regenerated from the sources rather than maintained by hand.

The completed dispatch correspondence covers the finite-store and runtime projections required by the dispatch kernel, finite class chains, unrestricted lookup, the independent defining-class judgment, public and protected lookup, ordinary dispatch, super dispatch including the explicit Object/Top errors, complete lexical-class chains and lexical depth, enclosing receivers, outer and self dispatch, all four annotated implicit dispatch cases, the allocation-bearing DNU fallback, the positive exhaustive complement of direct closure invocation, and all five non-eventual request-resolution and request-completion branches. It then continues through method-activation allocation, tail and non-tail method invocation, installation of completed requests into machine control, ordinary and cascade closure construction, direct closure-value dispatch, closure-activation allocation, both closure invocation modes, and the complete expression-order fragment: literal and self values, ordinary and eventual receivers, the four non-ordinary request forms, and left-to-right argument accumulation for both sends and deferred messages. The simultaneous-local expansion and its installation-before-resolution ordering are also represented, as are the complete body/local machine (including normal halt and resumption), canonical atoms, direct, current, and lazy object-slot transitions, return targets and dependent activations, heap-wide continuation invalidation, and all successful and failing return transitions. Sequential and simultaneous instance-slot groups, their PastFuture installation-before-resolution expansion, and their internal continuation sequence are covered as well. The bridge also covers Factory-New from parameter extraction and root-to-leaf layout through nil-filled object allocation, initializer-activation allocation, and the resulting machine transition. The superclass/mixin coordinator is now covered as well, including initializer-message validation, wrong-factory outcomes, and fresh applied-mixin initializer activations; a final correspondence theorem identifies the union of the factory, coordinator, and slot-group clauses with the complete Lean instance-initialization relation. The class-construction bridge additionally covers inheritance validation, enclosing-object selection, both factory forms, class/metaclass-pair allocation, and all nine class-body and mixin-application clauses. It exposes one current representational mismatch rather than hiding it: the operational rules above and their Prolog

clauses have no global heap-validity test, while Lean's two commit constructors currently carry a proof that the allocated heap is well formed. Accordingly, `prologClassConstructionStep_iff` pairs a raw construction step with the explicitly named `PrologClassConstructionCommitSafe` obligation. Both exception-library boundary rules are connected bidirectionally too. The Horn presentation uses an explicit proof-carrying graph application for reifyError; this is precisely the equality premise printed in Rules Runtime-Error-Signal and Throw-Signal, not a second allocator or an appeal to Prolog side effects. The bridge now continues through object-literal allocation, receiver-local nested-class caching, tuple, cascade, and recursive pattern expansion, and Top-Halt. Each staged sequential union is itself represented by explicit Horn clauses, preventing a proved leaf rule from being accidentally omitted from the machine relation. The final theorem `prologSequentialStepWithTopLevel_iff` identifies the complete admissible Horn union with Lean's `SequentialStepWithTopLevel`. Here "admissible" adds only the post-allocation well-formedness certificates already stored in Lean's class and object-literal commit constructors; the raw relations remain the literal reading of the operational rules above. The actor continuation is complete as well. Its Horn clauses cover all six remote-representation cases, componentwise message transfer, target selection, all routing and promise-settlement branches, Rule Async-Return, turn entry and lifted sequential execution, local and remote fulfillment and breakage, settlement and wake dequeue, and eligible network delivery. Explicit unions assemble these clauses into a selected-actor step, a scheduler-labeled action, and the complete existential actor-system step. The theorem `prologActorSystemStep_iff` in `Newspeak.PrologActorCorrespondence` identifies that final relation with Lean's `ActorSystemStep`. Message materialization, value transfer, local eventual-handle interpretation, and promise-handle installation remain proof-carrying applications of platform contracts; the Horn layer does not invent implementations for them. The named reflection rules are now connected too: authorized inspection, attenuated mirror derivation, atomic commit or identity-preserving rejection, installation into one selected VM cohort, and capability-checked host primitives. The companion Lean relation names are `MirrorDerivationStep`, `ReflectionStep`, `VMReflectionStep`, and `VMPrimitiveInvocation`; their Prolog counterparts are proved bidirectionally equivalent in `Newspeak.PrologReflectionCorrespondence`. The source-command fold, re-elaboration request, runtime preparation, validation, rejection diagnostics, and versioned commit are represented by separate checked relations. Where the mathematical presentation deliberately specifies an abstract service or where Lean is the canonical executable definition, the Horn clause exposes a proof-carrying graph application rather than silently reimplementing that service. Finally, the optional collection wrapper is represented by `GCLiftStep` and `GCCollectStep`. Their correspondence theorems cover the never-reuse identity-history check, monotone history extension, and independently enabled atomic pruning. Supporting Horn equations also expose weak-container validity, zero-based WeakArray and WeakMap operations, strong and conditional successors, staged reachability, weak-payload clearing, and heap/store pruning. The underlying transition and the highest-level canonical Lean computations remain modular proof-carrying graph applications. Thus every named natural-deduction rule in this document now has a checked Horn clause and a bidirectional Lean correspondence; the remaining coverage ledger obligations are supporting equations. The bridge also crosses the static source boundary. Separate Horn relations represent complete PEG recognition, concrete-tree projection, the Newspeak semantic-action map, stable source-node identification, the surface-validity gate, Program derivation, and annotated-core checking. Their composition is proved bidirectionally equivalent to `FrontEndDerivation` by `prologFrontEndDerivation_iff`, and the corresponding Horn derivation is deterministic. The grammar, lexical decoders, and Program-derivation services remain explicit parameters, so this correspondence does not freeze reflectively replaceable language services into the metalanguage. At the lower boundary this is a structural correspondence, not merely an opaque parser certificate:

the Horn program gives a clause for every atomic and composite PEG-recognition case and mutually recursive clauses for concrete-tree and forest projection. Lean proves these clauses sound and complete for `PEGRecognizes`, `projectConcreteTree`, and `projectConcreteForest`; complete parse and single-root AST projection are compositions of those results. Stable source-node identification is expanded structurally as well. The Horn clauses enumerate the compilation unit's source nodes, select a retained identity when available and a sort-specific fresh identity otherwise, recurse through the node list, validate the identified list, and extend the retention map in source order. Lean proves the recursive identification and retention-extension relations sound and complete before composing them into `prologSurfaceIdentification_iff`; identification is therefore not an opaque front-end premise. The successful static-elaboration branches are source-visible as well: expressions and expression lists, receiver-specialized sends, clause lists, patterns and pattern pairs, and class/object superclass selection have separate Horn clauses with explicit fuel, binding, receiver-kind, and retained identity premises. Lean graph theorems in the static-elaboration correspondence module identify each relation with the corresponding executable elaboration function. The next declaration layer is explicit too. Statement elaboration, return admissibility, the five successful ordinary and lazy local-declaration forms, declaration-list recursion, and sequential or simultaneous local groups have structural soundness and completeness proofs in `Newspeak.PrologDeclarationCorrespondence`. The same module fixes the Horn graphs for formal-parameter identities, physical-slot collection, eager slot initializers, and optional initializer groups to their executable Lean definitions. Access defaulting and member installation are exposed next: explicit methods, ordinary slots, lazy slots, and deferred nested classes have distinct Horn branches, followed by a recursive member-list fold. Explicit methods expose their source and synthesized identities, parameter and local identities, flattened declarations, activation context, elaborated locals and body, method definition, image insertion, and activation-member update. The outer construction layer is connected by `Newspeak.PrologProgramDerivationCorrespondence`. Separate Horn predicates expose class and object header projections, superclass syntax, expression, and message selection, lexical object ownership, recursive anonymous mixin-application folding, bodyless and bodied mixin chains, structural-artifact completion, and final image installation. The final four-way declaration-shape relation (bodyless mixin, bodied mixin, default inheritance, or explicit inheritance) is proved equivalent to `deriveCompleteClassProgram` and deterministic. Finally, `prologCompleteClassProgram_services_iff` identifies that relation with the exact `deriveProgram` field used by the complete front-end service value. The checked bridge also exposes the complete grammar by production groups, all semantic-action groups, auxiliary indexes, metadata attachment, compilation-unit projection, annotated-core validity, and reflective front-end replacement. Thus no numbered front-end equation remains unaccounted for; the full executable grammar/action tables remain canonical in Lean and are referenced by checked graph relations rather than duplicated. In Lean, the lookup, actor, and reflection correspondence modules give the relational clause schemas an inductive reading and prove soundness and completeness with respect to the named Lean rules. Composing these theorems with the existing named-rule/executable equivalences gives, for example,

$$\mathsf{PrologPublicLookup}(P, H, s, C, r) \quad \Longleftrightarrow \quad P.\mathsf{Lookup}(H, \mathsf{publicOnly}, s, C, r).$$

The DNU correspondence includes unrestricted selection of doesNotUnderstand:, allocation and installation of the fresh message mirror, and formation of the final invocation. The Prolog layer is therefore an auditable intermediate formalization, not a replacement proof assistant and not primarily an optimized interpreter. Extensions must retain the same two obligations: visually direct correspondence to the labeled clauses here, and checked bidirectional correspondence to Lean.

Execution harnesses, deterministic allocation policies, and scheduler strategies remain separate from this proof-facing source.

Remaining implementation verification.

Equation (11.67) is proved generically from a mandatory compiler/Simulator admission certificate. The currently shipped V5 compiler and Simulator must still be verified against that boundary: concrete bytecode artifacts, PCs, operand stacks, and frame objects must supply the certificate and representation relation. A replacement implementation has the same obligation. This is not a missing rule of the language semantics; it is the implementation-refinement project that instantiates the proved abstract theorem. A useful further strengthening, also outside the language definition, is a provenance theorem tracing every dequeued wake packet to its unique earlier promise-settlement event.

References

[Bra26] Gilad Bracha. *Newspeak Programming Language Draft Specification*, 0.109 edition, 2026.

[BvdAB+10] Gilad Bracha, Peter von der Ahé, Vassili Bykov, Yaron Kashai, William Maddox, and Eliot Miranda. Modules as objects in Newspeak. In *European Conference on Object-Oriented Programming*, June 2010.